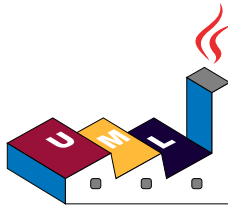


Drawing UML with PlantUML



PlantUML Language Reference Guide

(Version 1.2025.0)

PlantUML is a component that allows to quickly write :

- Sequence diagram
- Usecase diagram
- Class diagram
- Object diagram
- Activity diagram
- Component diagram
- Deployment diagram
- State diagram
- Timing diagram

The following non-UML diagrams are also supported:

- JSON Data
- YAML Data
- Network diagram (nwdiag)
- Wireframe graphical interface
- Archimate diagram
- Specification and Description Language (SDL)
- Dita diagram
- Gantt diagram
- MindMap diagram
- Work Breakdown Structure diagram
- Mathematic with AsciiMath or JLaTeXMath notation
- Entity Relationship diagram

Diagrams are defined using a simple and intuitive language.

1 Sequence Diagram

Creating sequence diagrams with PlantUML is remarkably straightforward. This ease of use is largely attributed to the user-friendly nature of its syntax, designed to be both intuitive and easy to remember.

- **Intuitive Syntax:**

First and foremost, users appreciate the straightforward and intuitive syntax that PlantUML employs. This well-thought-out design means that even those new to diagram creation find it easy to grasp the basics quickly and without hassle.

- **Text-to-Graphic Correlation:**

Another distinguishing feature is the close resemblance between the textual representation and the graphical output. This harmonious correlation ensures that the textual drafts translate quite accurately into graphical diagrams, providing a cohesive and predictable design experience without unpleasant surprises in the final output.

- **Efficient Crafting Process:**

The strong correlation between the text and the graphical result not only simplifies the crafting process but also significantly speeds it up. Users benefit from a more streamlined process with fewer requirements for time-consuming revisions and adjustments.

- **Visualization While Drafting:**

The ability to envisage the final graphical outcome while drafting the text is a feature that many find invaluable. It naturally fosters a smooth transition from initial draft to final presentation, enhancing productivity and reducing the likelihood of errors.

- **Easy Edits and Revisions:**

Importantly, editing existing diagrams is a hassle-free process. Since the diagrams are generated from text, users find that making adjustments is considerably easier and more precise than altering an image using graphical tools. It boils down to simply modifying the text, a process far more straightforward and less prone to errors than making changes through a graphical interface with a mouse.

PlantUML facilitates a straightforward and user-friendly approach to creating and editing sequence diagrams, meeting the needs of both novices and seasoned designers alike. It skillfully leverages the simplicity of textual inputs to craft visually descriptive and accurate diagrams, thereby establishing itself as a must-have tool in the diagram creation toolkit.

You can learn more about some of the common commands in PlantUML to enhance your diagram creation experience.

1.1 Basic Examples

In PlantUML sequence diagrams, the `->` sequence denotes a message sent between two participants, which are automatically recognized and do not need to be declared beforehand.

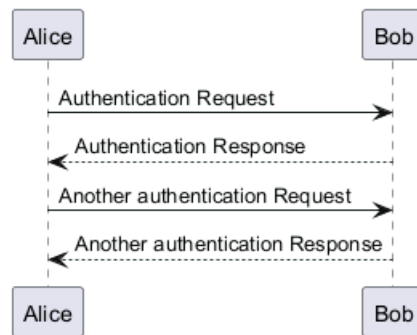
Utilize dotted arrows by employing the `-->` sequence, offering a distinct visualization in your diagrams.

To improve readability without affecting the visual representation, use reverse arrows like `<-` or `<--`. However, be aware that this is specifically for sequence diagrams and the rules differ for other diagram types.

```
@startuml
Alice -> Bob: Authentication Request
Bob --> Alice: Authentication Response

Alice -> Bob: Another authentication Request
Alice <-- Bob: Another authentication Response
@enduml
```





1.2 Declaring participant

If the keyword `participant` is used to declare a participant, more control on that participant is possible.

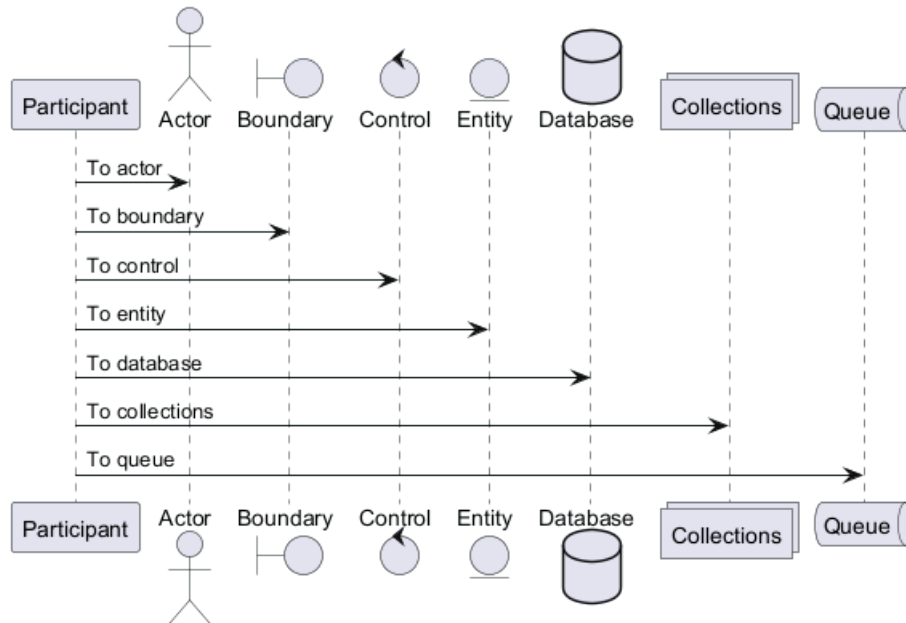
The order of declaration will be the (default) **order of display**.

Using these other keywords to declare participants will **change the shape** of the participant representation:

- actor
- boundary
- control
- entity
- database
- collections
- queue

```

@startuml
participant Participant as Foo
actor Actor as Foo1
boundary Boundary as Foo2
control Control as Foo3
entity Entity as Foo4
database Database as Foo5
collections Collections as Foo6
queue Queue as Foo7
Foo -> Foo1 : To actor
Foo -> Foo2 : To boundary
Foo -> Foo3 : To control
Foo -> Foo4 : To entity
Foo -> Foo5 : To database
Foo -> Foo6 : To collections
Foo -> Foo7: To queue
@enduml
  
```

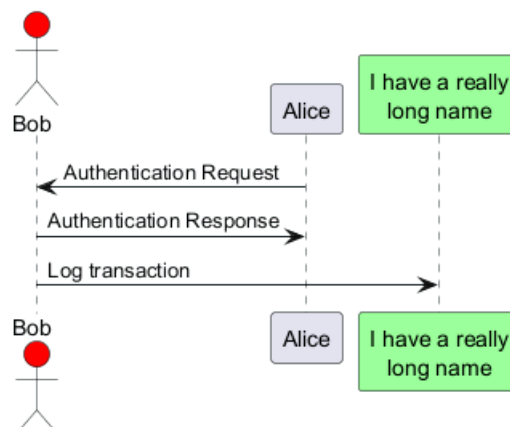


Rename a participant using the **as** keyword.

You can also change the background color of actor or participant.

```
@startuml
actor Bob #red
' The only difference between actor
'and participant is the drawing
participant Alice
participant "I have a really\nlong name" as L #99FF99
/' You can also declare:
    participant L as "I have a really\nlong name" #99FF99
  '/
```

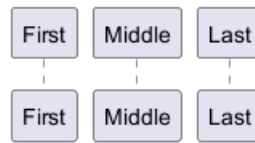
```
Alice->>Bob: Authentication Request
Bob->>Alice: Authentication Response
Bob->>L: Log transaction
@enduml
```



You can use the **order** keyword to customize the display order of participants.

```
@startuml
participant Last order 30
participant Middle order 20
participant First order 10
```

```
@enduml
```



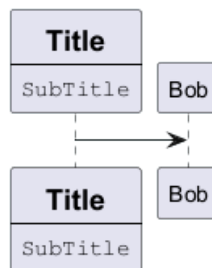
1.3 Declaring participant on multiline

You can declare participant on multi-line.

```
@startuml
participant Participant [
    =Title
    ----
    ""SubTitle""
]
```

```
participant Bob
```

```
Participant -> Bob
@enduml
```

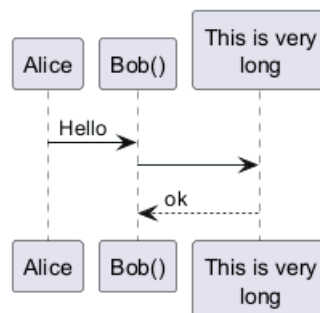


[Ref. QA-15232]

1.4 Use non-letters in participants

You can use quotes to define participants. And you can use the **as** keyword to give an alias to those participants.

```
@startuml
Alice -> "Bob()" : Hello
"Bob()" -> "This is very\nlong" as Long
' You can also declare:
' "Bob()" -> Long as "This is very\nlong"
Long --> "Bob()" : ok
@enduml
```

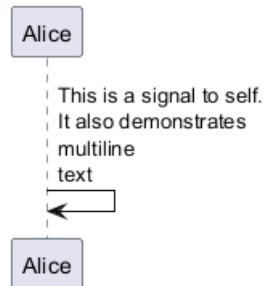


1.5 Message to Self

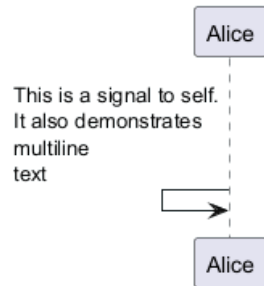
A participant can send a message to itself.

It is also possible to have multi-line using `\n`.

```
@startuml
Alice -> Alice: This is a signal to self.\nIt also demonstrates\nmultiline \ntext
@enduml
```



```
@startuml
Alice <- Alice: This is a signal to self.\nIt also demonstrates\nmultiline \ntext
@enduml
```



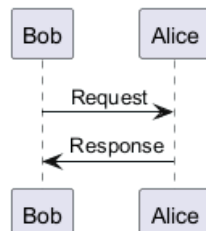
[Ref. QA-1361]

1.6 Text alignment

Text alignment on arrows can be set to `left`, `right` or `center` using `skinparam sequenceMessageAlign`.

You can also use `direction` or `reverseDirection` to align text depending on arrow direction. Further details and examples of this are available on the `skinparam` page.

```
@startuml
skinparam sequenceMessageAlign right
Bob -> Alice : Request
Alice -> Bob : Response
@enduml
```



1.6.1 Text of response message below the arrow

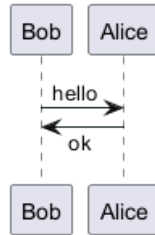
You can put the text of the response message below the arrow, with the `skinparam responseMessageBelowArrow true` command.



```

@startuml
skinparam responseMessageBelowArrow true
Bob -> Alice : hello
Bob <- Alice : ok
@enduml

```



1.7 Change arrow style

You can change arrow style by several ways:

- add a final x to denote a lost message
- use \ or / instead of < or > to have only the bottom or top part of the arrow
- repeat the arrow head (for example, >> or //) head to have a thin drawing
- use -- instead of - to have a dotted arrow
- add a final "o" at arrow head
- use bidirectional arrow <->

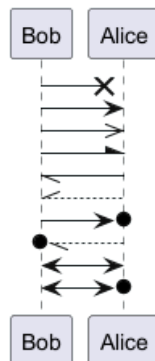
```

@startuml
Bob ->x Alice
Bob -> Alice
Bob ->> Alice
Bob -\ Alice
Bob \- Alice
Bob \\- Alice
Bob //-- Alice

Bob ->o Alice
Bob o\\-- Alice

Bob <-> Alice
Bob <->o Alice
@enduml

```



1.8 Change arrow color

You can change the color of individual arrows using the following notation:

```

@startuml

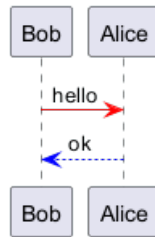
```



```

Bob -[#red]> Alice : hello
Alice -[#0000FF]->Bob : ok
@enduml

```



1.9 Message sequence numbering

The keyword `autonumber` is used to automatically add an incrementing number to messages.

```

@startuml
autonumber
Bob -> Alice : Authentication Request
Bob <- Alice : Authentication Response
@enduml

```



You can specify a startnumber with `autonumber <start>` , and also an increment with `autonumber <start> <increment>`.

```

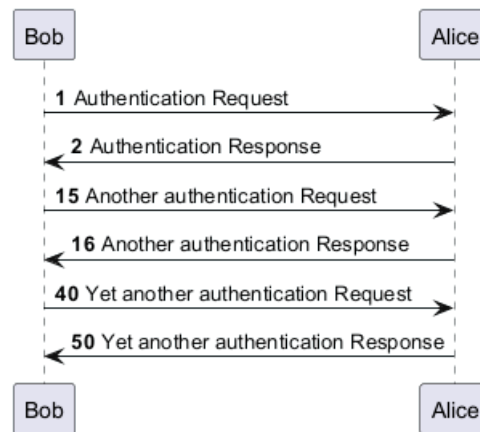
@startuml
autonumber
Bob -> Alice : Authentication Request
Bob <- Alice : Authentication Response

autonumber 15
Bob -> Alice : Another authentication Request
Bob <- Alice : Another authentication Response

autonumber 40 10
Bob -> Alice : Yet another authentication Request
Bob <- Alice : Yet another authentication Response

@enduml

```

You can specify a format for your number by using between double-quote.

The formatting is done with the Java class `DecimalFormat` (0 means digit, # means digit and zero if absent).

You can use some html tag in the format.

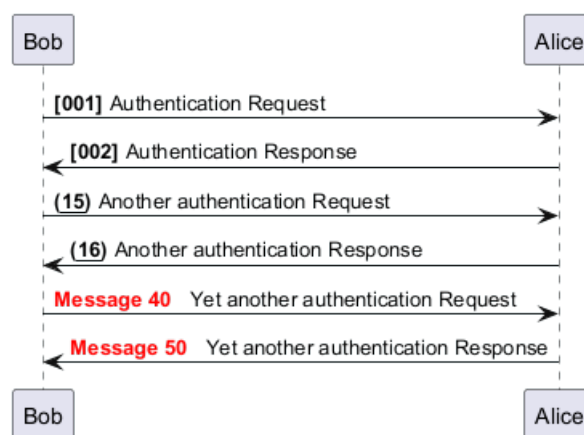
```

@startuml
autonumber "<b>[000]"
Bob -> Alice : Authentication Request
Bob <- Alice : Authentication Response

autonumber 15 "<b>(<u>##</u>)"
Bob -> Alice : Another authentication Request
Bob <- Alice : Another authentication Response

autonumber 40 10 "<font color=red><b>Message 0  "
Bob -> Alice : Yet another authentication Request
Bob <- Alice : Yet another authentication Response

@enduml
  
```



You can also use `autonumber stop` and `autonumber resume <increment> <format>` to respectively pause and resume automatic numbering.

```

@startuml
autonumber 10 10 "<b>[000]"
Bob -> Alice : Authentication Request
Bob <- Alice : Authentication Response

autonumber stop
Bob -> Alice : dummy
  
```



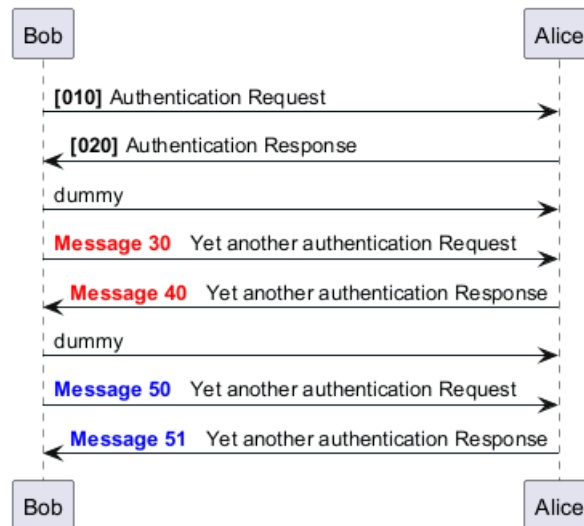
```

autonumber resume "<font color=red><b>Message 0  "
Bob -> Alice : Yet another authentication Request
Bob <- Alice : Yet another authentication Response

autonumber stop
Bob -> Alice : dummy

autonumber resume 1 "<font color=blue><b>Message 0  "
Bob -> Alice : Yet another authentication Request
Bob <- Alice : Yet another authentication Response
@enduml

```



Your startnumber can also be a 2 or 3 digit sequence using a field delimiter such as ., ;, ,, : or a mix of these. For example: 1.1.1 or 1.1:1.

Automatically the last digit will increment.

To increment the first digit, use: `autonumber inc A`. To increment the second digit, use: `autonumber inc B`.

```

@startuml
autonumber 1.1.1
Alice -> Bob: Authentication request
Bob --> Alice: Response

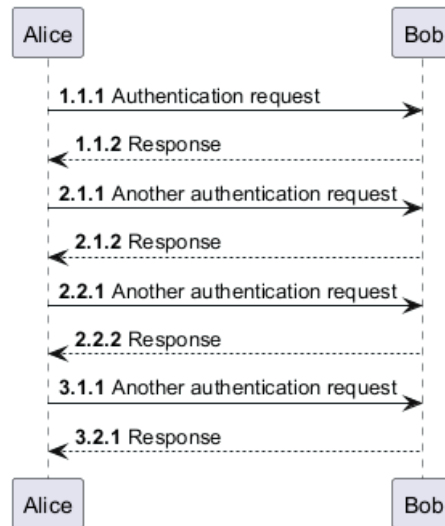
autonumber inc A
'Now we have 2.1.1
Alice -> Bob: Another authentication request
Bob --> Alice: Response

autonumber inc B
'Now we have 2.2.1
Alice -> Bob: Another authentication request
Bob --> Alice: Response

autonumber inc A
'Now we have 3.1.1
Alice -> Bob: Another authentication request
autonumber inc B
'Now we have 3.2.1
Bob --> Alice: Response
@enduml

```

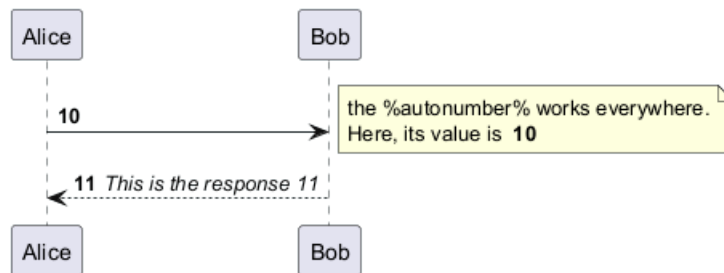




You can also use the value of autonumber with the `%autonumber%` variable:

```

@startuml
autonumber 10
Alice -> Bob
note right
    the <U+0025>autonumber<U+0025> works everywhere.
    Here, its value is ** %autonumber% **
end note
Bob --> Alice: //This is the response %autonumber%//
@enduml
  
```



[Ref. QA-7119]

1.10 Page Title, Header and Footer

The `title` keyword is used to add a title to the page.

Pages can display headers and footers using `header` and `footer`.

```

@startuml

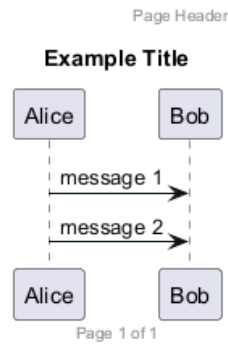
header Page Header
footer Page %page% of %lastpage%

title Example Title

Alice -> Bob : message 1
Alice -> Bob : message 2

@enduml
  
```





1.11 Splitting diagrams

The `newpage` keyword is used to split a diagram into several images.

You can put a title for the new page just after the `newpage` keyword. This title overrides the previously specified title if any.

This is very handy with *Word* to print long diagram on several pages.

(Note: this really does work. Only the first page is shown below, but it is a display artifact.)

```
@startuml
```

```
Alice -> Bob : message 1
```

```
Alice -> Bob : message 2
```

```
newpage
```

```
Alice -> Bob : message 3
```

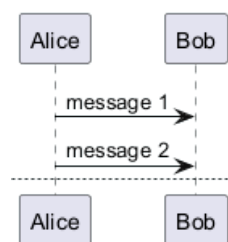
```
Alice -> Bob : message 4
```

```
newpage A title for the\last page
```

```
Alice -> Bob : message 5
```

```
Alice -> Bob : message 6
```

```
@enduml
```



1.12 Grouping message

It is possible to group messages together using the following keywords:

- `alt/else`
- `opt`
- `loop`
- `par`
- `break`
- `critical`



- **group**, followed by a text to be displayed

It is possible to add a text that will be displayed into the header (for **group**, see next paragraph 'Secondary group label').

The **end** keyword is used to close the group.

Note that it is possible to nest groups.

```
@startuml
Alice -> Bob: Authentication Request

alt successful case

    Bob -> Alice: Authentication Accepted

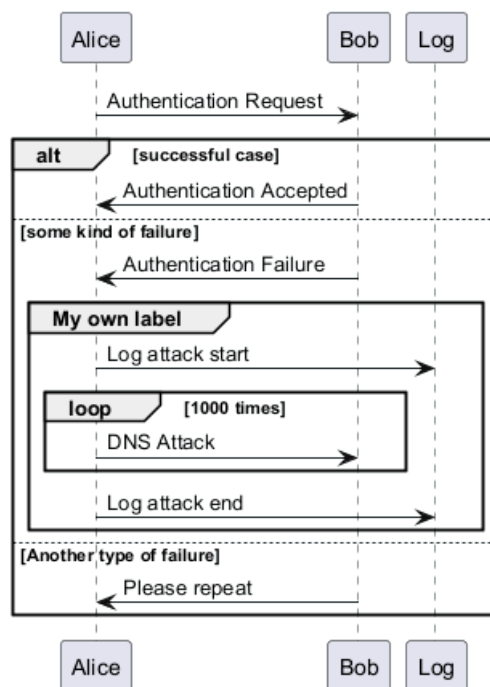
else some kind of failure

    Bob -> Alice: Authentication Failure
    group My own label
    Alice -> Log : Log attack start
        loop 1000 times
            Alice -> Bob: DNS Attack
        end
    Alice -> Log : Log attack end
end

else Another type of failure

    Bob -> Alice: Please repeat

end
@enduml
```



1.13 Secondary group label

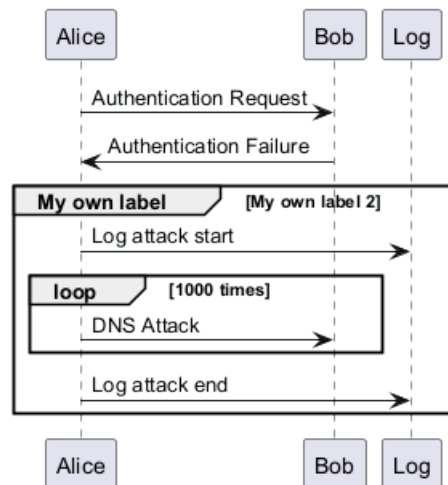
For **group**, it is possible to add, between [and], a secondary text or label that will be displayed into the header.



```

@startuml
Alice -> Bob: Authentication Request
Bob -> Alice: Authentication Failure
group My own label [My own label 2]
    Alice -> Log : Log attack start
    loop 1000 times
        Alice -> Bob: DNS Attack
    end
    Alice -> Log : Log attack end
end
@enduml

```



[Ref. QA-2503]

1.14 Notes on messages

It is possible to put notes on message using the `note left` or `note right` keywords *just after the message*.

You can have a multi-line note using the `end note` keywords.

```

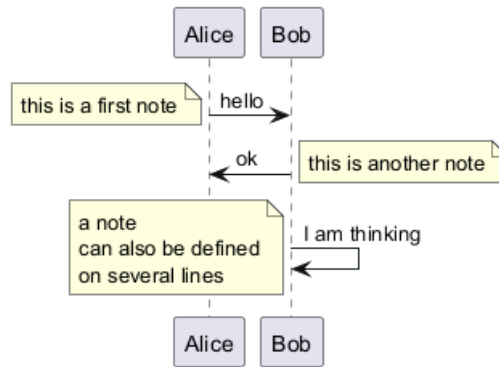
@startuml
Alice->Bob : hello
note left: this is a first note

Bob->Alice : ok
note right: this is another note

Bob->Bob : I am thinking
note left
a note
can also be defined
on several lines
end note
@enduml

```





1.15 Some other notes

It is also possible to place notes relative to participant with `note left of`, `note right of` or `note over` keywords.

It is possible to highlight a note by changing its background color.

You can also have a multi-line note using the `end note` keywords.

```

@startuml
participant Alice
participant Bob
note left of Alice #aqua
This is displayed
left of Alice.
end note

```

```

note right of Alice: This is displayed right of Alice.

```

```

note over Alice: This is displayed over Alice.

```

```

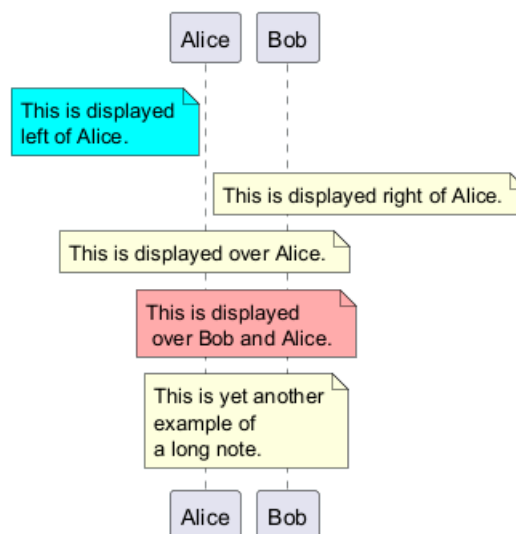
note over Alice, Bob #FFAAAA: This is displayed\n over Bob and Alice.

```

```

note over Bob, Alice
This is yet another
example of
a long note.
end note
@enduml

```

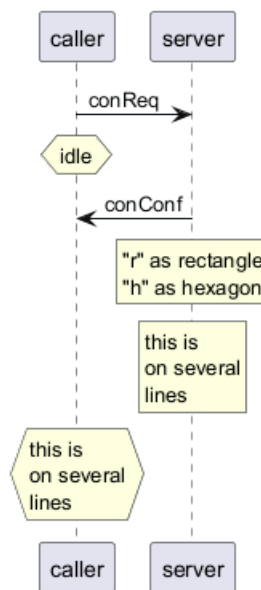


1.16 Changing notes shape [hnote, rnote]

You can use `hnote` and `rnote` keywords to change note shapes :

- `hnote` for hexagonal note;
- `rnote` for rectangle note.

```
@startuml
caller -> server : conReq
hnote over caller : idle
caller <- server : conConf
rnote over server
  "r" as rectangle
  "h" as hexagon
endrnote
rnote over server
  this is
  on several
  lines
endrnote
hnote over caller
  this is
  on several
  lines
endhnote
@enduml
```



[Ref. QA-1765]

1.17 Note over all participants [across]

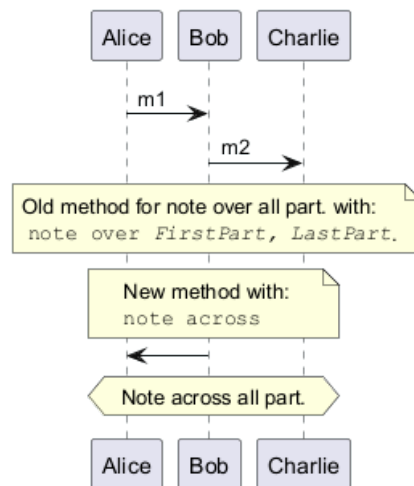
You can directly make a note over all participants, with the syntax:

- `note across: note_description`

```
@startuml
Alice->>Bob:m1
Bob->>Charlie:m2
note over Alice, Charlie: Old method for note over all part. with:\n ""note over //FirstPart, LastPart
note across: New method with:\n""note across""
Bob->>Alice
hnote across:Note across all part.
```




```
@enduml
```



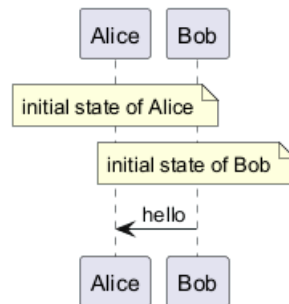
[Ref. QA-9738]

1.18 Several notes aligned at the same level [/]

You can make several notes aligned at the same level, with the syntax /:

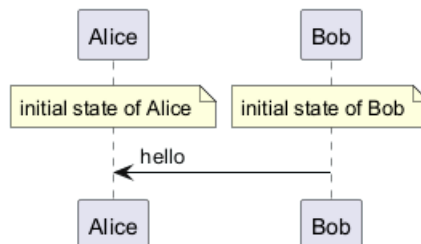
- without / (by default, the notes are not aligned)

```
@startuml
note over Alice : initial state of Alice
note over Bob : initial state of Bob
Bob -> Alice : hello
@enduml
```



- with / (the notes are aligned)

```
@startuml
note over Alice : initial state of Alice
/ note over Bob : initial state of Bob
Bob -> Alice : hello
@enduml
```



[Ref. QA-354]



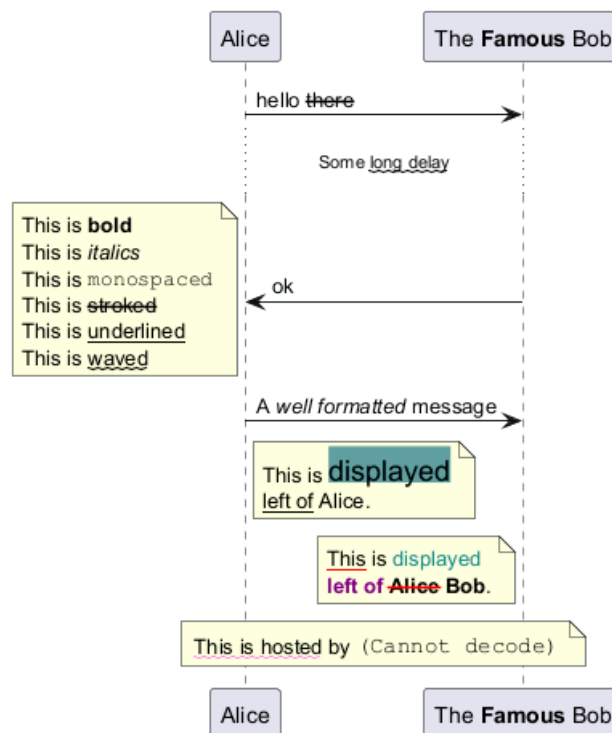
1.19 Creole and HTML

It is also possible to use creole formatting:

```
@startuml
participant Alice
participant "The **Famous** Bob" as Bob

Alice -> Bob : hello --there--
... Some ~~long delay~~ ...
Bob -> Alice : ok
note left
  This is **bold**
  This is //italics//
  This is "monospaced"
  This is --stoked--
  This is __underlined__
  This is ~~waved~~
end note

Alice -> Bob : A //well formatted// message
note right of Alice
  This is displayed
  __left of__ Alice.
end note
note left of Bob
  <u:red>This</u> is <color #118888>displayed</color>
  <color purple>left of</color> <s:red>Alice</strike> Bob.
end note
note over Alice, Bob
  <w:#FF33FF>This is hosted</w> by 
end note
@enduml
```



1.20 Divider or separator

If you want, you can split a diagram using == separator to divide your diagram into logical steps.

```
@startuml

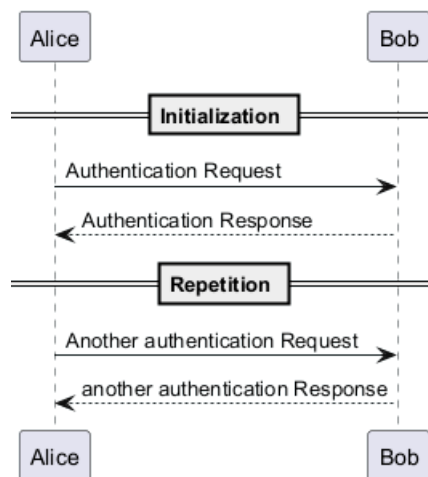
== Initialization ==

Alice -> Bob: Authentication Request
Bob --> Alice: Authentication Response

== Repetition ==

Alice -> Bob: Another authentication Request
Alice <-- Bob: another authentication Response

@enduml
```



1.21 Reference

You can use reference in a diagram, using the keyword **ref over**.

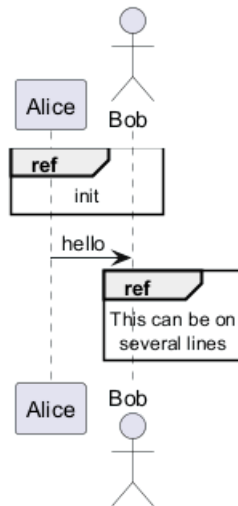
```
@startuml
participant Alice
actor Bob

ref over Alice, Bob : init

Alice -> Bob : hello

ref over Bob
    This can be on
    several lines
end ref
@enduml
```





1.22 Delay

You can use ... to indicate a delay in the diagram. And it is also possible to put a message with this delay.

```
@startuml
```

```
Alice -> Bob: Authentication Request
```

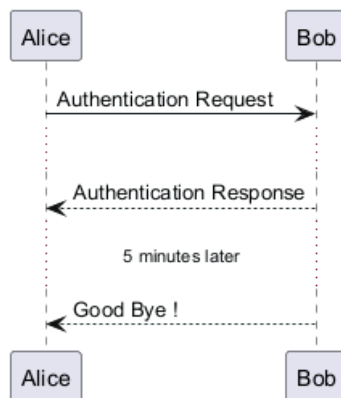
```
...
```

```
Bob --> Alice: Authentication Response
```

```
...5 minutes later...
```

```
Bob --> Alice: Good Bye !
```

```
@enduml
```



1.23 Text wrapping

To break long messages, you can manually add \n in your text.

Another option is to use `maxMessageSize` setting:

```
@startuml
```

```
skinparam maxMessageSize 50
```

```
participant a
```

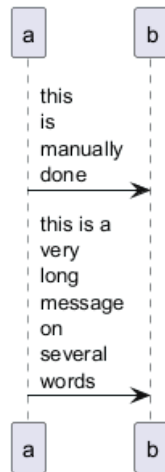
```
participant b
```

```
a -> b :this\nis\nmanually\ndone
```

```
a -> b :this is a very long message on several words
```

```
@enduml
```





1.24 Space

You can use `|||` to indicate some spacing in the diagram.

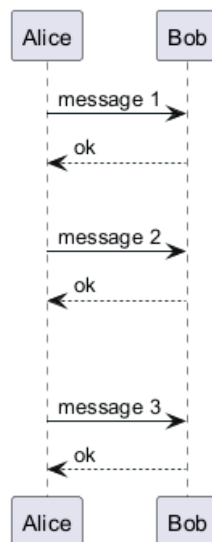
It is also possible to specify a number of pixel to be used.

`@startuml`

```

Alice -> Bob: message 1
Bob --> Alice: ok
|||
Alice -> Bob: message 2
Bob --> Alice: ok
||45||
Alice -> Bob: message 3
Bob --> Alice: ok
  
```

`@enduml`



1.25 Lifeline Activation and Destruction

The `activate` and `deactivate` are used to denote participant activation.

Once a participant is activated, its lifeline appears.

The `activate` and `deactivate` apply on the previous message.



The **destroy** denote the end of the lifeline of a participant.

```

@startuml
participant User

User -> A: DoWork
activate A

A -> B: << createRequest >>
activate B

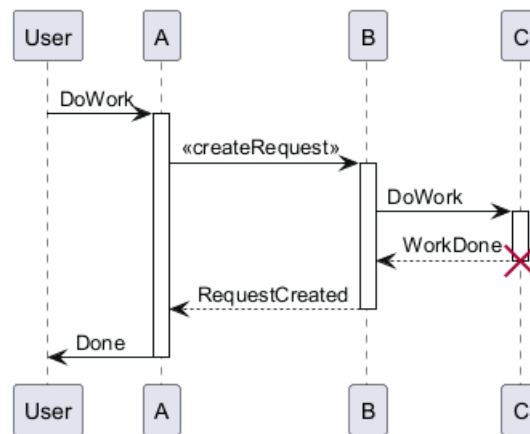
B -> C: DoWork
activate C
C --> B: WorkDone
destroy C

B --> A: RequestCreated
deactivate B

A -> User: Done
deactivate A

@enduml

```



Nested lifeline can be used, and it is possible to add a color on the lifeline.

```

@startuml
participant User

User -> A: DoWork
activate A #FFBBBB

A -> A: Internal call
activate A #DarkSalmon

A -> B: << createRequest >>
activate B

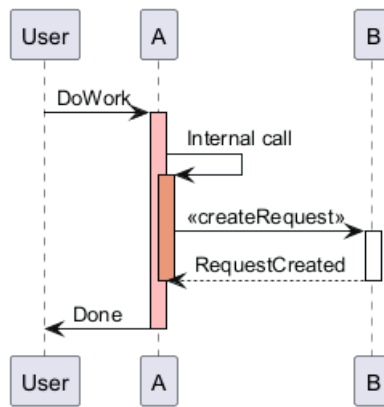
B --> A: RequestCreated
deactivate B
deactivate A

A -> User: Done
deactivate A

@enduml

```

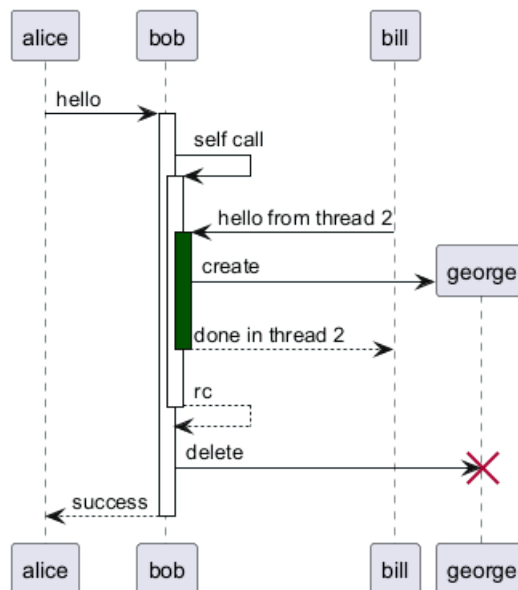




Autoactivation is possible and works with the return keywords:

```

@startuml
autoactivate on
alice -> bob : hello
bob -> bob : self call
bill -> bob #005500 : hello from thread 2
bob -> george ** : create
return done in thread 2
return rc
bob -> george !! : delete
return success
@enduml
  
```



1.26 Return

Command **return** generates a return message with optional text label.

The return point is that which caused the most recent life-line activation.

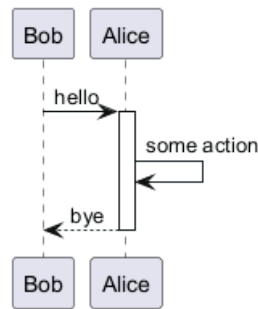
The syntax is **return label** where **label** if provided is any string acceptable for conventional messages.

```

@startuml
Bob -> Alice : hello
activate Alice
Alice -> Alice : some action
  
```



```
return bye
@enduml
```



1.27 Participant creation

You can use the **create** keyword just before the first reception of a message to emphasize the fact that this message is actually *creating* this new object.

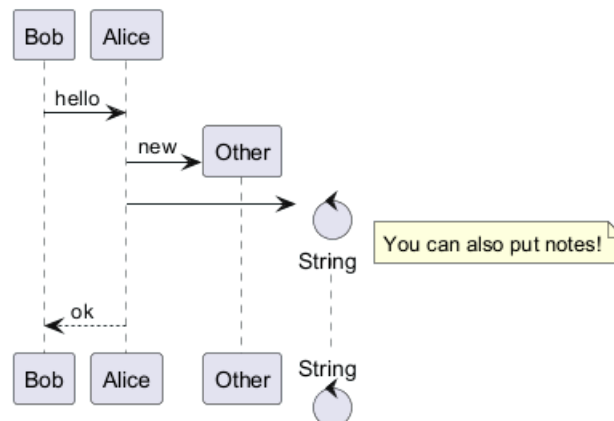
```
@startuml
Bob -> Alice : hello

create Other
Alice -> Other : new

create control String
Alice -> String
note right : You can also put notes!

Alice --> Bob : ok

@enduml
```



1.28 Shortcut syntax for activation, deactivation, creation

Immediately after specifying the target participant, the following syntax can be used:

- ++ Activate the target (optionally a color may follow this)
- -- Deactivate the source
- ** Create an instance of the target
- !! Destroy an instance of the target

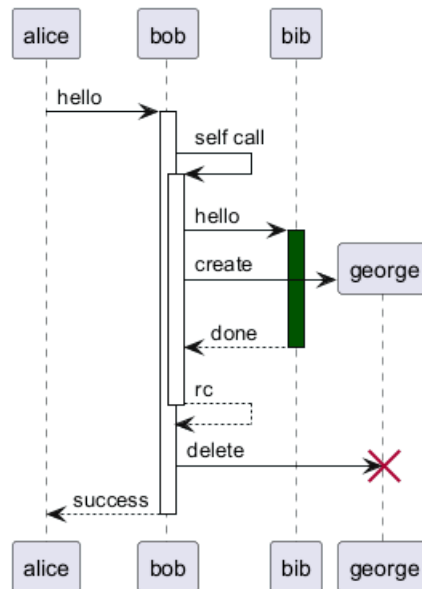
```
@startuml
alice -> bob ++ : hello
```




```

bob -> bob ++ : self call
bob -> bib ++ #005500 : hello
bob -> george ** : create
return done
return rc
bob -> george !! : delete
return success
@enduml

```

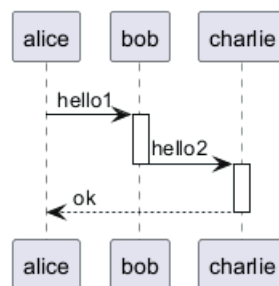


Then you can mix activation and deactivation, on same line:

```

@startuml
alice -> bob ++ : hello1
bob -> charlie ---+ : hello2
charlie --> alice -- : ok
@enduml

```

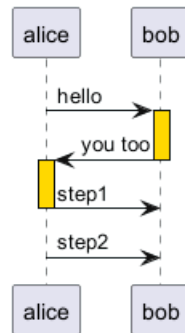


```

@startuml
@startuml
alice -> bob ---+ #gold: hello
bob -> alice ---+ #gold: you too
alice -> bob --: step1
alice -> bob : step2
@enduml
@enduml

```





[Ref. QA-4834, QA-9573 and QA-13234]

1.29 Incoming and outgoing messages

You can use incoming or outgoing arrows if you want to focus on a part of the diagram.

Use square brackets to denote the left "[" or the right "]" side of the diagram.

```

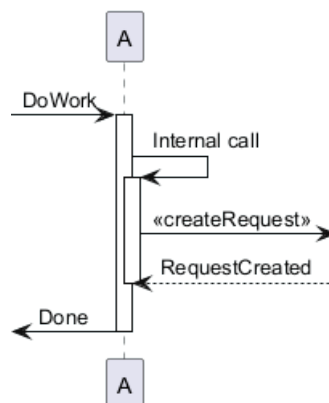
@startuml
[-> A: DoWork

activate A

A -> A: Internal call
activate A

A ->] : << createRequest >>

A<--] : RequestCreated
deactivate A
[<- A: Done
deactivate A
@enduml
  
```



You can also have the following syntax:

```

@startuml
participant Alice
participant Bob #lightblue
Alice -> Bob
Bob -> Carol
...
[-> Bob
[o-> Bob
[o->o Bob
[x-> Bob
  
```

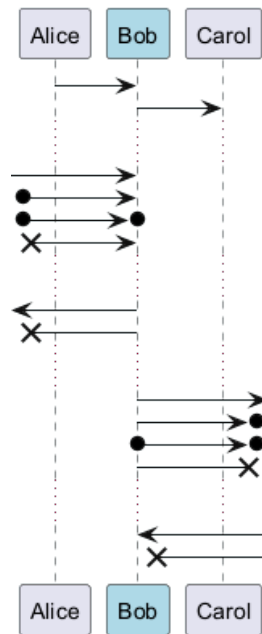


```

...
[<- Bob
[x<- Bob
...
Bob ->]
Bob ->o]
Bob o->o]
Bob ->x]
...
Bob <-]
Bob x<-]

@enduml

```



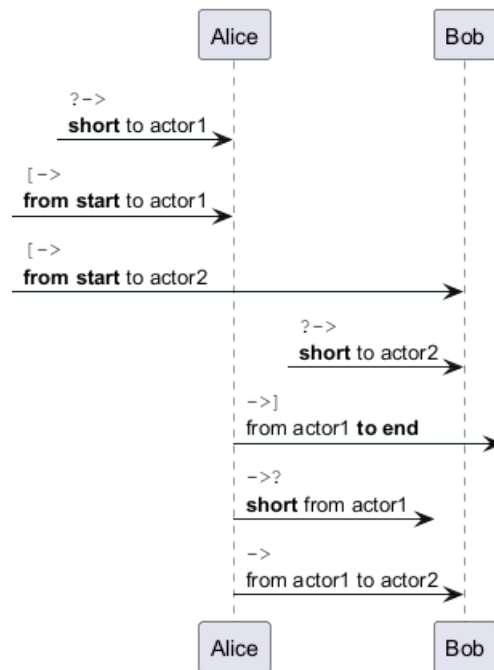
1.30 Short arrows for incoming and outgoing messages

You can have **short** arrows with using ?.

```

@startuml
?-> Alice : ""?->""\n**short** to actor1
[-> Alice : ""[->""\n**from start** to actor1
[-> Bob : ""[->""\n**from start** to actor2
?-> Bob : ""?->""\n**short** to actor2
Alice ->] : ""->""\nfrom actor1 **to end**
Alice ->? : ""->?""\n**short** from actor1
Alice -> Bob : ""->"" \nfrom actor1 to actor2
@enduml

```



[Ref. QA-310]

1.31 Anchors and Duration

With teoz it is possible to add anchors to the diagram and use the anchors to specify duration time.

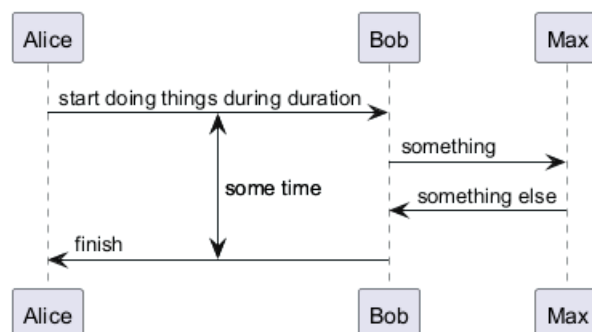
```

@startuml
!pragma teoz true

{start} Alice -> Bob : start doing things during duration
Bob -> Max : something
Max -> Bob : something else
{end} Bob -> Alice : finish

{start} <-> {end} : some time

@enduml
  
```



You can use the -P command-line option to specify the pragma:

```
java -jar plantuml.jar -Pteoz=true
```

[Ref. issue-582]

1.32 Stereotypes and Spots

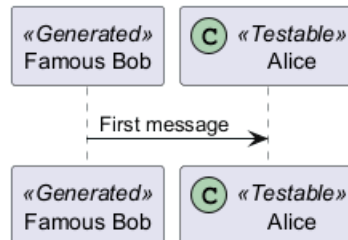
It is possible to add stereotypes to participants using << and >>.



In the stereotype, you can add a spotted character in a colored circle using the syntax (X,color).

```
@startuml
participant "Famous Bob" as Bob << Generated >>
participant Alice << (C,#ADD1B2) Testable >>

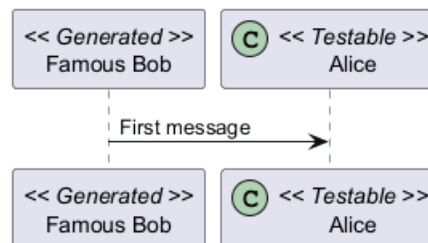
Bob->>Alice: First message
@enduml
```



By default, the *guillemet* character is used to display the stereotype. You can change this behaviour using the skinparam *guillemet*:

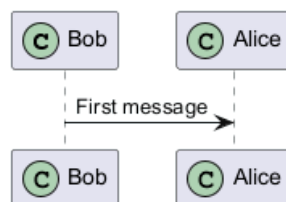
```
@startuml
skinparam guillemet false
participant "Famous Bob" as Bob << Generated >>
participant Alice << (C,#ADD1B2) Testable >>

Bob->>Alice: First message
@enduml
```



```
@startuml
participant Bob << (C,#ADD1B2) >>
participant Alice << (C,#ADD1B2) >>

Bob->>Alice: First message
@enduml
```



1.33 Position of the stereotypes

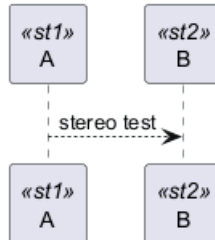
It is possible to define stereotypes position (top or bottom) with the command `skinparam stereotypePosition`.



1.33.1 Top postion (*by default*)

```
@startuml
skinparam stereotypePosition top

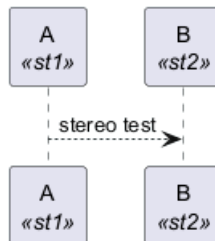
participant A<<st1>>
participant B<<st2>>
A --> B : stereo test
@enduml
```



1.33.2 Bottom postion

```
@startuml
skinparam stereotypePosition bottom

participant A<<st1>>
participant B<<st2>>
A --> B : stereo test
@enduml
```



[Ref. QA-18650]

1.34 More information on titles

You can use creole formatting in the title.

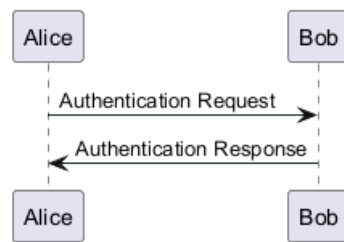
```
@startuml

title __Simple__ **communication** example

Alice -> Bob: Authentication Request
Bob -> Alice: Authentication Response

@enduml
```



Simple communication example

You can add newline using \n in the title description.

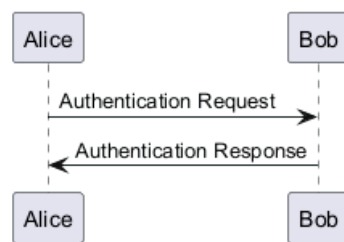
```

@startuml

title __Simple__ communication example\nnon several lines

Alice -> Bob: Authentication Request
Bob -> Alice: Authentication Response

@enduml
  
```

Simple communication example on several lines

You can also define title on several lines using `title` and `end title` keywords.

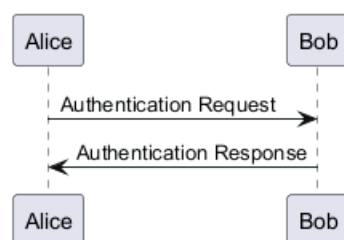
```

@startuml

title
  <u>Simple</u> communication example
  on <i>several</i> lines and using <font color=red>html</font>
  This is hosted by <img:sourceforge.jpg>
end title

Alice -> Bob: Authentication Request
Bob -> Alice: Authentication Response

@enduml
  
```

Simple communication example on several lines and using html
 This is hosted by (Cannot decode)


1.35 Participants encompass

It is possible to draw a box around some participants, using **box** and **end box** commands.

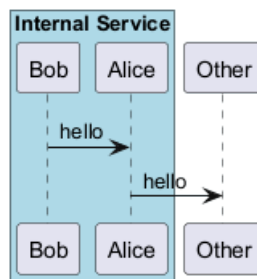
You can add an optional title or a optional background color, after the **box** keyword.

```
@startuml

box "Internal Service" #LightBlue
participant Bob
participant Alice
end box
participant Other

Bob -> Alice : hello
Alice -> Other : hello

@enduml
```



It is also possible to nest boxes - to draw a box within a box - when using the teoz rendering engine, for example:

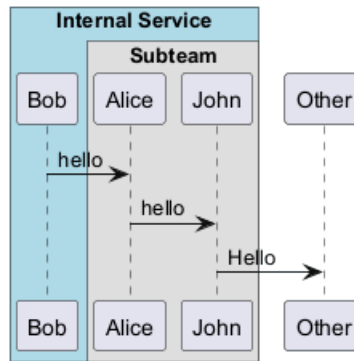
```
@startuml

!pragma teoz true
box "Internal Service" #LightBlue
participant Bob
box "Subteam"
participant Alice
participant John
end box

end box
participant Other

Bob -> Alice : hello
Alice -> John : hello
John -> Other : Hello

@enduml
```

1.36 Removing Foot Boxes

You can use the `hide footbox` keywords to remove the foot boxes of the diagram.

```

@startuml

hide footbox
title Foot Box removed

Alice -> Bob: Authentication Request
Bob --> Alice: Authentication Response

@enduml
  
```



1.37 Skinparam

You can use the `skinparam` command to change colors and fonts for the drawing.

You can use this command:

- In the diagram definition, like any other commands,
- In an included file,
- In a configuration file, provided in the command line or the ANT task.

You can also change other rendering parameter, as seen in the following examples:

```

@startuml
skinparam sequenceArrowThickness 2
skinparam roundcorner 20
skinparam maxmessageSize 60
skinparam sequenceParticipant underline
  
```

```

actor User
participant "First Class" as A
participant "Second Class" as B
participant "Last Class" as C
  
```

```

User -> A: DoWork
activate A
  
```



```

A -> B: Create Request
activate B

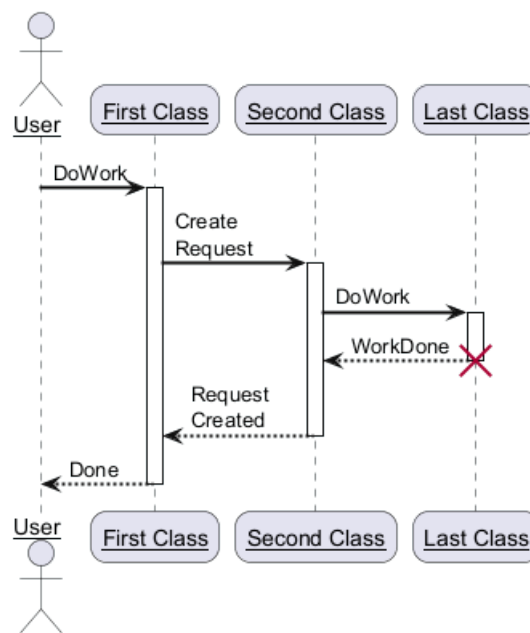
B -> C: DoWork
activate C
C --> B: WorkDone
destroy C

B --> A: Request Created
deactivate B

A --> User: Done
deactivate A

@enduml

```



```

@startuml
skinparam backgroundColor #EEEBDC
skinparam handwritten true

skinparam sequence {
    ArrowColor DeepSkyBlue
    ActorBorderColor DeepSkyBlue
    LifeLineBorderColor blue
    LifeLineBackgroundColor #A9DCDF

    ParticipantBorderColor DeepSkyBlue
    ParticipantBackgroundColor DodgerBlue
    ParticipantFontName Impact
    ParticipantFontSize 17
    ParticipantFontColor #A9DCDF

    ActorBackgroundColor aqua
    ActorFontColor DeepSkyBlue
    ActorFontSize 17
    ActorFontName Aapex
}

```



```

actor User
participant "First Class" as A
participant "Second Class" as B
participant "Last Class" as C

User -> A: DoWork
activate A

A -> B: Create Request
activate B

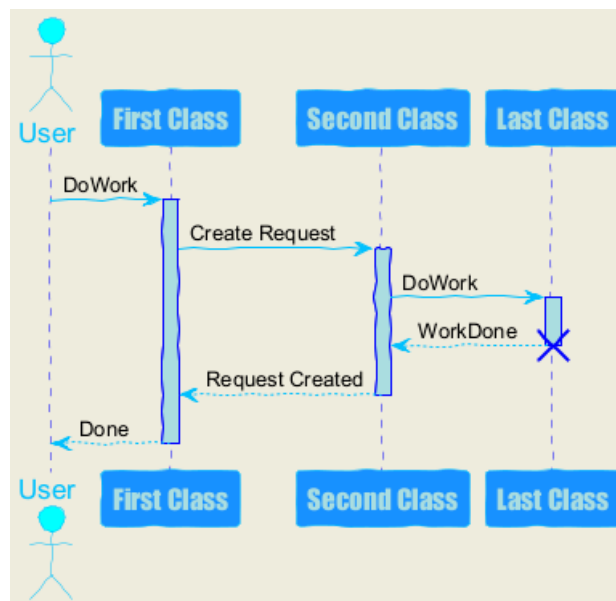
B -> C: DoWork
activate C
C --> B: WorkDone
destroy C

B --> A: Request Created
deactivate B

A --> User: Done
deactivate A

@enduml

```



1.38 Changing padding

It is possible to tune some padding settings.

```

@startuml
skinparam ParticipantPadding 20
skinparam BoxPadding 10

box "Foo1"
    participant Alice1
    participant Alice2
end box

box "Foo2"
    participant Bob1
    participant Bob2
end box

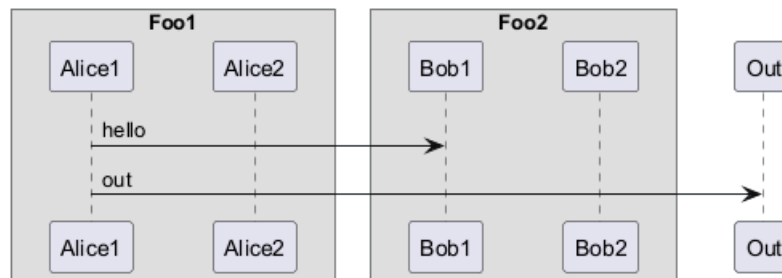
```



```

end box
Alice1 -> Bob1 : hello
Alice1 -> Out : out
@enduml

```



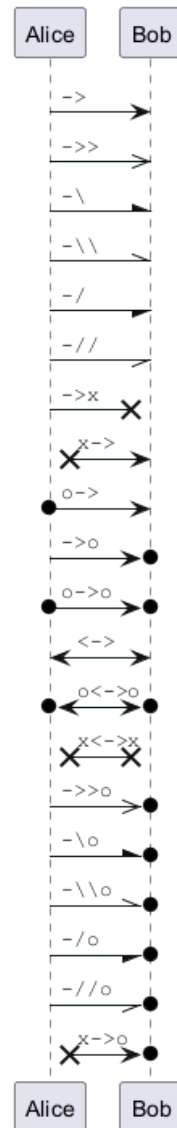
1.39 Appendix: Examples of all arrow type

1.39.1 Normal arrow

```

@startuml
participant Alice as a
participant Bob as b
a -> b : ""-> ""
a ->> b : ""->> ""
a -\ b : ""-\ ""
a -\\ b : ""-\\ ""
a -/ b : ""-/ ""
a -// b : ""-// ""
a ->x b : ""->x ""
a x-> b : ""x-> ""
a o-> b : ""o-> ""
a ->o b : ""->o ""
a o->o b : ""o->o ""
a <-> b : ""<-> ""
a o<->o b : ""o<->o ""
a x<->x b : ""x<->x ""
a ->>o b : ""->>o ""
a -\o b : ""-\o ""
a -\\o b : ""-\\o ""
a -/o b : ""-/o ""
a -//o b : ""-//o ""
a x->o b : ""x->o ""
@enduml

```



1.39.2 Itself arrow

```

@startuml
participant Alice as a
participant Bob as b
a -> a : ""-> ""
a ->> a : ""->> ""
a -\ a : ""-\ ""
a -\\ a : ""-\\ ""
a -/ a : ""-/ ""
a -// a : ""-// ""
a ->x a : ""->x ""
a x-> a : ""x-> ""
a o-> a : ""o-> ""
a ->o a : ""->o ""
a o->o a : ""o->o ""
a <-> a : ""<-> ""
a o<->o a : ""o<->o ""
a x<->x a : ""x<->x ""
a ->>o a : ""->>o ""
a -\o a : ""-\o ""
a -\\o a : ""-\\o ""

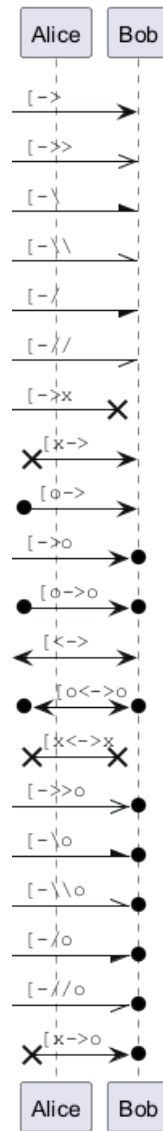
```




```

participant Alice as a
participant Bob as b
[->      b : "" [->  ""
[->>     b : "" [->> ""
[-\      b : "" [-\   ""
[-\\     b : "" [-\\\  ""
[-/      b : "" [-/   ""
[-//     b : "" [-//  ""
[->x     b : "" [->x  ""
[x->     b : "" [x->  ""
[o->     b : "" [o->  ""
[->o     b : "" [->o  ""
[o->o     b : "" [o->o ""
[<->    b : "" [<->  ""
[o<->o   b : "" [o<->o""
[x<->x   b : "" [x<->x""
[->>o    b : "" [->>o ""
[-\o     b : "" [-\o  ""
[-\\o    b : "" [-\\\o""
[-/o     b : "" [-/o  ""
[-//o    b : "" [-//o ""
[x->o     b : "" [x->o ""
@enduml

```



1.39.5 Outgoing messages (with '']')

```

@startuml
participant Alice as a
participant Bob as b
a ->]      : ""->]      ""
a ->>]     : ""->>]     ""
a -\]      : ""-\]      ""
a -\\]     : ""-\\]     ""
a -/]      : ""-/]      ""
a -//]     : ""-//]     ""
a ->x]    : ""->x]    ""
a x->]    : ""x->]    ""
a o->]    : ""o->]    ""
a ->o]    : ""->o]    ""
a o->o]   : ""o->o]   ""
a <->]    : ""<->]    ""
a o<->o]  : ""o<->o]  ""
a x<->x]  : ""x<->x]  ""
a ->>o]   : ""->>o]   ""
a -\o]    : ""-\o]    ""
a -\\o]   : ""-\\o]   ""

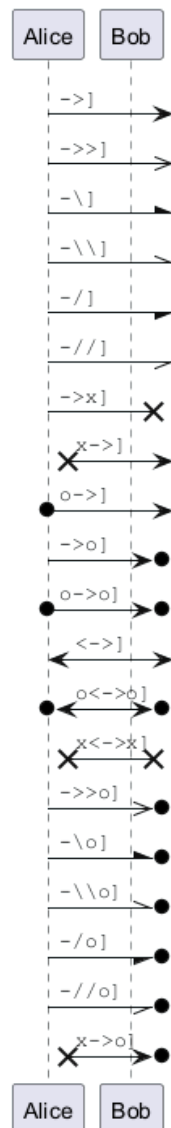
```




```

a -/o]      : ""-/o]  ""
a -//o]     : ""-//o] ""
a x->o]     : ""x->o]  ""
@enduml

```



1.39.6 Short incoming and outgoing messages (with '?')

1.39.7 Short incoming (with '?')

```

@startuml
participant Alice as a
participant Bob as b
a -> b : //Long long label//
?-> b : ""?->  ""
?->> b : ""?->>  ""
?-\ b : ""?-\  ""
?-\\ b : ""?-\\\\  ""
?-/ b : ""?-/  ""
?-// b : ""?-//  ""
?->x b : ""?->x  ""
?x-> b : ""?x->  ""
?o-> b : ""?o->  ""
?->o b : ""?->o  ""

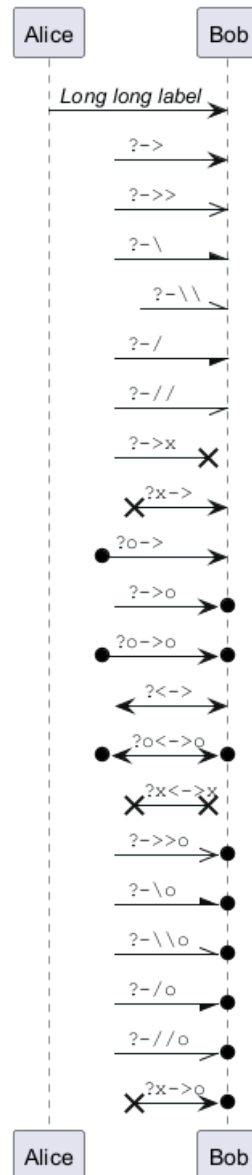
```



```

?o->o    b : ""?o->o ""
?<->     b : ""?<-> ""
?o<->o   b : ""?o<->o""
?x<->x   b : ""?x<->x""
?->>o    b : ""?->>o ""
?-\o     b : ""?-\o ""
?-\o     b : ""?-\o ""
?-\o     b : ""?-\o ""
?-\o     b : ""?-\o ""
?-/o     b : ""?-/o ""
?-/o     b : ""?-/o ""
?x->o    b : ""?x->o ""
@enduml

```



1.39.8 Short outgoing (with '?')

```

@startuml
participant Alice as a
participant Bob as b
a->b : //Long long label//
a->b : ""->? ""
a->>b : ""->>? ""
a-\b : ""-\? ""

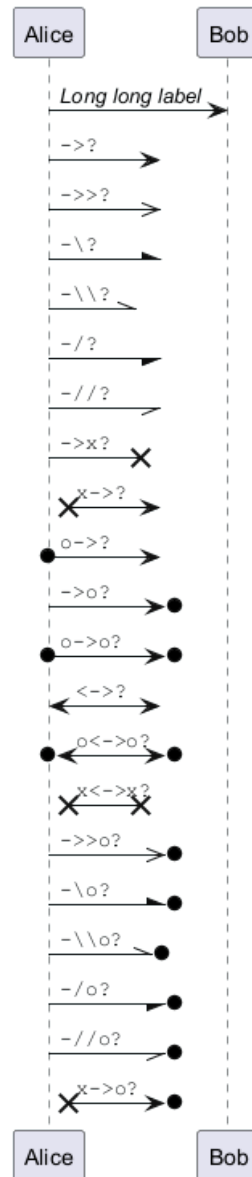
```



```

a -\\?      : "\"-\\\\\\?"
a -/?       : "\"-/?  \""
a -//?      : "\"-//?  \""
a ->x?      : "\"->x?  \""
a x->?      : "\"x->?  \""
a o->?      : "\"o->?  \""
a ->o?      : "\"->o?  \""
a o->o?     : "\"o->o?  \""
a <->?      : "\"<->?  \""
a o<->o?    : "\"o<->o?\""
a x<->x?    : "\"x<->x?\""
a ->>o?     : "\"->>o?  \""
a -\o?      : "\"-\o?  \""
a -\\o?     : "\"-\\\\o?\""
a -/o?      : "\"-/o?  \""
a -//o?     : "\"-//o?  \""
a x->o?     : "\"x->o?  \""
@enduml

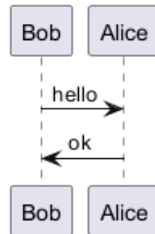
```



1.40 Specific SkinParameter

1.40.1 By default

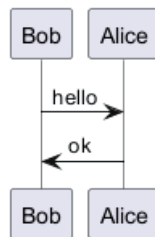
```
@startuml
Bob -> Alice : hello
Alice -> Bob : ok
@enduml
```



1.40.2 LifelineStrategy

- nosolid (*by default*)

```
@startuml
skinparam lifelineStrategy nosolid
Bob -> Alice : hello
Alice -> Bob : ok
@enduml
```

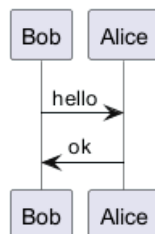


[Ref. QA-9016]

- solid

In order to have solid life line in sequence diagrams, you can use: `skinparam lifelineStrategy solid`

```
@startuml
skinparam lifelineStrategy solid
Bob -> Alice : hello
Alice -> Bob : ok
@enduml
```



[Ref. QA-2794]

1.40.3 style strictuml

To be conform to strict UML (*for arrow style: emits triangle rather than sharp arrowheads*), you can use:

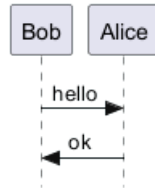
- `skinparam style strictuml`



```

@startuml
skinparam style strictuml
Bob -> Alice : hello
Alice -> Bob : ok
@enduml

```



[Ref. QA-1047]

1.41 Hide unlinked participant

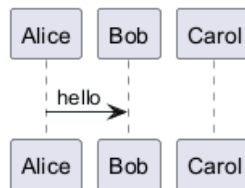
By default, all participants are displayed.

```

@startuml
participant Alice
participant Bob
participant Carol

Alice -> Bob : hello
@enduml

```



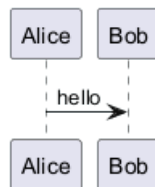
But you can hide unlinked participant.

```

@startuml
hide unlinked
participant Alice
participant Bob
participant Carol

Alice -> Bob : hello
@enduml

```



[Ref. QA-4247]

1.42 Color a group message

It is possible to color a group messages:

```

@startuml
Alice -> Bob: Authentication Request
alt#Gold #LightBlue Successful case
    Bob -> Alice: Authentication Accepted
end

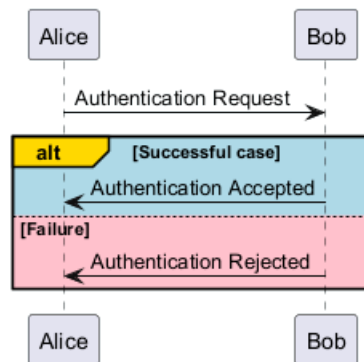
```



```

else #Pink Failure
    Bob -> Alice: Authentication Rejected
end
@enduml

```



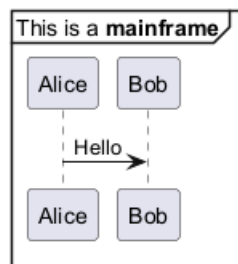
[Ref. QA-4750 and QA-6410]

1.43 Mainframe

```

@startuml
mainframe This is a **mainframe**
Alice->Bob : Hello
@enduml

```



[Ref. QA-4019 and Issue#148]

1.44 Slanted or odd arrows

You can use the (nn) option (before or after arrow) to make the arrows slanted, where *nn* is the number of shift pixels.

[Available only after v1.2022.6beta+]

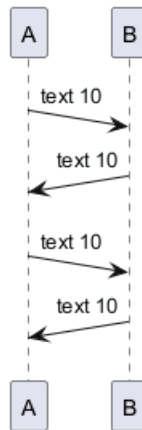
```

@startuml
A ->(10) B: text 10
B ->(10) A: text 10

A ->(10) B: text 10
A (10)<- B: text 10
@enduml

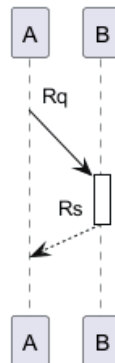
```





```

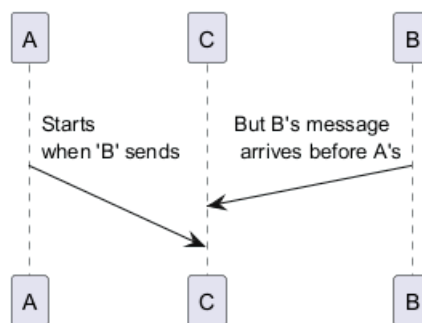
@startuml
A ->(40) B++: Rq
B -->(20) A--: Rs
@enduml
  
```



[Ref. QA-14145]

```

@startuml
!pragma teoz true
A ->(50) C: Starts\nwhen 'B' sends
& B ->(25) C: \nBut B's message\n arrives before A's
@enduml
  
```



[Ref. QA-6684]

```

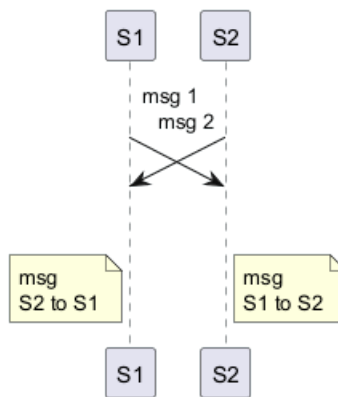
@startuml
!pragma teoz true

S1 ->(30) S2: msg 1\n
& S2 ->(30) S1: msg 2

note left S1: msg\nS2 to S1
& note right S2: msg\nS1 to S2
  
```



```
@enduml
```

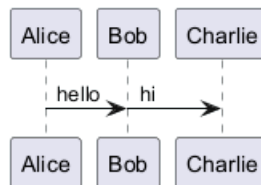


[Ref. QA-1072]

1.45 Parallel messages (*with teoz*)

You can use the `& teoz` command to display parallel messages:

```
@startuml
!pragma teoz true
Alice -> Bob : hello
& Bob -> Charlie : hi
@enduml
```



(See also *Teoz architecture*)

2 Use Case Diagram

A **use case diagram** is a visual representation used in software engineering to depict the interactions between **system actors** and the **system itself**. It captures the dynamic behavior of a system by illustrating its **use cases** and the roles that interact with them. These diagrams are essential in specifying the system's **functional requirements** and understanding how users will interact with the system. By providing a high-level view, use case diagrams help stakeholders understand the system's functionality and its potential value.

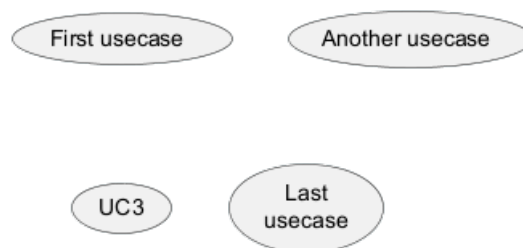
PlantUML offers a unique approach to creating use case diagrams through its text-based language. One of the primary advantages of using PlantUML is its **simplicity and efficiency**. Instead of manually drawing shapes and connections, users can define their diagrams using intuitive and concise textual descriptions. This not only speeds up the diagram creation process but also ensures **consistency and accuracy**. The ability to integrate with various documentation platforms and its wide range of supported output formats make PlantUML a versatile tool for both developers and non-developers. Lastly, being **open-source**, PlantUML boasts a strong community that continually contributes to its improvement and offers a wealth of resources for users at all levels.

2.1 Usecases

Use cases are enclosed using between parentheses (because two parentheses looks like an oval).

You can also use the **usecase** keyword to define a usecase. And you can define an alias, using the **as** keyword. This alias will be used later, when defining relations.

```
@startuml
(First usecase)
(Another usecase) as (UC2)
usecase UC3
usecase (Last\nusecase) as UC4
@enduml
```



2.2 Actors

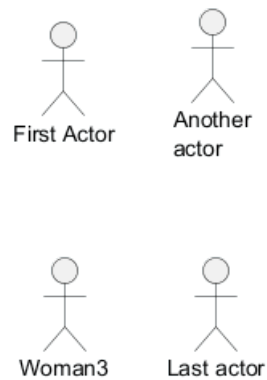
The name defining an actor is enclosed between colons.

You can also use the **actor** keyword to define an actor. An alias can be assigned using the **as** keyword and can be used later instead of the actor's name, e. g. when defining relations.

You can see from the following examples, that the actor definitions are optional.

```
@startuml
:First Actor:
:Another\nactor: as Man2
actor Woman3
actor :Last actor: as Person1
@enduml
```





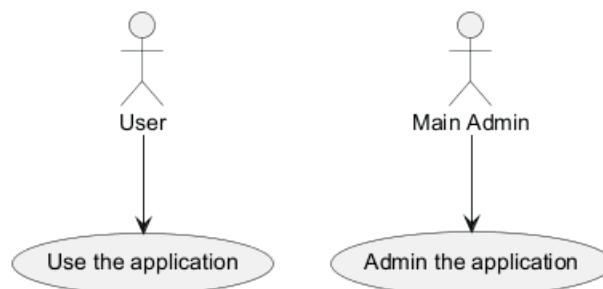
2.3 Change Actor style

You can change the actor style from stick man (*by default*) to:

- an awesome man with the `skinparam actorStyle awesome` command;
- a hollow man with the `skinparam actorStyle hollow` command.

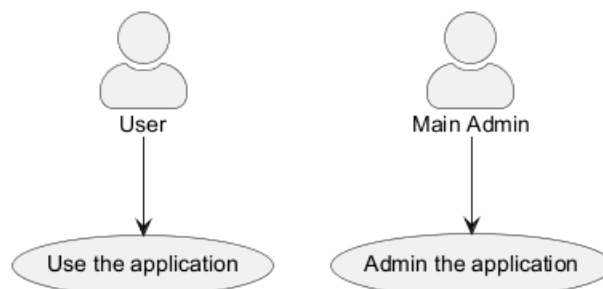
2.3.1 Stick man (*by default*)

```
@startuml
:User: --> (Use)
"Main Admin" as Admin
"Use the application" as (Use)
Admin --> (Admin the application)
@enduml
```



2.3.2 Awesome man

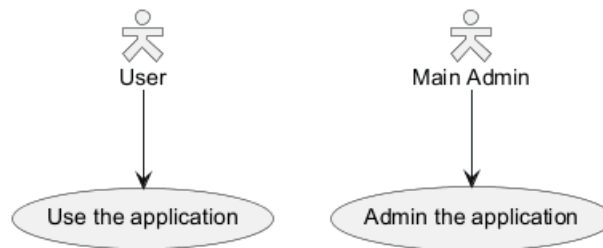
```
@startuml
skinparam actorStyle awesome
:User: --> (Use)
"Main Admin" as Admin
"Use the application" as (Use)
Admin --> (Admin the application)
@enduml
```



[Ref. QA-10493]

2.3.3 Hollow man

```
@startuml
skinparam actorStyle Hollow
:User: --> (Use)
"Main Admin" as Admin
"Use the application" as (Use)
Admin --> (Admin the application)
@enduml
```



[Ref. PR#396]

2.4 Usecases description

If you want to have a description spanning several lines, you can use quotes.

You can also use the following separators:

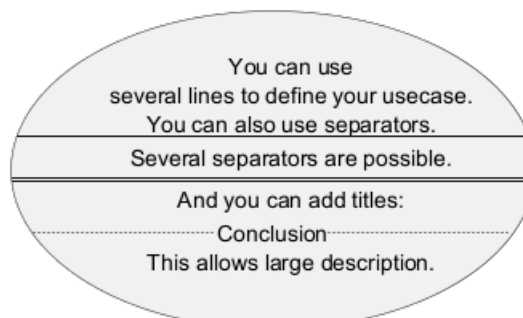
- -- (dashes)
- .. (periods)
- == (equals)
- __ (underscores)

By using them pairwise and enclosing text between them, you can create separators with titles.

```
@startuml

usecase UC1 as "You can use
several lines to define your usecase.
You can also use separators.
--
Several separators are possible.
==
And you can add titles:
..Conclusion..
This allows large description."

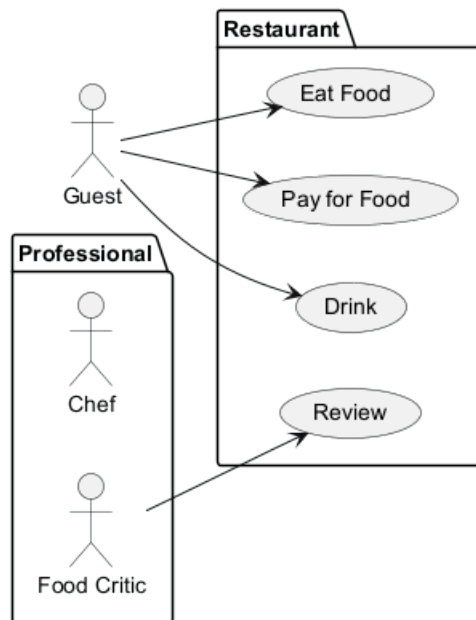
@enduml
```



2.5 Use package

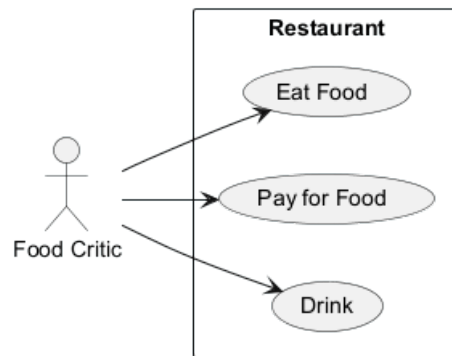
You can use packages to group actors or use cases.

```
@startuml
left to right direction
actor Guest as g
package Professional {
    actor Chef as c
    actor "Food Critic" as fc
}
package Restaurant {
    usecase "Eat Food" as UC1
    usecase "Pay for Food" as UC2
    usecase "Drink" as UC3
    usecase "Review" as UC4
}
fc --> UC4
g --> UC1
g --> UC2
g --> UC3
@enduml
```



You can use `rectangle` to change the display of the package.

```
@startuml
left to right direction
actor "Food Critic" as fc
rectangle Restaurant {
    usecase "Eat Food" as UC1
    usecase "Pay for Food" as UC2
    usecase "Drink" as UC3
}
fc --> UC1
fc --> UC2
fc --> UC3
@enduml
```



2.6 Basic example

To link actors and use cases, the arrow --> is used.

The more dashes - in the arrow, the longer the arrow. You can add a label on the arrow, by adding a : character in the arrow definition.

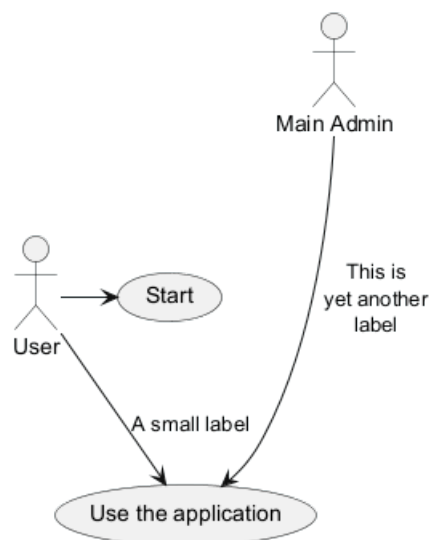
In this example, you see that *User* has not been defined before, and is used as an actor.

```

@startuml
User -> (Start)
User --> (Use the application) : A small label

:Main Admin: ---> (Use the application) : This is\nyet another\nlabel

@enduml
  
```



2.7 Extension

If one actor/use case extends another one, you can use the symbol <|--.

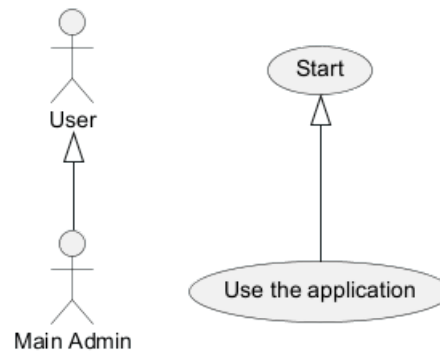
```

@startuml
:Main Admin: as Admin
(Use the application) as (Use)

User <|-- Admin
(Start) <|-- (Use)

@enduml
  
```





2.8 Using notes

You can use the `note left of`, `note right of`, `note top of`, `note bottom of` keywords to define notes related to a single object.

A note can be also define alone with the `note` keywords, then linked to other objects using the `..` symbol.

```

@startuml
:Main Admin: as Admin
(Use the application) as (Use)
  
```

```
User -> (Start)
```

```
User --> (Use)
```

```
Admin ---> (Use)
```

```
note right of Admin : This is an example.
```

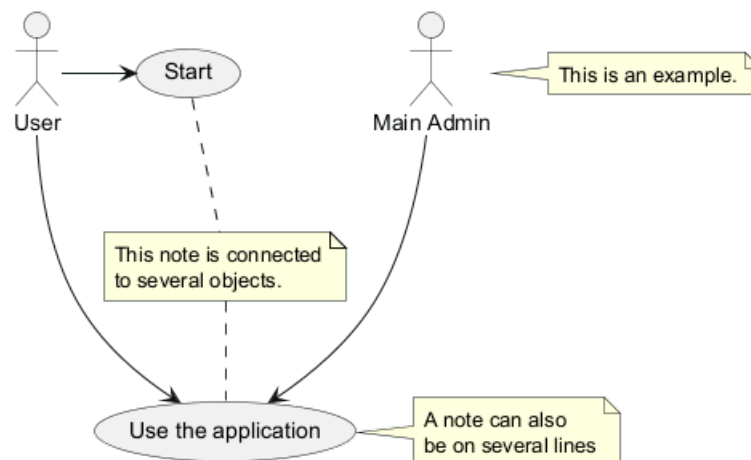
```
note right of (Use)
```

```

  A note can also
  be on several lines
end note
  
```

```

note "This note is connected\nto several objects." as N2
(Start) .. N2
N2 .. (Use)
@enduml
  
```



2.9 Stereotypes

You can add stereotypes while defining actors and use cases using `<<` and `>>`.



```

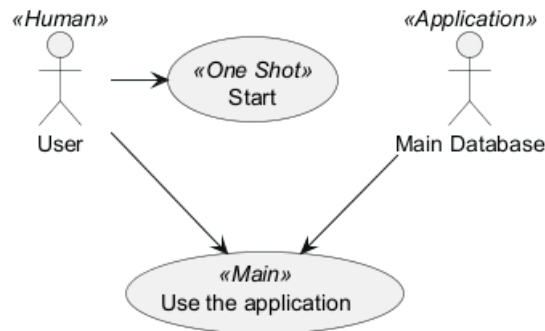
@startuml
User << Human >>
:Main Database: as MySql << Application >>
(Start) << One Shot >>
(Use the application) as (Use) << Main >>

User -> (Start)
User --> (Use)

MySql --> (Use)

@enduml

```



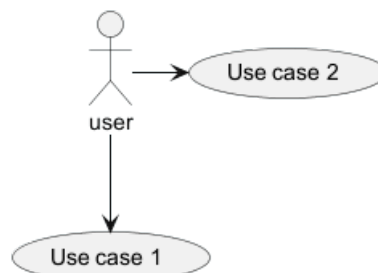
2.10 Changing arrows direction

By default, links between classes have two dashes -- and are vertically oriented. It is possible to use horizontal link by putting a single dash (or dot) like this:

```

@startuml
:user: --> (Use case 1)
:user: -> (Use case 2)
@enduml

```

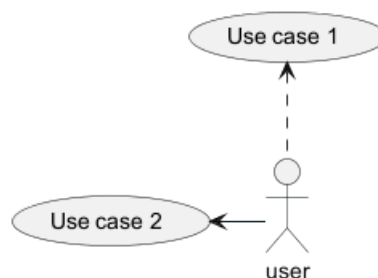


You can also change directions by reversing the link:

```

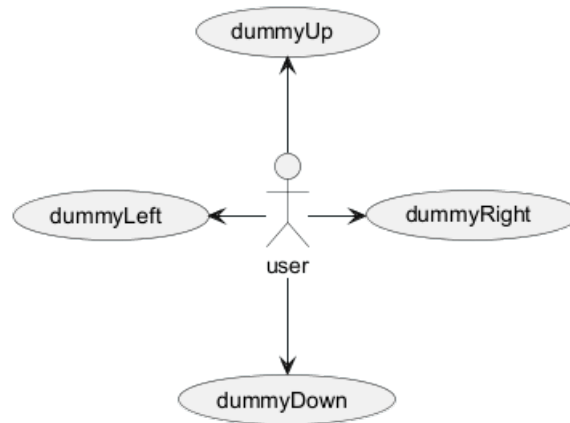
@startuml
(Use case 1) <.. :user:
(Use case 2) <- :user:
@enduml

```



It is also possible to change arrow direction by adding `left`, `right`, `up` or `down` keywords inside the arrow:

```
@startuml
:user: -left-> (dummyLeft)
:user: -right-> (dummyRight)
:user: -up-> (dummyUp)
:user: -down-> (dummyDown)
@enduml
```

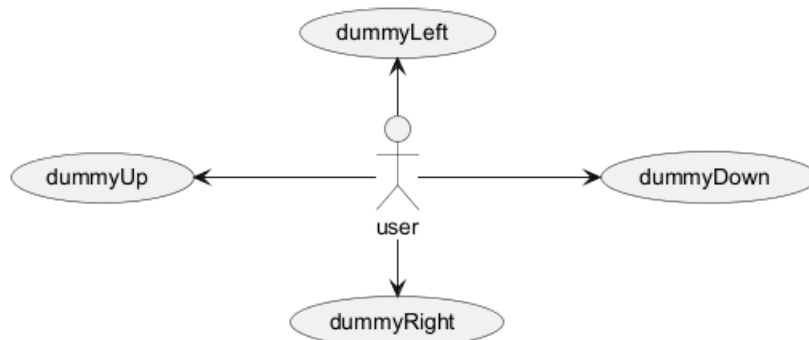


You can shorten the arrow by using only the first character of the direction (for example, `-d-` instead of `-down-`) or the two first characters (`-do-`).

Please note that you should not abuse this functionality : *Graphviz* gives usually good results without tweaking.

And with the `left to right direction` parameter:

```
@startuml
left to right direction
:user: -left-> (dummyLeft)
:user: -right-> (dummyRight)
:user: -up-> (dummyUp)
:user: -down-> (dummyDown)
@enduml
```

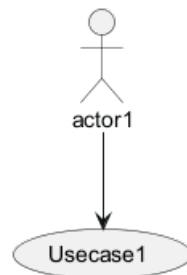


2.11 Splitting diagrams

The `newpage` keywords to split your diagram into several pages or images.

```
@startuml
:actor1: --> (Usecase1)
newpage
:actor2: --> (Usecase2)
@enduml
```



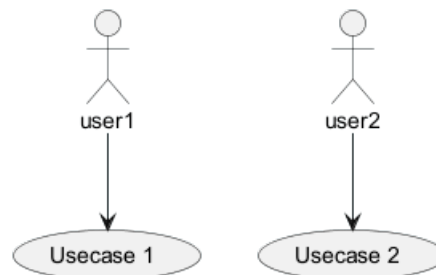


2.12 Left to right direction

The general default behavior when building diagram is **top to bottom**.

```

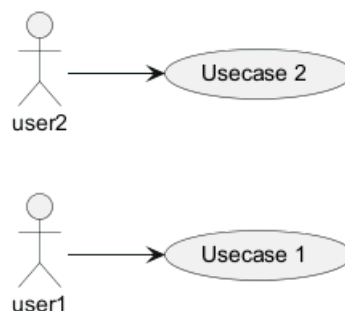
@startuml
'default
top to bottom direction
user1 --> (Usecase 1)
user2 --> (Usecase 2)
@enduml
  
```



You may change to **left to right** using the **left to right direction** command. The result is often better with this direction.

```

@startuml
left to right direction
user1 --> (Usecase 1)
user2 --> (Usecase 2)
@enduml
  
```



See also 'Change diagram orientation' on Deployment diagram page.

2.13 Skinparam

You can use the skinparam command to change colors and fonts for the drawing.

You can use this command :



- In the diagram definition, like any other commands,
- In an included file,
- In a configuration file, provided in the command line or the ANT task.

You can define specific color and fonts for stereotyped actors and usecases.

```
@startuml
skinparam handwritten true

skinparam usecase {
  BackgroundColor DarkSeaGreen
  BorderColor DarkSlateGray

  BackgroundColor<< Main >> YellowGreen
  BorderColor<< Main >> YellowGreen

  ArrowColor Olive
  ActorBorderColor black
  ActorFontName Courier

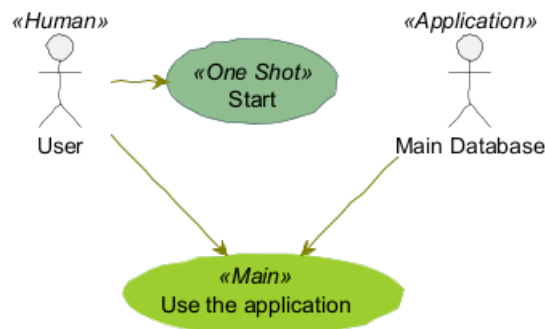
  ActorBackgroundColor<< Human >> Gold
}

User << Human >>
:Main Database: as MySql << Application >>
(Start) << One Shot >>
(Use the application) as (Use) << Main >>

User -> (Start)
User --> (Use)

MySql --> (Use)

@enduml
```



2.14 Complete example

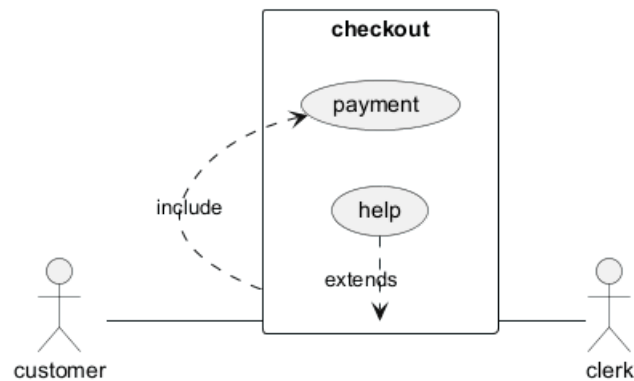
```
@startuml
left to right direction
skinparam packageStyle rectangle
actor customer
actor clerk
rectangle checkout {
  customer -- (checkout)
  (checkout) .> (payment) : include
  (help) .> (checkout) : extends
  (checkout) -- clerk
}
```



```

}
@enduml

```



2.15 Business Use Case

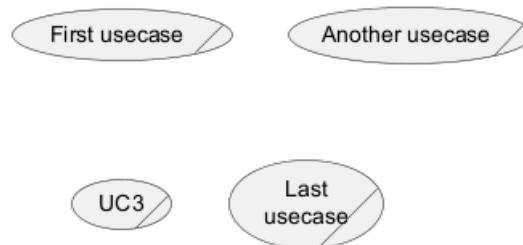
You can add / to make Business Use Case.

2.15.1 Business Usecase

```

@startuml
(First usecase)/
(Another usecase)/ as UC2
usecase/ UC3
usecase/ (Last\nusecase) as UC4
@enduml

```

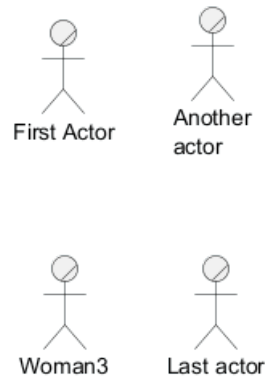


2.15.2 Business Actor

```

@startuml
:First Actor:/
:Another\nactor:/ as Man2
actor/ Woman3
actor/ :Last actor: as Person1
@enduml

```



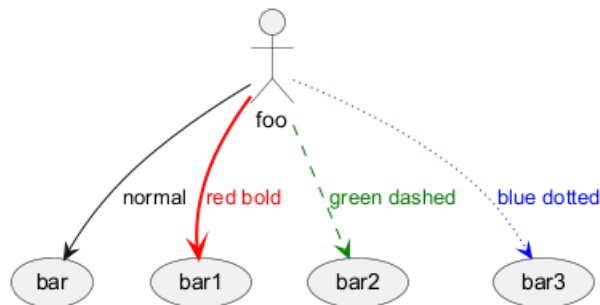
[Ref. QA-12179]

2.16 Change arrow color and style (inline style)

You can change the color or style of individual arrows using the inline following notation:

- `#color;line.[bold|dashed|dotted];text:color`

```
@startuml
actor foo
foo --> (bar) : normal
foo --> (bar1) #line:red;line.bold;text:red : red bold
foo --> (bar2) #green;line.dashed;text:green : green dashed
foo --> (bar3) #blue;line.dotted;text:blue : blue dotted
@enduml
```



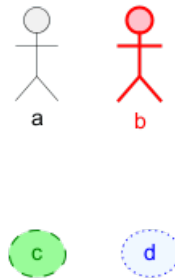
[Ref. QA-3770 and QA-3816] [See similar feature on deployment-diagram or class diagram]

2.17 Change element color and style (inline style)

You can change the color or style of individual element using the following notation:

- `#[color|back:color];line:color;line.[bold|dashed|dotted];text:color`

```
@startuml
actor a
actor b #pink;line:red;line.bold;text:red
usecase c #palegreen;line:green;line.dashed;text:green
usecase d #aliceblue;line:blue;line.dotted;text:blue
@enduml
```



[Ref. QA-5340 and adapted from QA-6852]

2.18 Display JSON Data on Usecase diagram

2.18.1 Simple example

```
@startuml
allowmixing

actor Actor
usecase Usecase

json JSON {
    "fruit": "Apple",
    "size": "Large",
    "color": ["Red", "Green"]
}
@enduml
```



JSON	
fruit	Apple
size	Large
color	Red
	Green

[Ref. QA-15481]

For another example, see on JSON page.



3 Class Diagram

Class diagrams are designed using a syntax that mirrors those traditionally employed in programming languages. This resemblance fosters a familiar environment for developers, thereby facilitating an easier and more intuitive diagram creation process.

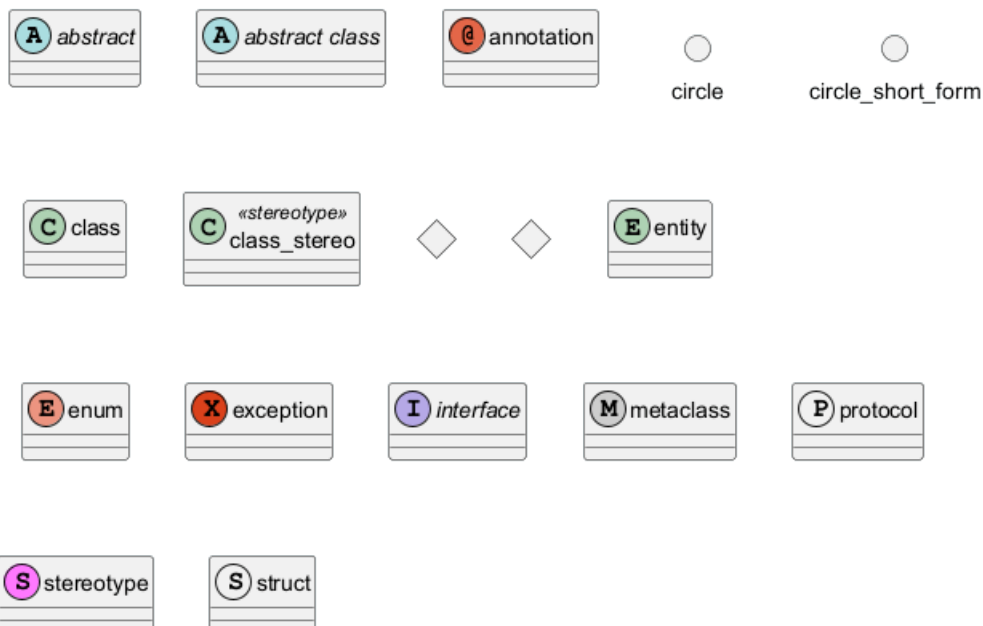
This design approach is not only succinct but also enables the creation of representations that are both concise and expressive. Moreover, it allows for the portrayal of relationships between classes through a syntax that echoes that of sequence diagrams, paving the way for a fluid and insightful depiction of class interactions.

Beyond structural and relational representations, the class diagram syntax supports further enrichments such as the inclusion of notes and the application of colors, empowering users to create diagrams that are both informative and visually appealing.

You can learn more about some of the common commands in PlantUML to enhance your diagram creation experience.

3.1 Declaring element

```
@startuml
abstract      abstract
abstract class "abstract class"
annotation    annotation
circle        circle
()            circle_short_form
class         class
class         class_stereo <<stereotype>>
diamond       diamond
<>           diamond_short_form
entity        entity
enum          enum
exception     exception
interface     interface
metaclass     metaclass
protocol      protocol
stereotype    stereotype
struct        struct
@enduml
```



[Ref. for *protocol* and *struct*: GH-1028, for *exception*: QA-16258]

3.2 Relations between classes

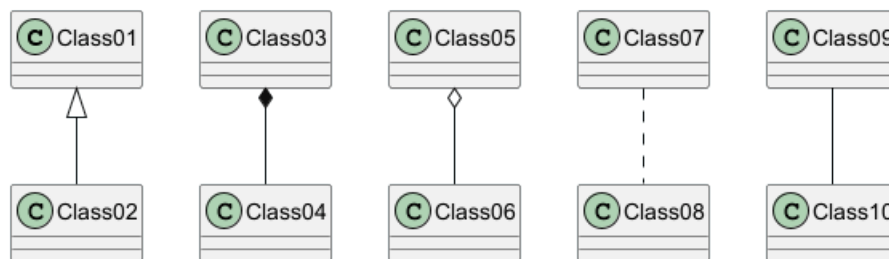
Relations between classes are defined using the following symbols :

Type	Symbol	Purpose
Extension	< --	Specialization of a class in a hierarchy
Implementation	< ..	Realization of an interface by a class
Composition	*--	The part cannot exist without the whole
Aggregation	o--	The part can exist independently of the whole
Dependency	-->	The object uses another object
Dependency	..>	A weaker form of dependency

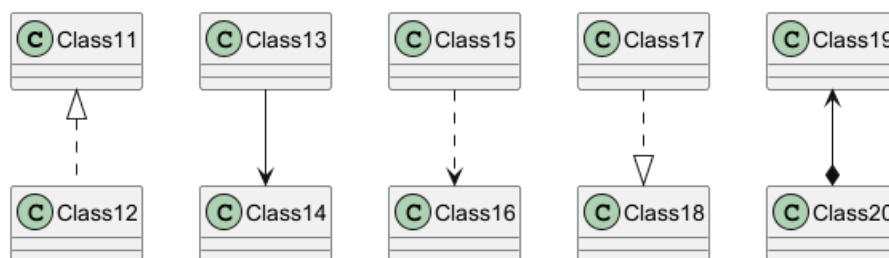
It is possible to replace -- by .. to have a dotted line.

Knowing those rules, it is possible to draw the following drawings:

```
@startuml
Class01 <|-- Class02
Class03 *-- Class04
Class05 o-- Class06
Class07 .. Class08
Class09 -- Class10
@enduml
```

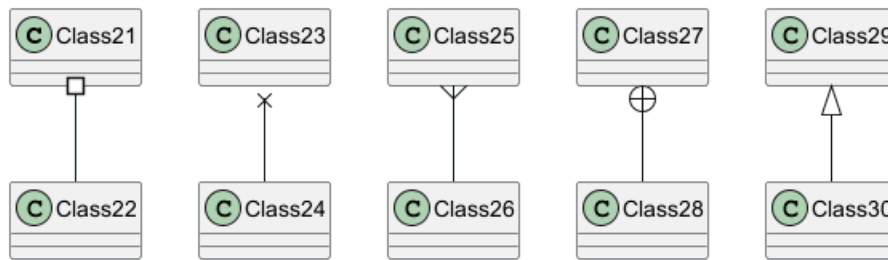


```
@startuml
Class11 <|.. Class12
Class13 --> Class14
Class15 ..> Class16
Class17 ..|> Class18
Class19 <--* Class20
@enduml
```



```
@startuml
Class21 #-- Class22
Class23 x-- Class24
Class25 }-- Class26
Class27 +-- Class28
Class29 ^-- Class30
@enduml
```





3.3 Label on relations

It is possible to add a label on the relation, using `:`, followed by the text of the label.

For cardinality, you can use double-quotes `"` on each side of the relation.

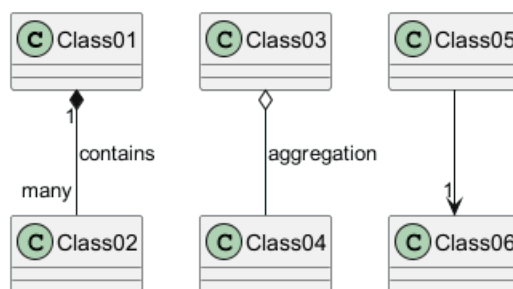
```
@startuml
```

```
Class01 "1" *-- "many" Class02 : contains
```

```
Class03 o-- Class04 : aggregation
```

```
Class05 --> "1" Class06
```

```
@enduml
```



You can add an extra arrow pointing at one object showing which object acts on the other object, using `<` or `>` at the begin or at the end of the label.

```
@startuml
```

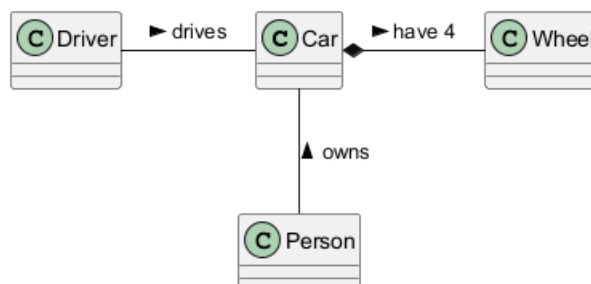
```
class Car
```

```
Driver - Car : drives >
```

```
Car *- Wheel : have 4 >
```

```
Car -- Person : < owns
```

```
@enduml
```



3.4 Using non-letters in element names and relation labels

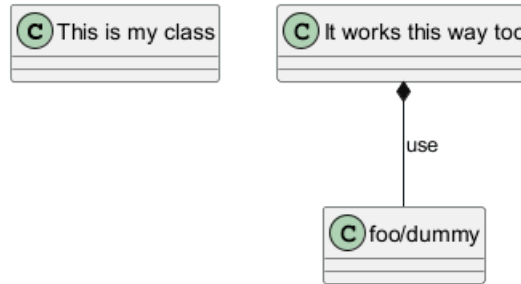
If you want to use non-letters in the class (or enum...) display name, you can either :



- Use the **as** keyword in the class definition to assign an alias
- Put quotes "" around the class name

```
@startuml
class "This is my class" as class1
class class2 as "It works this way too"

class2 *-- "foo/dummy" : use
@enduml
```



If an alias is assigned to an element, the rest of the file must refer to the element by the alias instead of the name.

3.4.1 Starting names with \$

Note that names starting with \$ cannot be hidden or removed later, because **hide** and **remove** command will consider the name a **\$tag** instead of a component name. To later remove such elements they must have an alias or must be tagged.

```
@startuml
class $C1
class $C2 $C2
class "$C2" as dollarC2
remove $C1
remove $C2
remove dollarC2
@enduml
```



Also note that names starting with \$ are valid, but to assign an alias to such element the name must be put between quotes "".

3.5 Adding methods

To declare fields and methods, you can use the symbol **:** followed by the field's or method's name.

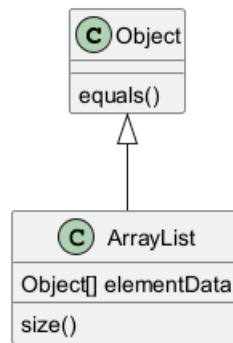
The system checks for parenthesis to choose between methods and fields.

```
@startuml
Object <|-- ArrayList

Object : equals()
ArrayList : Object[] elementData
ArrayList : size()

@enduml
```





It is also possible to group between brackets `{}` all fields and methods.

Note that the syntax is highly flexible about type/name order.

```

@startuml
class Dummy {
    String data
    void methods()
}

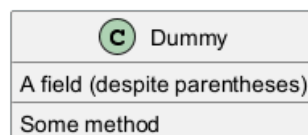
class Flight {
    flightNumber : Integer
    departureTime : Date
}
@enduml
  
```



You can use `{field}` and `{method}` modifiers to override default behaviour of the parser about fields and methods.

```

@startuml
class Dummy {
    {field} A field (despite parentheses)
    {method} Some method
}
@enduml
  
```



3.6 Defining visibility

3.6.1 Visibility for methods or fields

When you define methods or fields, you can use characters to define the visibility of the corresponding item:

Character	Icon for field	Icon for method	Visibility
-	□	■	private
#	◇	◆	protected
~	△	▲	package private
+	○	●	public



```
@startuml
class Dummy {
    -field1
    #field2
    ~method1()
    +method2()
}
@enduml
```



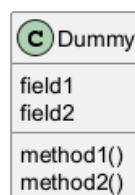
You can turn off this feature using the `skinparam classAttributeIconSize 0` command :

```
@startuml
skinparam classAttributeIconSize 0
class Dummy {
    -field1
    #field2
    ~method1()
    +method2()
}
@enduml
```



Visibility indicators are optional and can be omitted individually without turning off the icons globally using `skinparam classAttributeIconSize 0`.

```
@startuml
class Dummy {
    field1
    field2
    method1()
    method2()
}
@enduml
```

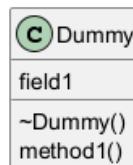


In such case if you'd like to use methods or fields that start with -, #, ~ or + characters such as a destructor

in some languages for Dummy class (), escape the first character with a \ character:

```
@startuml
class Dummy {
    field1
    \~Dummy()
    method1()
}

@enduml
```



3.6.2 Visibility for class

Similar to methods or fields, you can use same characters to define the Class visibility:

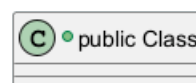
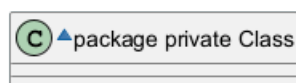
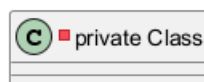
```
@startuml
-class "private Class" {

#class "protected Class" {

~class "package private Class" {

+class "public Class" {
}

@enduml
```



[Ref. QA-4755]

3.7 Abstract and Static

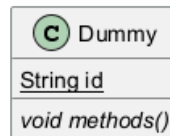
You can define static or abstract methods or fields using the {static} or {abstract} modifier.

These modifiers can be used at the start or at the end of the line. You can also use {classifier} instead of {static}.

```
@startuml
class Dummy {
    {static} String id
    {abstract} void methods()
}

@enduml
```





3.8 Advanced class body

By default, methods and fields are automatically regrouped by PlantUML. You can use separators to define your own way of ordering fields and methods. The following separators are possible : -- .. == __.

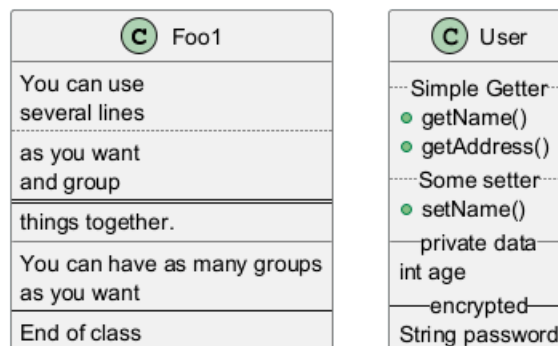
You can also use titles within the separators:

```

@startuml
class Foo1 {
    You can use
    several lines
    ..
    as you want
    and group
    ==
    things together.
    --
    You can have as many groups
    as you want
    --
    End of class
}

class User {
    .. Simple Getter ..
    + getName()
    + getAddress()
    .. Some setter ..
    + setName()
    __ private data __
    int age
    -- encrypted --
    String password
}

@enduml
  
```



3.9 Notes and stereotypes

Stereotypes are defined with the `class` keyword, `<<` and `>>`.



You can also define notes using `note left of`, `note right of`, `note top of`, `note bottom of` keywords.

You can also define a note on the last defined class using `note left`, `note right`, `note top`, `note bottom`.

A note can be also define alone with the `note` keywords, then linked to other objects using the `..` symbol.

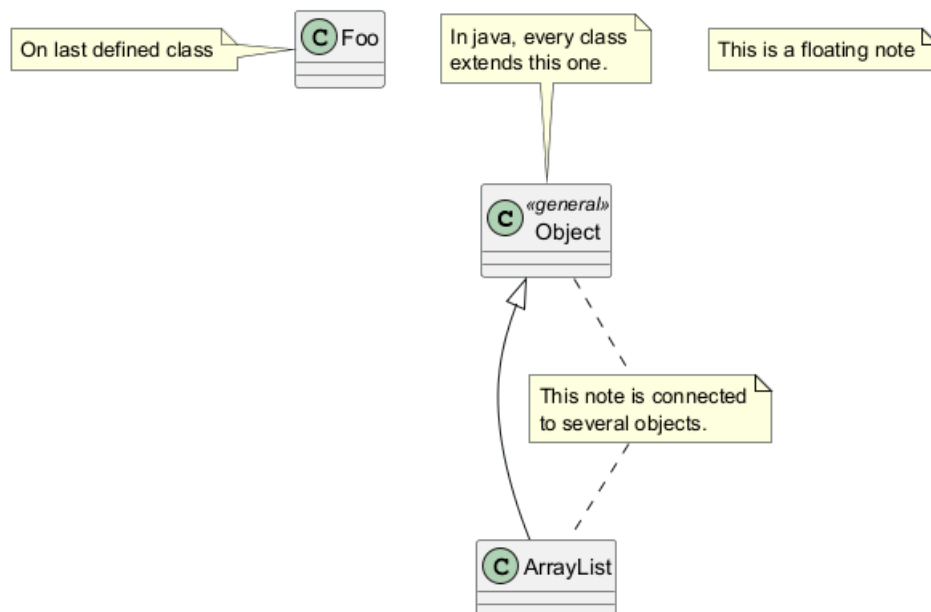
```
@startuml
class Object << general >>
Object <|--- ArrayList
```

```
note top of Object : In java, every class\nextends this one.
```

```
note "This is a floating note" as N1
note "This note is connected\nto several objects." as N2
Object .. N2
N2 .. ArrayList
```

```
class Foo
note left: On last defined class
```

```
@enduml
```



3.10 More on notes

It is also possible to use few HTML tags (See Creole expression) like :

- `<b>`
- `<u>`
- `<i>`
- `<s>`, `<del>`, `<strike>`
- `<font color="#AAAAAA">` or `<font color="colorName">`
- `<color:#AAAAAA>` or `<color:colorName>`
- `<size:nn>` to change font size
- `` or `<img:file>`: the file must be accessible by the filesystem



You can also have a note on several lines.

You can also define a note on the last defined class using `note left`, `note right`, `note top`, `note bottom`.

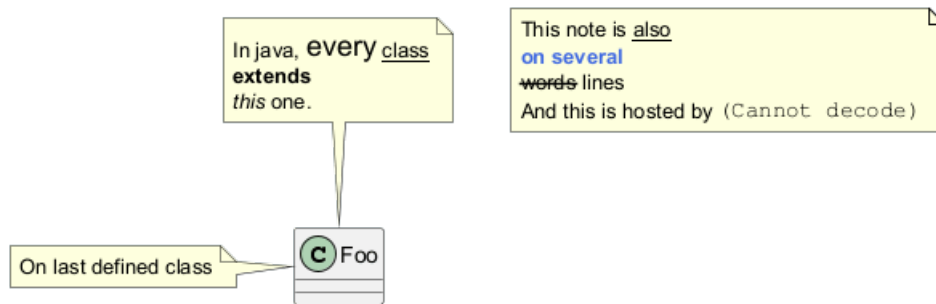
```
@startuml

class Foo
note left: On last defined class

note top of Foo
  In java, <size:18>every</size> <u>class</u>
  <b>extends</b>
  <i>this</i> one.
end note

note as N1
  This note is <u>also</u>
  <b><color:royalBlue>on several</color>
  <s>words</s> lines
  And this is hosted by <img:sourceforge.jpg>
end note

@enduml
```



3.11 Note on field (field, attribute, member) or method

It is possible to add a note on field (field, attribute, member) or on method.

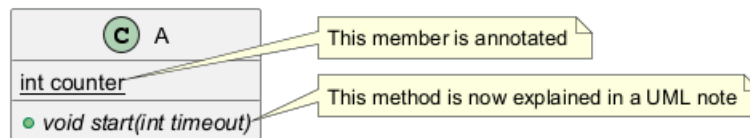
3.11.1 Constraint

- This cannot be used with `top` or `bottom` (*only left and right are implemented*)
- This cannot be used with `namespaceSeparator ::`

3.11.2 Note on field or method

```
@startuml
class A {
{static} int counter
+void {abstract} start(int timeout)
}
note right of A::counter
  This member is annotated
end note
note right of A::start
  This method is now explained in a UML note
end note
@enduml
```

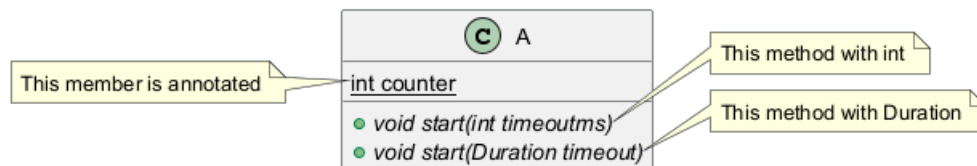




3.11.3 Note on method with the same name

```

@startuml
class A {
{static} int counter
+void {abstract} start(int timeouts)
+void {abstract} start(Duration timeout)
}
note left of A::counter
  This member is annotated
end note
note right of A::"start(int timeouts)"
  This method with int
end note
note right of A::"start(Duration timeout)"
  This method with Duration
end note
@enduml
  
```



[Ref. QA-3474 and QA-5835]

3.12 Note on links

It is possible to add a note on a link, just after the link definition, using `note on link`.

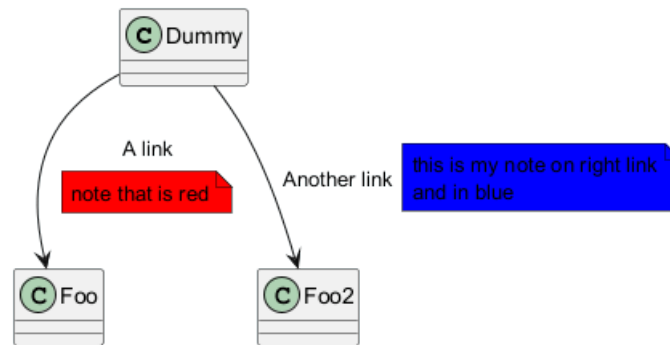
You can also use `note left on link`, `note right on link`, `note top on link`, `note bottom on link` if you want to change the relative position of the note with the label.

```

@startuml
class Dummy
Dummy --> Foo : A link
note on link #red: note that is red

Dummy --> Foo2 : Another link
note right on link #blue
this is my note on right link
and in blue
end note
@enduml
  
```





3.13 Abstract class and interface

You can declare a class as abstract using `abstract` or `abstract class` keywords.

The class will be printed in *italic*.

You can use the `interface`, `annotation` and `enum` keywords too.

```

@startuml

abstract class AbstractList
abstract AbstractCollection
interface List
interface Collection

List <|-- AbstractList
Collection <|-- AbstractCollection

Collection <|-- List
AbstractCollection <|-- AbstractList
AbstractList <|-- ArrayList

class ArrayList {
    Object[] elementData
    size()
}

enum TimeUnit {
    DAYS
    HOURS
    MINUTES
}

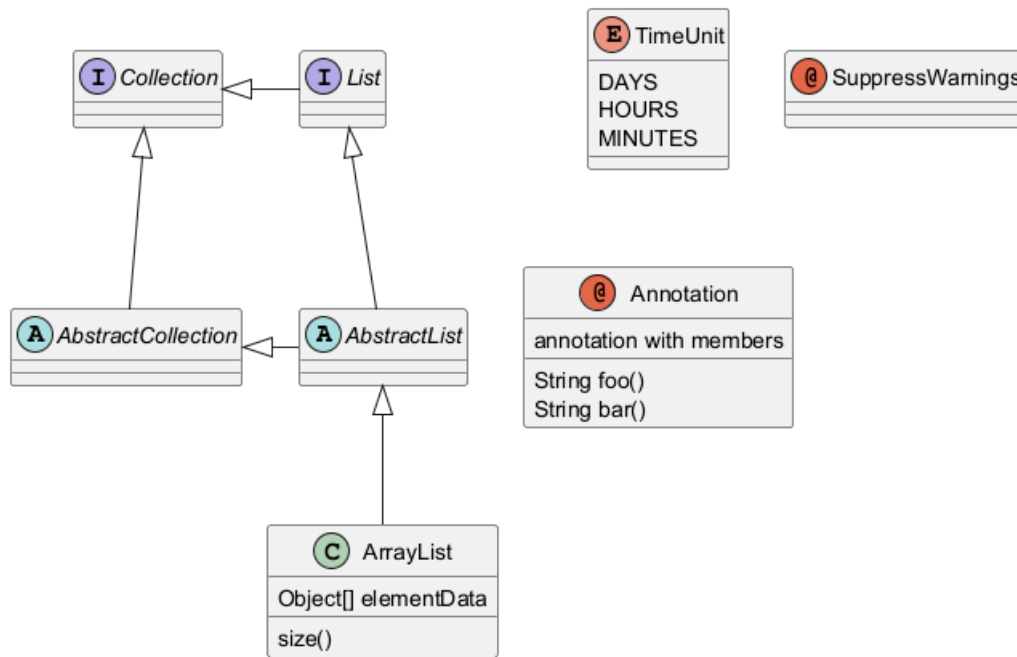
annotation SuppressWarnings

annotation Annotation {
    annotation with members
    String foo()
    String bar()
}

@enduml

```





[Ref. 'Annotation with members' Issue#458]

3.14 Hide attributes, methods...

You can parameterize the display of classes using the **hide/show** command.

The basic command is: **hide empty members**. This command will hide attributes or methods if they are empty.

Instead of **empty members**, you can use:

- **empty fields** or **empty attributes** for empty fields,
- **empty methods** for empty methods,
- **fields** or **attributes** which will hide fields, even if they are described,
- **methods** which will hide methods, even if they are described,
- **members** which will hide fields and methods, even if they are described,
- **circle** for the circled character in front of class name,
- **stereotype** for the stereotype.

You can also provide, just after the **hide** or **show** keyword:

- **class** for all classes,
- **interface** for all interfaces,
- **enum** for all enums,
- **<<foo1>>** for classes which are stereotyped with *foo1*,
- an existing class name.

You can use several **show/hide** commands to define rules and exceptions.

@startuml

```

class Dummy1 {
    +myMethods()
}

```



```

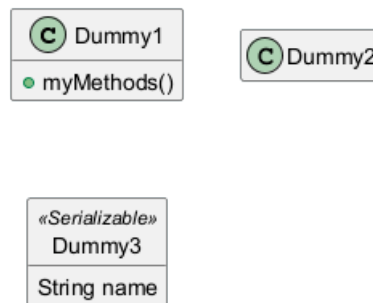
class Dummy2 {
    +hiddenMethod()
}

class Dummy3 <<Serializable>> {
    String name
}

hide members
hide <<Serializable>> circle
show Dummy1 methods
show <<Serializable>> fields

@enduml

```



You can also mix with visibility:

```

@startuml
hide private members
hide protected members
hide package members

class Foo {
    - private
    # protected
    ~ package
}

@enduml

```



[Ref. QA-2913]

3.15 Hide classes

You can also use the **show/hide** commands to hide classes.

This may be useful if you define a large !included file, and if you want to hide some classes after file inclusion.

```

@startuml

class Foo1
class Foo2

Foo2 *-- Foo1

hide Foo2

```



```
@enduml
```



3.16 Remove classes

You can also use the **remove** commands to remove classes.

This may be useful if you define a large !included file, and if you want to remove some classes after file inclusion.

```
@startuml
class Foo1
class Foo2

Foo2 *-- Foo1

remove Foo2

@enduml
```

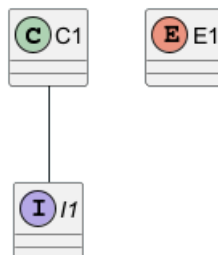


3.17 Hide, Remove or Restore tagged element or wildcard

You can put **\$tags** (using **\$**) on elements, then remove, hide or restore components either individually or by tags.

By default, all components are displayed:

```
@startuml
class C1 $tag13
enum E1
interface I1 $tag13
C1 -- I1
@enduml
```



But you can:

- hide \$tag13 components:

```
@startuml
```



```
class C1 $tag13
enum E1
interface I1 $tag13
C1 -- I1
```

```
hide $tag13
@enduml
```



- or remove \$tag13 components:

```
@startuml
class C1 $tag13
enum E1
interface I1 $tag13
C1 -- I1
```

```
remove $tag13
@enduml
```



- or remove \$tag13 and restore \$tag1 components:

```
@startuml
class C1 $tag13 $tag1
enum E1
interface I1 $tag13
C1 -- I1
```

```
remove $tag13
restore $tag1
@enduml
```



- or remove * and restore \$tag1 components:

```
@startuml
class C1 $tag13 $tag1
enum E1
interface I1 $tag13
C1 -- I1
```

```
remove *
restore $tag1
@enduml
```



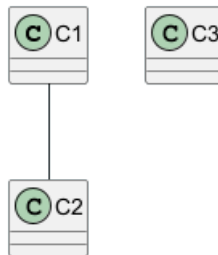


3.18 Hide or Remove unlinked class

By default, all classes are displayed:

```

@startuml
class C1
class C2
class C3
C1 -- C2
@enduml
  
```



But you can:

- hide @unlinked classes:

```

@startuml
class C1
class C2
class C3
C1 -- C2

hide @unlinked
@enduml
  
```



- or remove @unlinked classes:

```

@startuml
class C1
class C2
class C3
C1 -- C2

remove @unlinked
@enduml
  
```



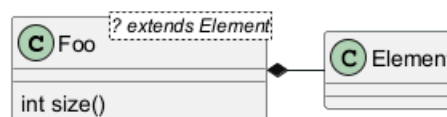
[Adapted from QA-11052]

3.19 Use generics

You can also use bracket < and > to define generics usage in a class.

```

@startuml
class Foo<? extends Element> {
    int size()
}
Foo *- Element
@enduml
  
```



It is possible to disable this drawing using `skinparam genericDisplay old` command.

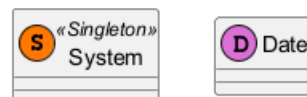
3.20 Specific Spot

Usually, a spotted character (C, I, E or A) is used for classes, interface, enum and abstract classes.

But you can define your own spot for a class when you define the stereotype, adding a single character and a color, like in this example:

```

@startuml
class System << (S,#FF7700) Singleton >>
class Date << (D,orchid) >>
@enduml
  
```



3.21 Packages

You can define a package using the `package` keyword, and optionally declare a background color for your package (Using a html color code or name).

Note that package definitions can be nested.

```

@startuml
package "Classic Collections" #DDDDDD {
    Object <|-- ArrayList
}

package com.plantuml {
  
```

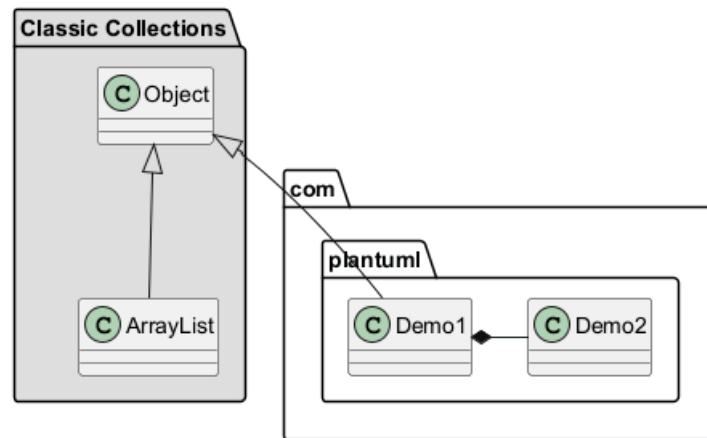


```

Object <|-- Demo1
Demo1 *-- Demo2
}

@enduml

```



3.22 Packages style

There are different styles available for packages.

You can specify them either by setting a default style with the command : `skinparam packageStyle`, or by using a stereotype on the package:

```

@startuml
scale 750 width
package foo1 <<Node>> {
    class Class1
}

package foo2 <<Rectangle>> {
    class Class2
}

package foo3 <<Folder>> {
    class Class3
}

package foo4 <<Frame>> {
    class Class4
}

package foo5 <<Cloud>> {
    class Class5
}

package foo6 <<Database>> {
    class Class6
}

@enduml

```




You can also define links between packages, like in the following example:

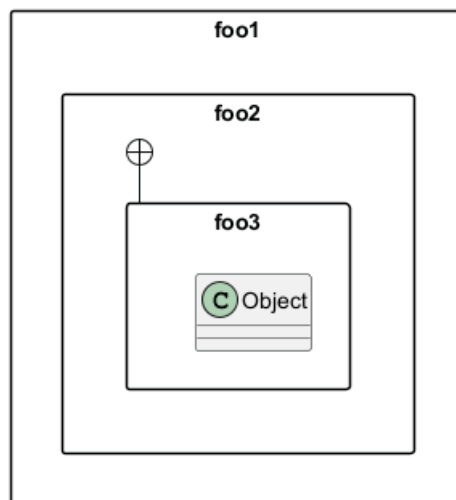
```
@startuml
skinparam packageStyle rectangle

package foo1.foo2 {
}

package foo1.foo2.foo3 {
    class Object
}

foo1.foo2 +-- foo1.foo2.foo3

@enduml
```



3.23 Namespaces

Starting with version 1.2023.2 (which is online as a beta), PlantUML handles differently namespaces and packages.

There won't be any difference between namespaces and packages anymore: both keywords are now synonymous.

3.24 Automatic package creation

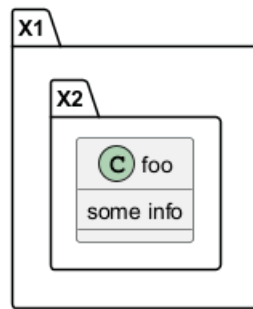
You can define another separator (other than the dot) using the command : `set separator ???`.

```
@startuml

set separator ::
class X1::X2::foo {
    some info
}
```



```
@enduml
```

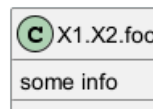


You can disable automatic namespace creation using the command `set separator none`.

```
@startuml
```

```
set separator none
class X1.X2.foo {
    some info
}
```

```
@enduml
```



3.25 Lollipop interface

You can also define lollipops interface on classes, using the following syntax:

- `bar ()- foo`
- `bar ()-- foo`
- `foo -() bar`

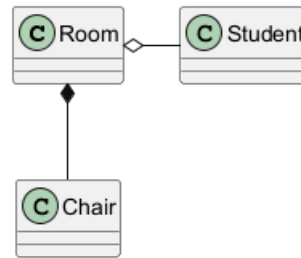
```
@startuml
class foo
bar ()- foo
@enduml
```



3.26 Changing arrows orientation

By default, links between classes have two dashes `--` and are vertically oriented. It is possible to use horizontal link by putting a single dash (or dot) like this:

```
@startuml
Room o- Student
Room *-- Chair
@enduml
```

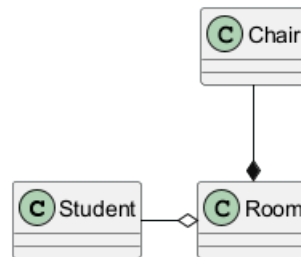


You can also change directions by reversing the link:

```

@startuml
Student -o Room
Chair --* Room
@enduml

```

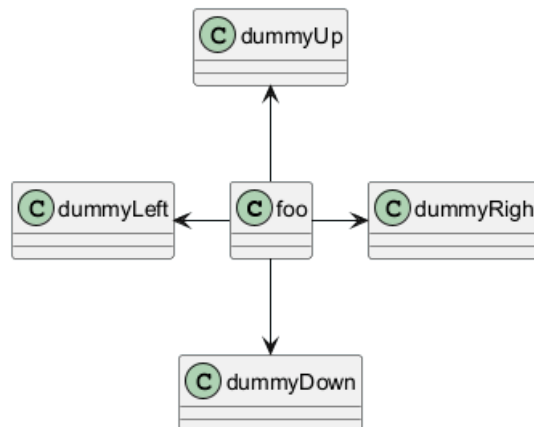


It is also possible to change arrow direction by adding *left*, *right*, *up* or *down* keywords inside the arrow:

```

@startuml
foo -left-> dummyLeft
foo -right-> dummyRight
foo -up-> dummyUp
foo -down-> dummyDown
@enduml

```



You can shorten the arrow by using only the first character of the direction (for example, *-d-* instead of *-down-*) or the two first characters (*-do-*).

Please note that you should not abuse this functionality : *Graphviz* gives usually good results without tweaking.

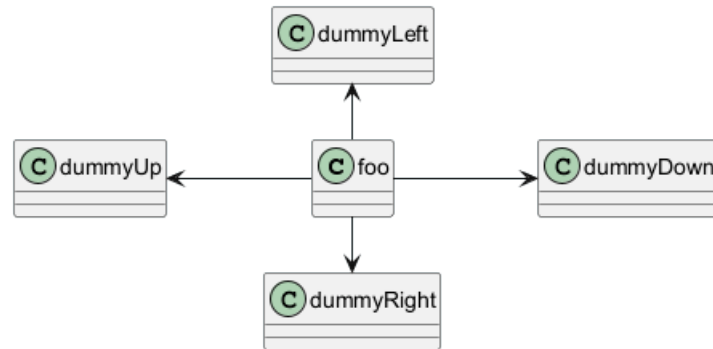
And with the *left to right direction* parameter:

```

@startuml
left to right direction
foo -left-> dummyLeft
foo -right-> dummyRight
foo -up-> dummyUp

```

```
foo -down-> dummyDown
@enduml
```

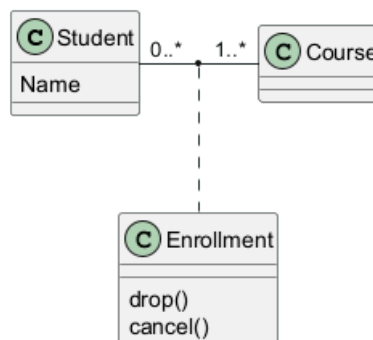


3.27 Association classes

You can define *association class* after that a relation has been defined between two classes, like in this example:

```
@startuml
class Student {
    Name
}
Student "0..*" -- "1..*" Course
(Student, Course) .. Enrollment

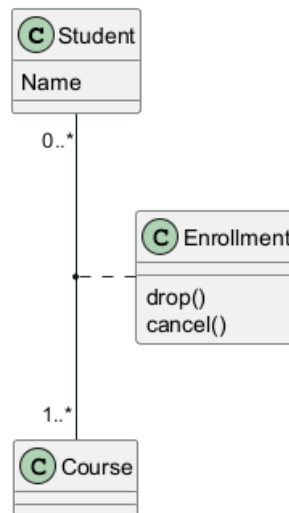
class Enrollment {
    drop()
    cancel()
}
@enduml
```



You can define it in another direction:

```
@startuml
class Student {
    Name
}
Student "0..*" -- "1..*" Course
(Student, Course) . Enrollment

class Enrollment {
    drop()
    cancel()
}
@enduml
```



3.28 Association on same class

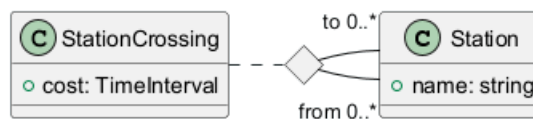
```

@startuml
class Station {
    +name: string
}

class StationCrossing {
    +cost: TimeInterval
}

<> diamond

StationCrossing . diamond
diamond - "from 0..*" Station
diamond - "to 0..*" Station
@enduml
  
```



[Ref. Incubation: Associations]

3.29 Skinparam

You can use the skinparam command to change colors and fonts for the drawing.

You can use this command :

- In the diagram definition, like any other commands,
- In an included file,
- In a configuration file, provided in the command line or the ANT task.

```

@startuml
skinparam class {
  BackgroundColor PaleGreen
  ArrowColor SeaGreen
  BorderColor SpringGreen
}
  
```



```

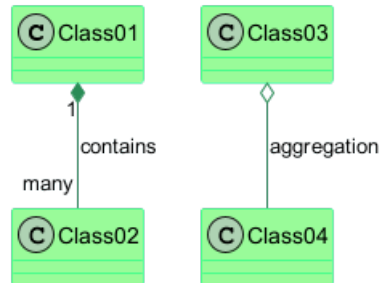
skinparam stereotypeCBackgroundColor YellowGreen

Class01 "1" *-- "many" Class02 : contains

Class03 o-- Class04 : aggregation

@enduml

```



3.30 Skinned Stereotypes

You can define specific color and fonts for stereotyped classes.

```

@startuml

skinparam class {
  BackgroundColor PaleGreen
  ArrowColor SeaGreen
  BorderColor SpringGreen
  BackgroundColor<<Foo>> Wheat
  BorderColor<<Foo>> Tomato
}

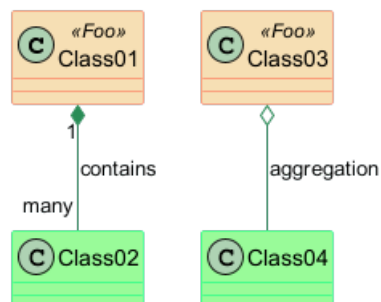
skinparam stereotypeCBackgroundColor YellowGreen
skinparam stereotypeCBackgroundColor<< Foo >> DimGray

class Class01 <<Foo>>
class Class03 <<Foo>>
Class01 "1" *-- "many" Class02 : contains

Class03 o-- Class04 : aggregation

@enduml

```



Important: unlike class stereotypes, there must be no space between the skin parameter and the following stereotype.

Any of the spaces shown as _ below will cause **all** skinparams to be ignored, see discord discussion and issue #1932:

- BackgroundColor_<<Foo>> Wheat



- `skinparam stereotypeCBackgroundColor_<<Foo>> DimGray`

3.31 Color gradient

You can declare individual colors for classes, notes etc using the `#` notation.

You can use standard color names or RGB codes in various notations, see Colors.

You can also use color gradient for background colors, with the following syntax: two colors names separated either by:

- `|`,
- `/`,
- `\`, or
- `-`

depending on the direction of the gradient.

For example:

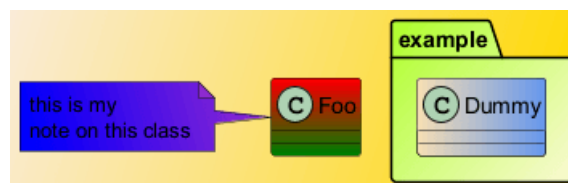
```
@startuml

skinparam backgroundColor AntiqueWhite/Gold
skinparam classBackgroundColor Wheat|CornflowerBlue

class Foo #red-green
note left of Foo #blue\9932CC
    this is my
    note on this class
end note

package example #GreenYellow/LightGoldenRodYellow {
    class Dummy
}

@enduml
```



3.32 Help on layout

Sometimes, the default layout is not perfect...

You can use `together` keyword to group some classes together : the layout engine will try to group them (as if they were in the same package).

You can also use `hidden` links to force the layout.

```
@startuml

class Bar1
class Bar2
together {
    class Together1
    class Together2
    class Together3
}


```

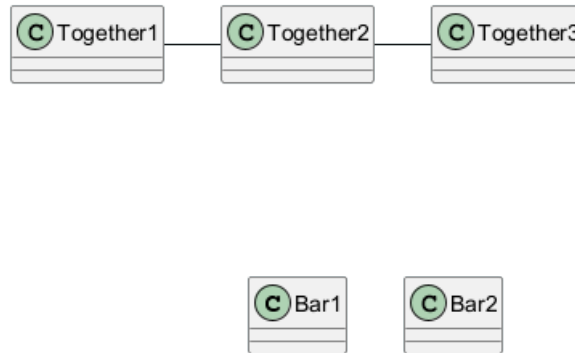


```

Together1 - Together2
Together2 - Together3
Together2 -[hidden]--> Bar1
Bar1 -[hidden]> Bar2

```

```
@enduml
```



3.33 Splitting large files

Sometimes, you will get some very large image files.

You can use the `page (hpages)x(vpages)` command to split the generated image into several files :

`hpages` is a number that indicated the number of horizontal pages, and `vpages` is a number that indicated the number of vertical pages.

You can also use some specific `skinparam` settings to put borders on splitted pages (see example).

```

@startuml
' Split into 4 pages
page 2x2
skinparam pageMargin 10
skinparam pageExternalColor gray
skinparam pageBorderColor black

class BaseClass

namespace net.dummy #DDDDDD {
    .BaseClass <|-- Person
    Meeting o-- Person

    .BaseClass <|-- Meeting
}

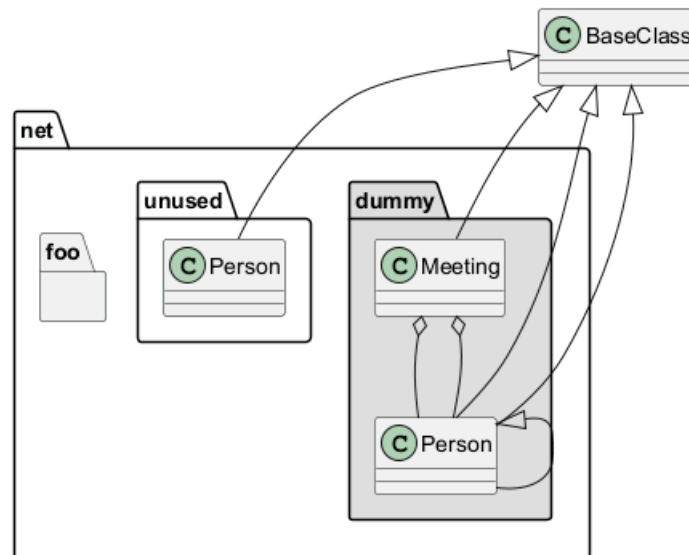
namespace net.foo {
    net.dummy.Person <|-- Person
    .BaseClass <|-- Person

    net.dummy.Meeting o-- Person
}

BaseClass <|-- net.unused.Person
@enduml

```

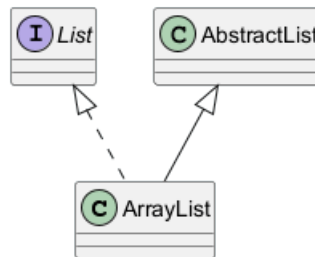




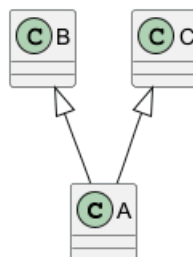
3.34 Extends and implements

It is also possible to use `extends` and `implements` keywords.

```
@startuml
class ArrayList implements List
class ArrayList extends AbstractList
@enduml
```



```
@startuml
class A extends B, C {
}
@enduml
```



[Ref. QA-2239]

3.35 Bracketed relations (linking or arrow) style

3.35.1 Line style

It's also possible to have explicitly `bold`, `dashed`, `dotted`, `hidden` or `plain` relation, links or arrows:

- without label

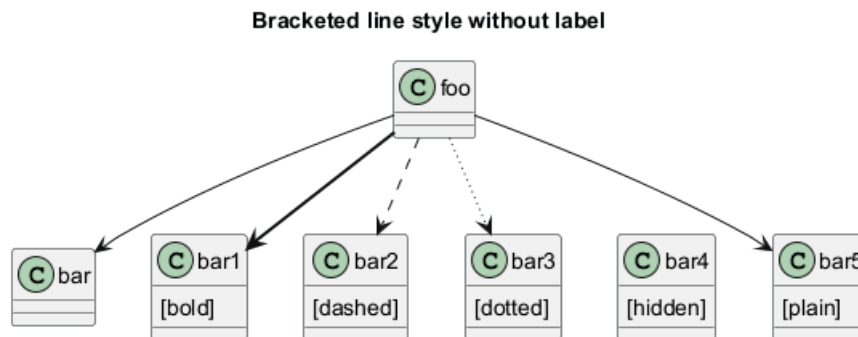


```

@startuml
title Bracketed line style without label
class foo
class bar
bar1 : [bold]
bar2 : [dashed]
bar3 : [dotted]
bar4 : [hidden]
bar5 : [plain]

foo --> bar
foo -[bold]-> bar1
foo -[dashed]-> bar2
foo -[dotted]-> bar3
foo -[hidden]-> bar4
foo -[plain]-> bar5
@enduml

```



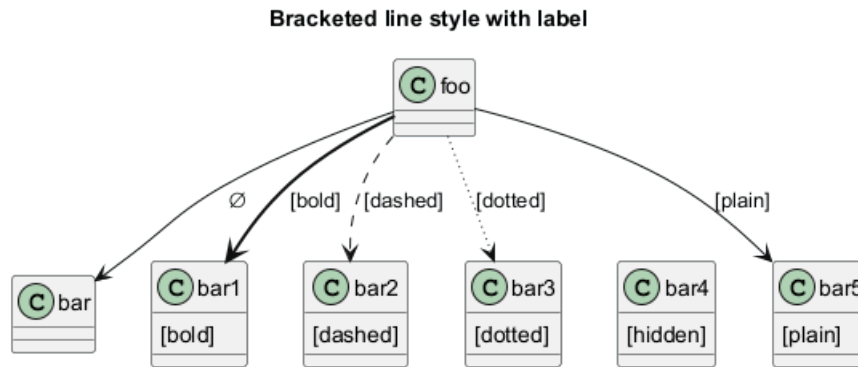
- with label

```

@startuml
title Bracketed line style with label
class foo
class bar
bar1 : [bold]
bar2 : [dashed]
bar3 : [dotted]
bar4 : [hidden]
bar5 : [plain]

foo --> bar :
foo -[bold]-> bar1 : [bold]
foo -[dashed]-> bar2 : [dashed]
foo -[dotted]-> bar3 : [dotted]
foo -[hidden]-> bar4 : [hidden]
foo -[plain]-> bar5 : [plain]
@enduml

```

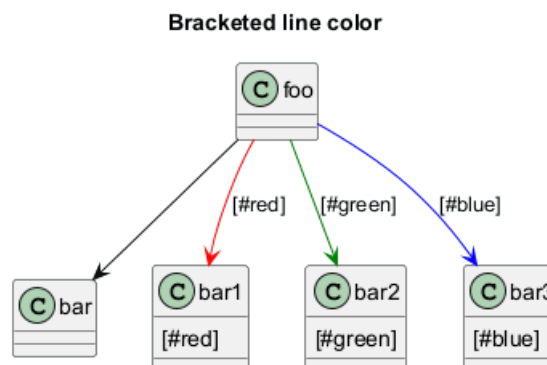


[Adapted from QA-4181]

3.35.2 Line color

```
@startuml
title Bracketed line color
class foo
class bar
bar1 : [#red]
bar2 : [#green]
bar3 : [#blue]

foo --> bar
foo -[#red]-> bar1 : [#red]
foo -[#green]-> bar2 : [#green]
foo -[#blue]-> bar3 : [#blue]
'foo -[#blue;#yellow;#green]-> bar4
@enduml
```



3.35.3 Line thickness

```
@startuml
title Bracketed line thickness
class foo
class bar
bar1 : [thickness=1]
bar2 : [thickness=2]
bar3 : [thickness=4]
bar4 : [thickness=8]
bar5 : [thickness=16]

foo --> bar :
foo -[thickness=1]-> bar1 : [1]
```

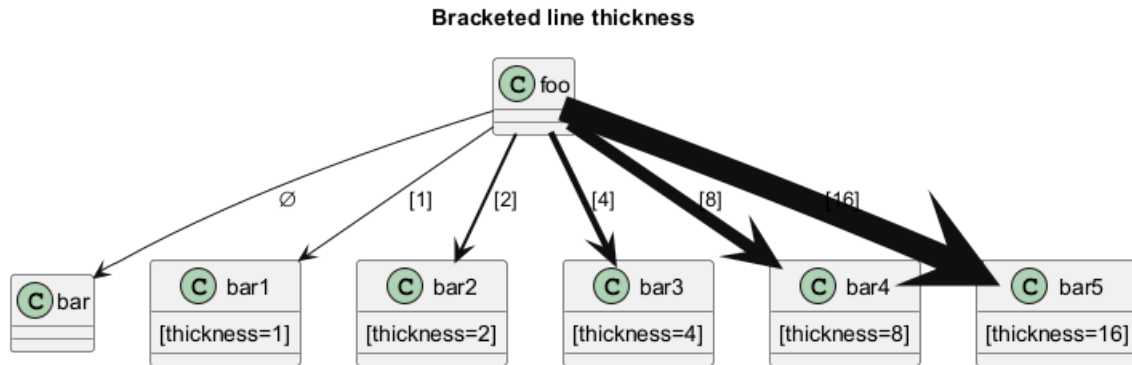


```

foo -[thickness=2]-> bar2 : [2]
foo -[thickness=4]-> bar3 : [4]
foo -[thickness=8]-> bar4 : [8]
foo -[thickness=16]-> bar5 : [16]

```

```
@enduml
```



[Ref. QA-4949]

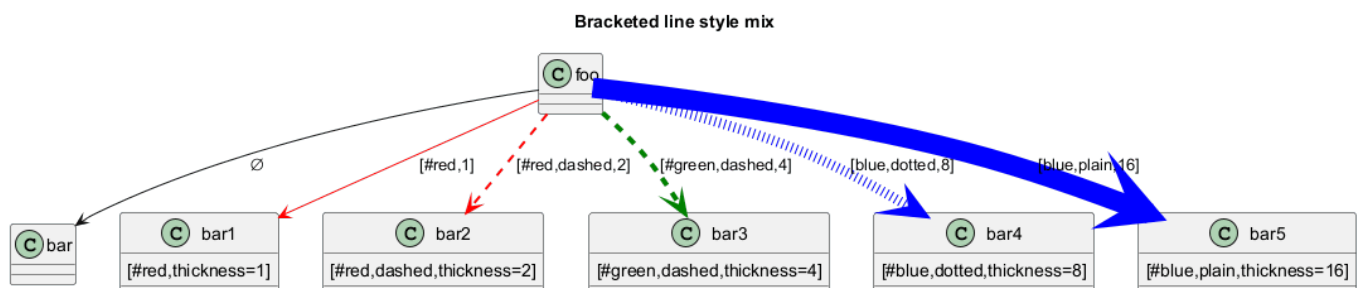
3.35.4 Mix

```

@startuml
title Bracketed line style mix
class foo
class bar
bar1 : [#red,thickness=1]
bar2 : [#red,dashed,thickness=2]
bar3 : [#green,dashed,thickness=4]
bar4 : [#blue,dotted,thickness=8]
bar5 : [#blue,plain,thickness=16]

foo --> bar :
foo -[#red,thickness=1]-> bar1 : [#red,1]
foo -[#red,dashed,thickness=2]-> bar2 : [#red,dashed,2]
foo -[#green,dashed,thickness=4]-> bar3 : [#green,dashed,4]
foo -[#blue,dotted,thickness=8]-> bar4 : [blue,dotted,8]
foo -[#blue,plain,thickness=16]-> bar5 : [blue,plain,16]
@enduml

```



3.36 Change relation (linking or arrow) color and style (inline style)

You can change the color or style of individual relation or arrows using the inline following notation:

- #color;line.[bold|dashed|dotted];text:color

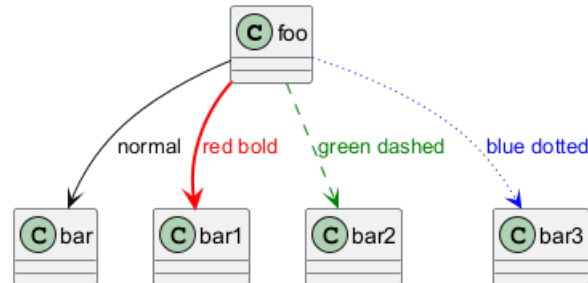
```
@startuml
```



```

class foo
foo --> bar : normal
foo --> bar1 #line:red;line.bold;text:red : red bold
foo --> bar2 #green;line.dashed;text:green : green dashed
foo --> bar3 #blue;line.dotted;text:blue : blue dotted
@enduml

```



[See similar feature on deployment]

3.37 Change class color and style (inline style)

You can change the color or style of individual class using the two following notations:

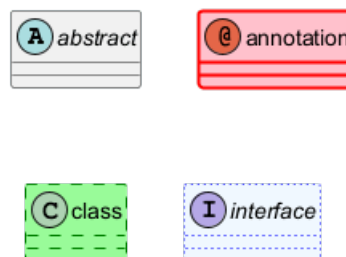
- #color ##[style]color

With background color first (#color), then line style and line color (##[style]color)

```

@startuml
abstract    abstract
annotation  annotation #pink ##[bold]red
class       class      #palegreen ##[dashed]green
interface   interface   #aliceblue ##[dotted]blue
@enduml

```



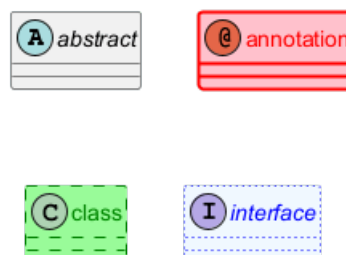
[Ref. QA-1487]

- #[color|back:color];header:color;line:color;line.[bold|dashed|dotted];text:color

```

@startuml
abstract    abstract
annotation  annotation #pink;line:red;line.bold;text:red
class       class      #palegreen;line:green;line.dashed;text:green
interface   interface   #aliceblue;line:blue;line.dotted;text:blue
@enduml

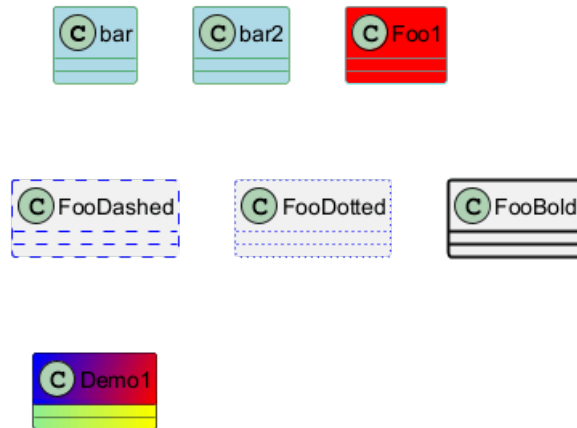
```



First original example:

```
@startuml
class bar #line:green;back:lightblue
class bar2 #lightblue;line:green

class Foo1 #back:red;line:00FFFF
class FooDashed #line.dashed:blue
class FooDotted #line.dotted:blue
class FooBold #line.bold
class Demo1 #back:lightgreen|yellow;header:blue/red
@enduml
```



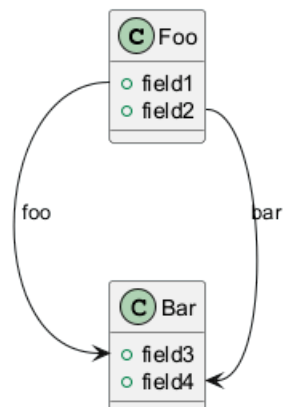
[Ref. QA-3770]

3.38 Arrows from/to class members

```
@startuml
class Foo {
+ field1
+ field2
}

class Bar {
+ field3
+ field4
}

Foo::field1 --> Bar::field3 : foo
Foo::field2 --> Bar::field4 : bar
@enduml
```



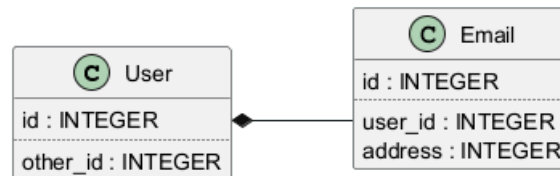
[Ref. QA-3636]

```
@startuml
left to right direction

class User {
    id : INTEGER
    ..
    other_id : INTEGER
}

class Email {
    id : INTEGER
    ..
    user_id : INTEGER
    address : INTEGER
}

User::id *-- Email::user_id
@enduml
```



[Ref. QA-5261]

3.39 Grouping inheritance arrow heads

You can merge all arrow heads using the skinparam `groupInheritance`, with a threshold as parameter.

3.39.1 GroupInheritance 1 (no grouping)

```
@startuml
skinparam groupInheritance 1

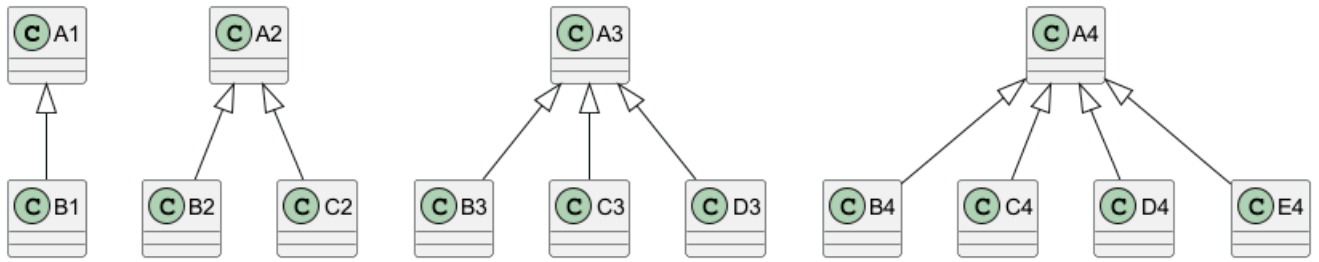
A1 <|-- B1

A2 <|-- B2
A2 <|-- C2

A3 <|-- B3
A3 <|-- C3
A3 <|-- D3

A4 <|-- B4
A4 <|-- C4
A4 <|-- D4
A4 <|-- E4
@enduml
```





3.39.2 GroupInheritance 2 (grouping from 2)

```

@startuml
skinparam groupInheritance 2

```

```

A1 <|-- B1

```

```

A2 <|-- B2

```

```

A2 <|-- C2

```

```

A3 <|-- B3

```

```

A3 <|-- C3

```

```

A3 <|-- D3

```

```

A4 <|-- B4

```

```

A4 <|-- C4

```

```

A4 <|-- D4

```

```

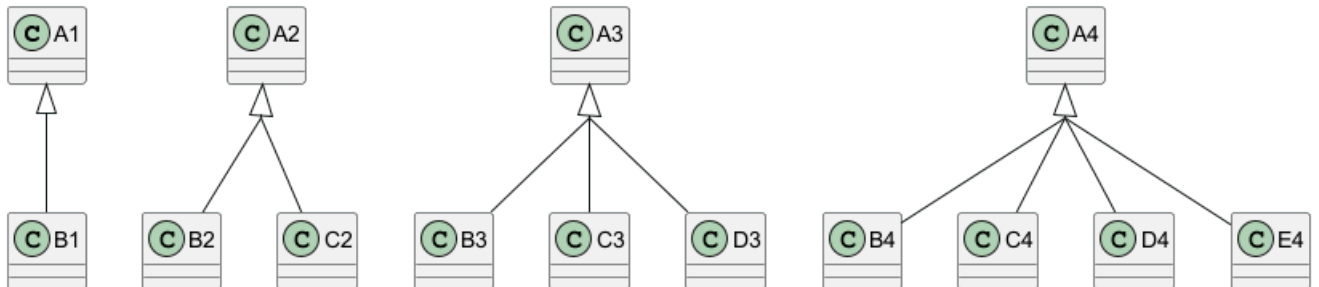
A4 <|-- E4

```

```

@enduml

```



3.39.3 GroupInheritance 3 (grouping only from 3)

```

@startuml
skinparam groupInheritance 3

```

```

A1 <|-- B1

```

```

A2 <|-- B2

```

```

A2 <|-- C2

```

```

A3 <|-- B3

```

```

A3 <|-- C3

```

```

A3 <|-- D3

```

```

A4 <|-- B4

```

```

A4 <|-- C4

```

```

A4 <|-- D4

```

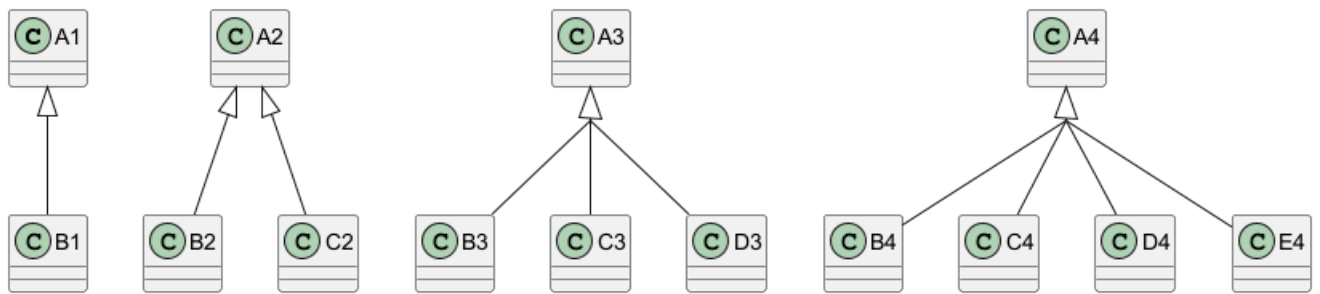
```

A4 <|-- E4

```



@enduml



3.39.4 GroupInheritance 4 (grouping only from 4)

@startuml

skinparam groupInheritance 4

A1 <|-- B1

A2 <|-- B2

A2 <|-- C2

A3 <|-- B3

A3 <|-- C3

A3 <|-- D3

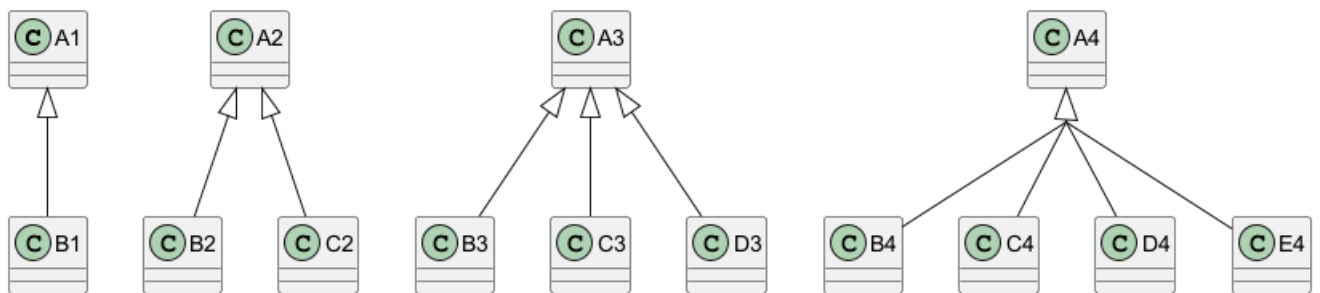
A4 <|-- B4

A4 <|-- C4

A4 <|-- D4

A4 <|-- E4

@enduml



[Ref. QA-3193, and Defect QA-13532]

3.40 Display JSON Data on Class or Object diagram

3.40.1 Simple example

@startuml

class Class

object Object

json JSON {

 "fruit": "Apple",

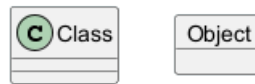
 "size": "Large",

 "color": ["Red", "Green"]

}

@enduml





JSON	
fruit	Apple
size	Large
color	Red
	Green

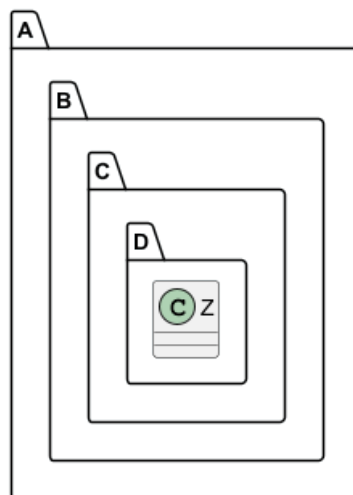
[Ref. QA-15481]

For another example, see on JSON page.

3.41 Packages and Namespaces Enhancement

[From V1.2023.2+, and V1.2023.5]

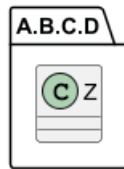
```
@startuml
class A.B.C.D.Z {
}
@enduml
```



```
@startuml
set separator none
class A.B.C.D.Z {
}
@enduml
```

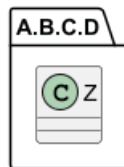


```
@startuml
!pragma useIntermediatePackages false
class A.B.C.D.Z {
}
@enduml
```



```

@startuml
set separator none
package A.B.C.D {
    class Z {
    }
}
@enduml
    
```



[Ref. GH-1352]

3.42 Qualified associations

3.42.1 Minimal example

```

@startuml
class class1
class class2

class1 [Qualifier] - class2
@enduml
    
```



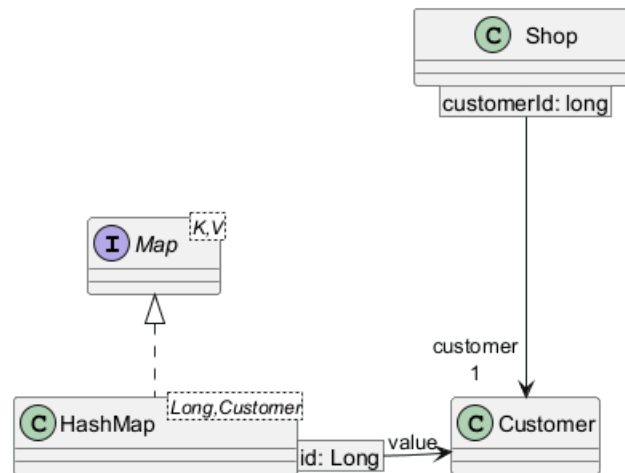
[Ref. QA-16397, GH-1467]

3.42.2 Another example

```

@startuml
    interface Map<K,V>
    class HashMap<Long, Customer>

    Map <|.. HashMap
    Shop [customerId: long] ---> "customer\n1" Customer
    HashMap [id: Long] -r-> "value" Customer
@enduml
    
```



3.43 Change diagram orientation

You can change (whole) diagram orientation with:

- top to bottom direction (*by default*)
- left to right direction

3.43.1 Top to bottom (*by default*)

3.43.2 With Graphviz (*layout engine by default*)

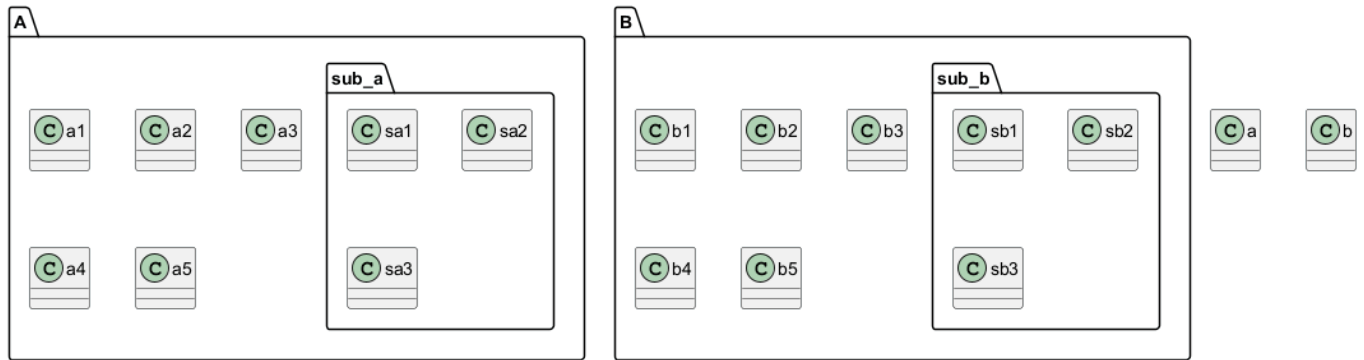
The main rule is: **Nested element first, then simple element.**

```

@startuml
class a
class b
package A {
  class a1
  class a2
  class a3
  class a4
  class a5
  package sub_a {
    class sa1
    class sa2
    class sa3
  }
}

package B {
  class b1
  class b2
  class b3
  class b4
  class b5
  package sub_b {
    class sb1
    class sb2
    class sb3
  }
}
@enduml
  
```



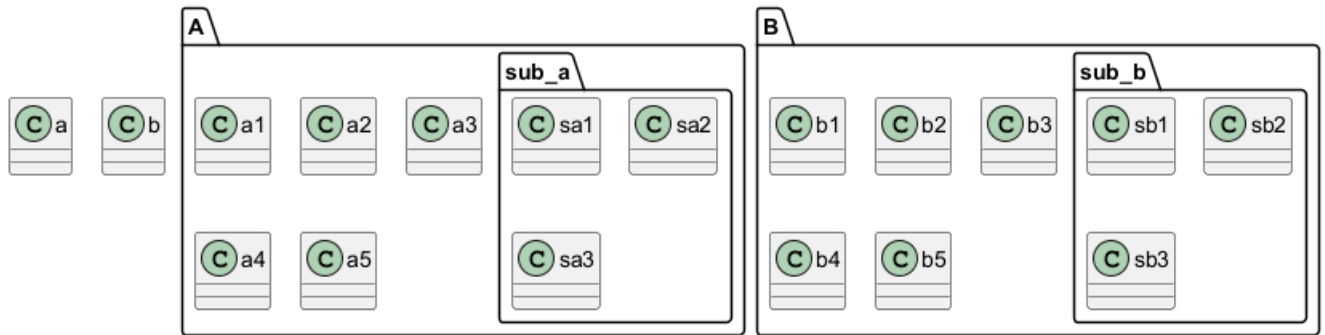


3.43.3 With Smetana (*internal layout engine*)

The main rule is the opposite: **Simple element first, then nested element.**

```
@startuml
!pragma layout smetana
class a
class b
package A {
  class a1
  class a2
  class a3
  class a4
  class a5
  package sub_a {
    class sa1
    class sa2
    class sa3
  }
}

package B {
  class b1
  class b2
  class b3
  class b4
  class b5
  package sub_b {
    class sb1
    class sb2
    class sb3
  }
}
@enduml
```

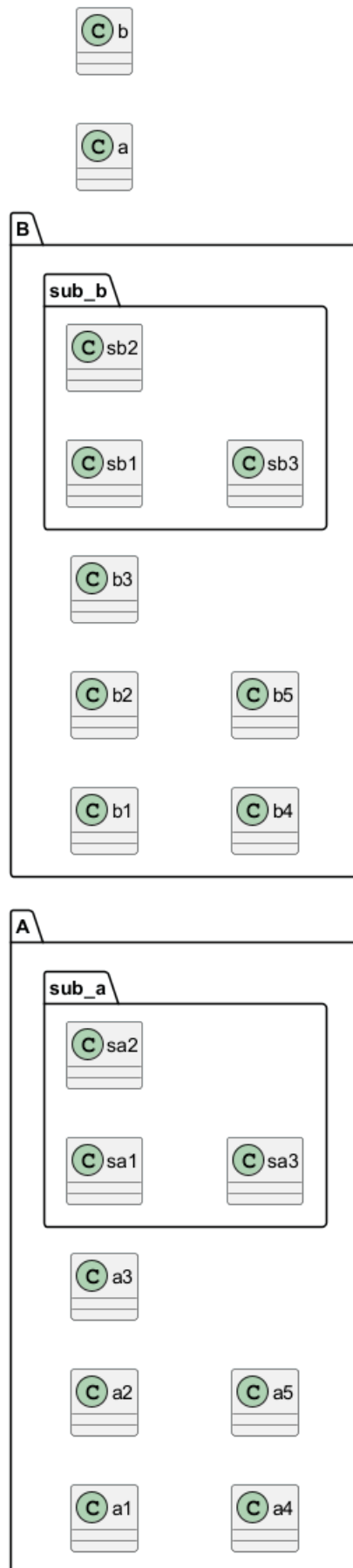


3.43.4 Left to right

3.43.5 With Graphviz (*layout engine by default*)

```
@startuml
left to right direction
class a
class b
package A {
    class a1
    class a2
    class a3
    class a4
    class a5
    package sub_a {
        class sa1
        class sa2
        class sa3
    }
}

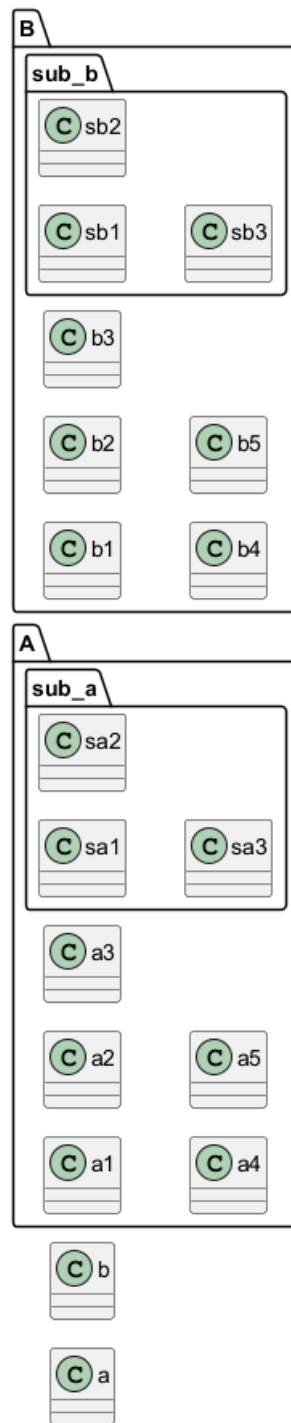
package B {
    class b1
    class b2
    class b3
    class b4
    class b5
    package sub_b {
        class sb1
        class sb2
        class sb3
    }
}
@enduml
```



3.43.6 With Smetana (*internal layout engine*)

```
@startuml
!pragma layout smetana
left to right direction
class a
class b
package A {
    class a1
    class a2
    class a3
    class a4
    class a5
    package sub_a {
        class sa1
        class sa2
        class sa3
    }
}

package B {
    class b1
    class b2
    class b3
    class b4
    class b5
    package sub_b {
        class sb1
        class sb2
        class sb3
    }
}
@enduml
```

4 Object Diagram

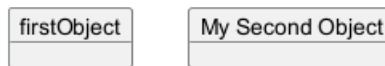
An **object diagram** is a graphical representation that showcases objects and their relationships at a specific moment in time. It provides a snapshot of the system's structure, capturing the static view of the instances present and their associations.

PlantUML offers a simple and intuitive way to create object diagrams using plain text. Its user-friendly syntax allows for quick diagram creation without the need for complex GUI tools. Moreover, the PlantUML forum provides a platform for users to discuss, share, and seek assistance, fostering a collaborative community. By choosing PlantUML, users benefit from both the efficiency of markdown-based diagramming and the support of an active community.

4.1 Definition of objects

You define instances of objects using the **object** keyword.

```
@startuml
object firstObject
object "My Second Object" as o2
@enduml
```



4.2 Relations between objects

Relations between objects are defined using the following symbols :

Type	Symbol	Purpose
Extension	< --	Specialization of a class in a hierarchy
Implementation	< ..	Realization of an interface by a class
Composition	*--	The part cannot exist without the whole
Aggregation	o--	The part can exist independently of the whole
Dependency	-->	The object uses another object
Dependency	..>	A weaker form of dependency

It is possible to replace -- by .. to have a dotted line.

Knowing those rules, it is possible to draw the following drawings.

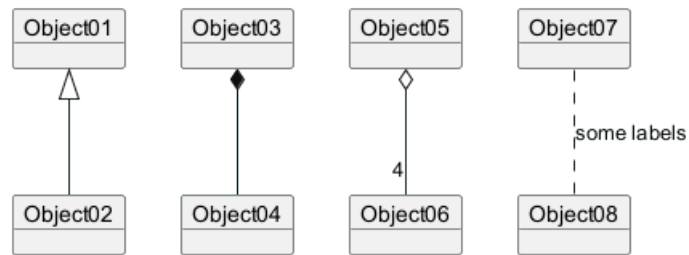
It is possible to add a label on the relation, using : followed by the text of the label.

For cardinality, you can use double-quotes "" on each side of the relation.

```
@startuml
object Object01
object Object02
object Object03
object Object04
object Object05
object Object06
object Object07
object Object08

Object01 <|-- Object02
Object03 *-- Object04
Object05 o-- "4" Object06
Object07 .. Object08 : some labels
@enduml
```





4.3 Associations objects

```

@startuml
object o1
object o2
diamond dia
object o3

o1 --> dia
o2 --> dia
dia --> o3
@enduml

```



4.4 Adding fields

To declare fields, you can use the symbol `:` followed by the field's name.

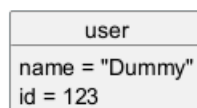
```

@startuml
object user

user : name = "Dummy"
user : id = 123

@enduml

```



It is also possible to group all fields between brackets `{}`.

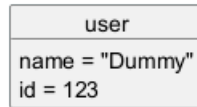
```

@startuml
object user {
  name = "Dummy"
  id = 123
}

```



```
@enduml
```



4.5 Common features with class diagrams

- Hide attributes, methods...
- Defines notes
- Use packages
- Skin the output

4.6 Map table or associative array

You can define a map table or associative array, with `map` keyword and `=>` separator.

```
@startuml
map CapitalCity {
  UK => London
  USA => Washington
  Germany => Berlin
}
@enduml
```

CapitalCity	
UK	London
USA	Washington
Germany	Berlin

```
@startuml
map "Map **Contry => CapitalCity**" as CC {
  UK => London
  USA => Washington
  Germany => Berlin
}
@enduml
```

Map Contry => CapitalCity	
UK	London
USA	Washington
Germany	Berlin

```
@startuml
map "map: Map<Integer, String>" as users {
  1 => Alice
  2 => Bob
  3 => Charlie
}
@enduml
```

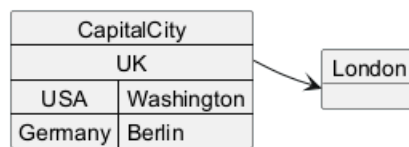


map: Map<Integer, String>	
1	Alice
2	Bob
3	Charlie

And add link with object.

```
@startuml
object London
```

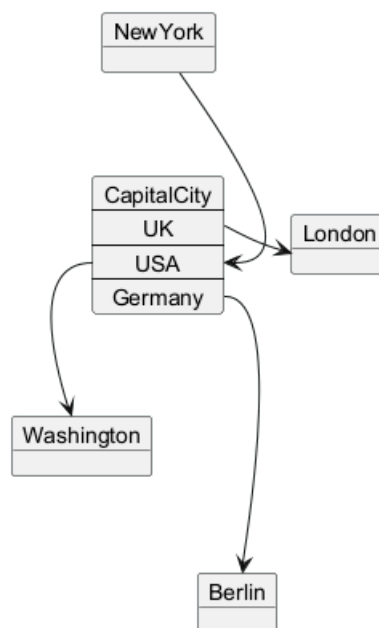
```
map CapitalCity {
  UK *-> London
  USA => Washington
  Germany => Berlin
}
@enduml
```



```
@startuml
object London
object Washington
object Berlin
object NewYork
```

```
map CapitalCity {
  UK *-> London
  USA *--> Washington
  Germany *---> Berlin
}
}
```

```
NewYork --> CapitalCity::USA
@enduml
```



[Ref. #307]



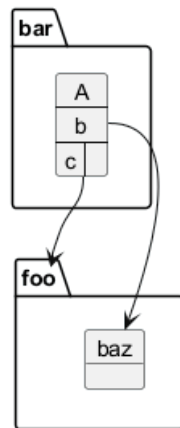
```

@startuml
package foo {
    object baz
}

package bar {
    map A {
        b *-> foo.baz
        c =>
    }
}

A::c --> foo
@enduml

```



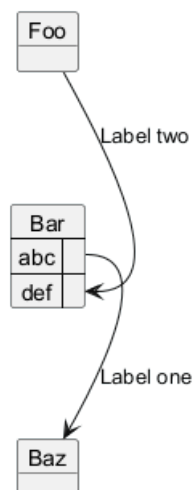
[Ref. QA-12934]

```

@startuml
object Foo
map Bar {
    abc=>
    def=>
}
object Baz

Bar::abc --> Baz : Label one
Foo --> Bar::def : Label two
@enduml

```



[Ref. #307]

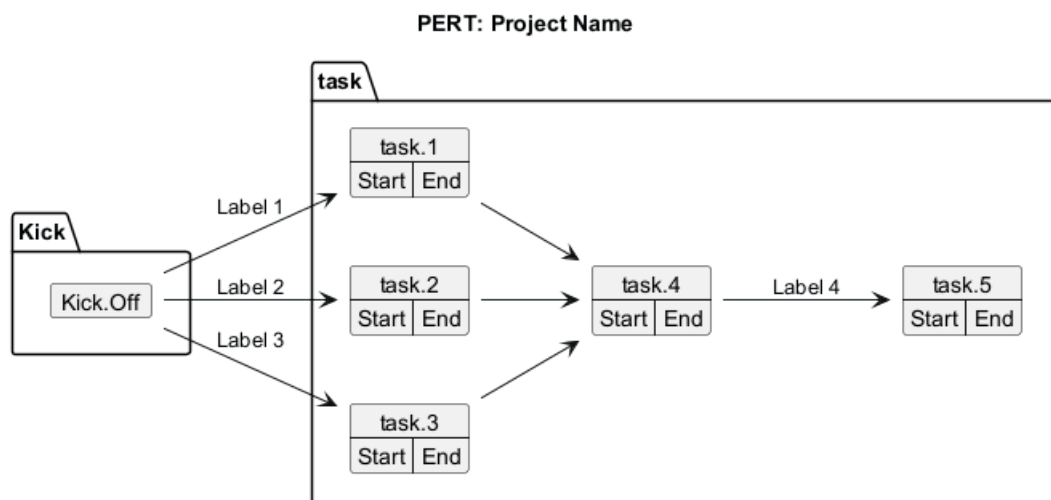
4.7 Program (or project) evaluation and review technique (PERT) with map

You can use `map` table in order to make Program (or project) evaluation and review technique (PERT) diagram.

```
@startuml PERT
left to right direction
' Horizontal lines: -->, <-->, <-->
' Vertical lines: ->, <-, <->
title PERT: Project Name

map Kick.Off {
}
map task.1 {
    Start => End
}
map task.2 {
    Start => End
}
map task.3 {
    Start => End
}
map task.4 {
    Start => End
}
map task.5 {
    Start => End
}

Kick.Off --> task.1 : Label 1
Kick.Off --> task.2 : Label 2
Kick.Off --> task.3 : Label 3
task.1 --> task.4
task.2 --> task.4
task.3 --> task.4
task.4 --> task.5 : Label 4
@enduml
```



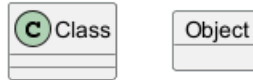
[Ref. QA-12337]



4.8 Display JSON Data on Class or Object diagram

4.8.1 Simple example

```
@startuml
class Class
object Object
json JSON {
    "fruit": "Apple",
    "size": "Large",
    "color": ["Red", "Green"]
}
@enduml
```



JSON	
fruit	Apple
size	Large
color	Red
	Green

[Ref. QA-15481]

For another example, see on JSON page.

5 Activity Diagram (legacy)

This is the old **Activity Diagram (legacy)** syntax, to see the new current version see: **Activity Diagram (new)**.

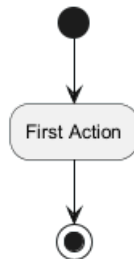
5.1 Simple Action

You can use (*) for the starting point and ending point of the activity diagram.

In some occasion, you may want to use (*top) to force the starting point to be at the top of the diagram.

Use --> for arrows.

```
@startuml
(*) --> "First Action"
"First Action" --> (*)
@enduml
```

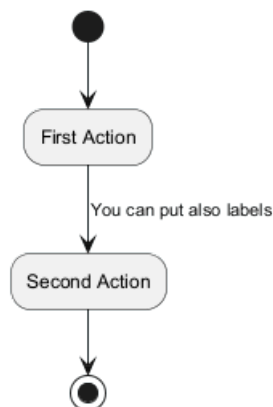


5.2 Label on arrows

By default, an arrow starts at the last used activity.

You can put a label on an arrow using brackets [and] just after the arrow definition.

```
@startuml
(*) --> "First Action"
-->[You can put also labels] "Second Action"
--> (*)
@enduml
```



5.3 Changing arrow direction

You can use -> for horizontal arrows. It is possible to force arrow's direction using the following syntax:

- -down-> (default arrow)

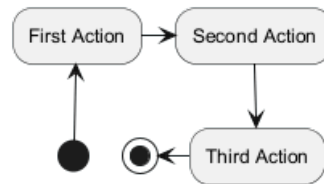


- -right-> or ->
- -left->
- -up->

```
@startuml
```

```
(*) -up-> "First Action"
-right-> "Second Action"
--> "Third Action"
-left-> (*)
```

```
@enduml
```



5.4 Branches

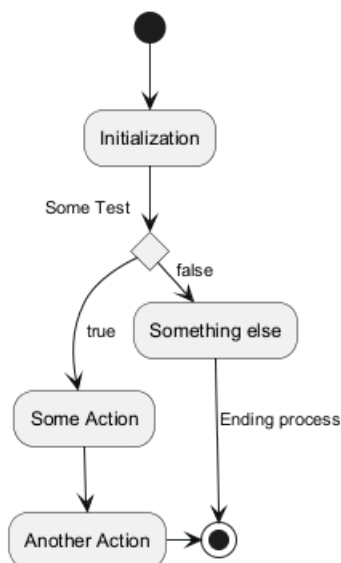
You can use if/then/else keywords to define branches.

```
@startuml
```

```
(*) --> "Initialization"
```

```
if "Some Test" then
  -->[true] "Some Action"
  --> "Another Action"
  -right-> (*)
else
  ->[false] "Something else"
  -->[Ending process] (*)
endif
```

```
@enduml
```



Unfortunately, you will have to sometimes repeat the same activity in the diagram text:

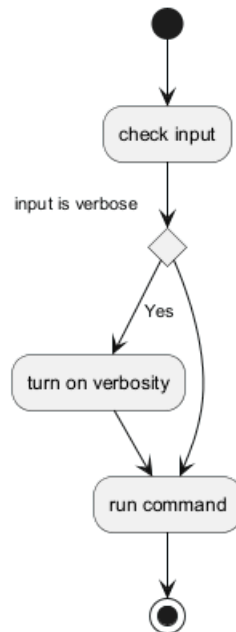
```
@startuml
```



```

(*) --> "check input"
If "input is verbose" then
--> [Yes] "turn on verbosity"
--> "run command"
else
--> "run command"
Endif
-->(*)
@enduml

```



5.5 More on Branches

By default, a branch is connected to the last defined activity, but it is possible to override this and to define a link with the `if` keywords.

It is also possible to nest branches.

```

@startuml
(*) --> if "Some Test" then
    -->[true] "action 1"
    if "" then
        -> "action 3" as a3
    else
        if "Other test" then
            -left-> "action 5"
        else
            --> "action 6"
        endif
    endif
endif
else
    ->[false] "action 2"
endif

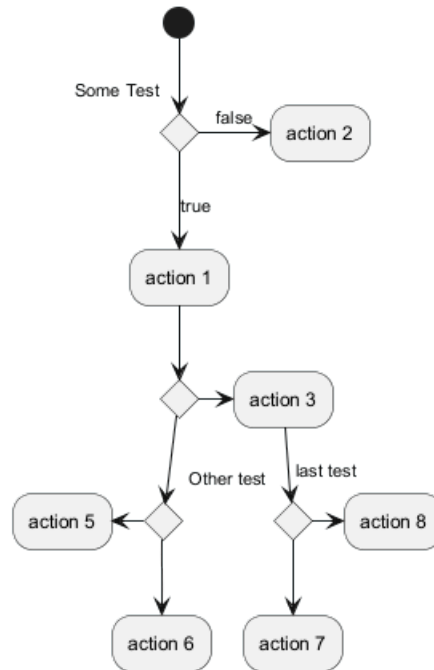
```



```

a3 --> if "last test" then
  --> "action 7"
else
  -> "action 8"
endif
@enduml

```



5.6 Synchronization

You can use `=== code ===` to display synchronization bars.

```

@startuml

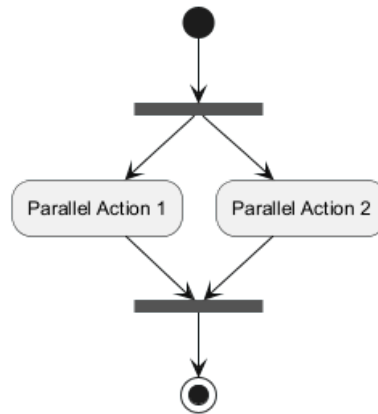
(*) --> ===B1===
--> "Parallel Action 1"
--> ===B2===

===B1=== --> "Parallel Action 2"
--> ===B2===

--> (*)

@enduml

```



5.7 Long action description

When you declare activities, you can span on several lines the description text. You can also add `\n` in the description.

You can also give a short code to the activity with the `as` keyword. This code can be used latter in the diagram description.

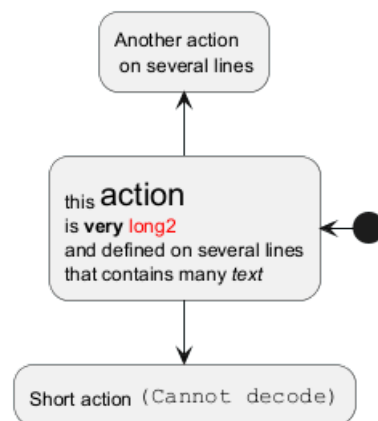
```

@startuml
(*) -left-> "this <size:20>action</size>
is <b>very</b> <color:red>long2</color>
and defined on several lines
that contains many <i>text</i>" as A1

-up-> "Another action\n on several lines"

A1 --> "Short action <img:sourceforge.jpg>"
@enduml

```



5.8 Notes

You can add notes on a activity using the commands `note left`, `note right`, `note top` or `note bottom`, just after the description of the activity you want to note.

If you want to put a note on the starting point, define the note at the very beginning of the diagram description.

You can also have a note on several lines, using the `endnote` keywords.

```

@startuml

(*) --> "Some action"
note right: This action has to be defined

```

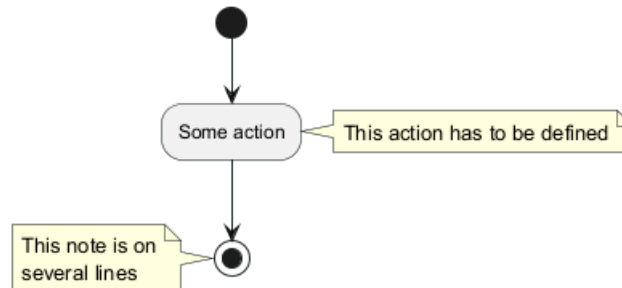


```

"Some action" --> (*)
note left
  This note is on
  several lines
end note

@enduml

```



5.9 Partition

You can define a partition using the **partition** keyword, and optionally declare a background color for your partition (Using a html color code or name)

When you declare activities, they are automatically put in the last used partition.

You can close the partition definition using a closing bracket **}**.

```

@startuml

partition Conductor {
  (*) --> "Climbs on Platform"
  --> === S1 ===
  --> Bows
}

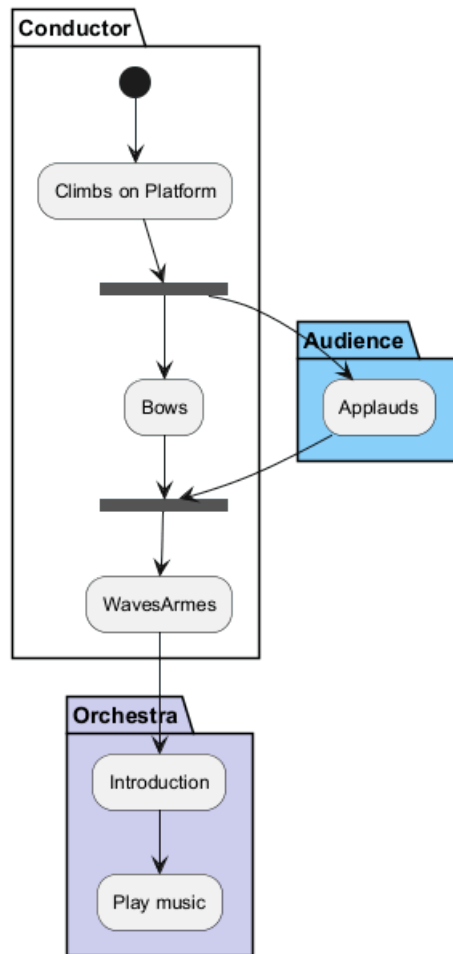
partition Audience #LightSkyBlue {
  === S1 === --> Applauds
}

partition Conductor {
  Bows --> === S2 ===
  --> WavesArmes
  Applauds --> === S2 ===
}

partition Orchestra #CCCCEE {
  WavesArmes --> Introduction
  --> "Play music"
}

@enduml

```



5.10 Skinparam

You can use the skinparam command to change colors and fonts for the drawing.

You can use this command :

- In the diagram definition, like any other commands,
- In an included file,
- In a configuration file, provided in the command line or the ANT task.

You can define specific color and fonts for stereotyped activities.

@startuml

```

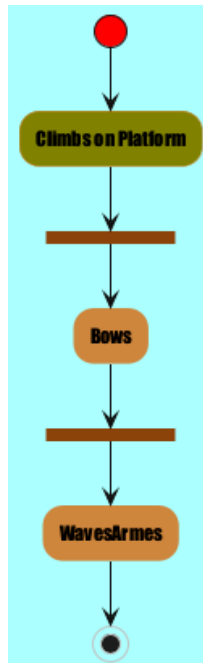
skinparam backgroundColor #AAFFFF
skinparam activity {
  StartColor red
  BarColor SaddleBrown
  EndColor Silver
  BackgroundColor Peru
  BackgroundColor<< Begin >> Olive
  BorderColor Peru
  FontName Impact
}

(*) --> "Climbs on Platform" << Begin >>
--> === S1 ===
--> Bows
  
```



```
--> === S2 ===
--> WavesArmes
--> (*)

@enduml
```



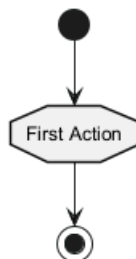
5.11 Octagon

You can change the shape of activities to octagon using the `skinparam activityShape octagon` command.

```
@startuml
'Default is skinparam activityShape roundBox
skinparam activityShape octagon

(*) --> "First Action"
"First Action" --> (*)

@enduml
```



5.12 Complete example

```
@startuml
title Servlet Container

(*) --> "ClickServlet.handleRequest()"
--> "new Page"

if "Page.onSecurityCheck" then
```




```

->[true] "Page.onInit()"

if "isForward?" then
  ->[no] "Process controls"

  if "continue processing?" then
    -->[yes] ===RENDERING===
  else
    -->[no] ===REDIRECT_CHECK===
  endif

else
  -->[yes] ===RENDERING===
endif

if "is Post?" then
  -->[yes] "Page.onPost()"
  --> "Page.onRender()" as render
  --> ===REDIRECT_CHECK===
else
  -->[no] "Page.onGet()"
  --> render
endif

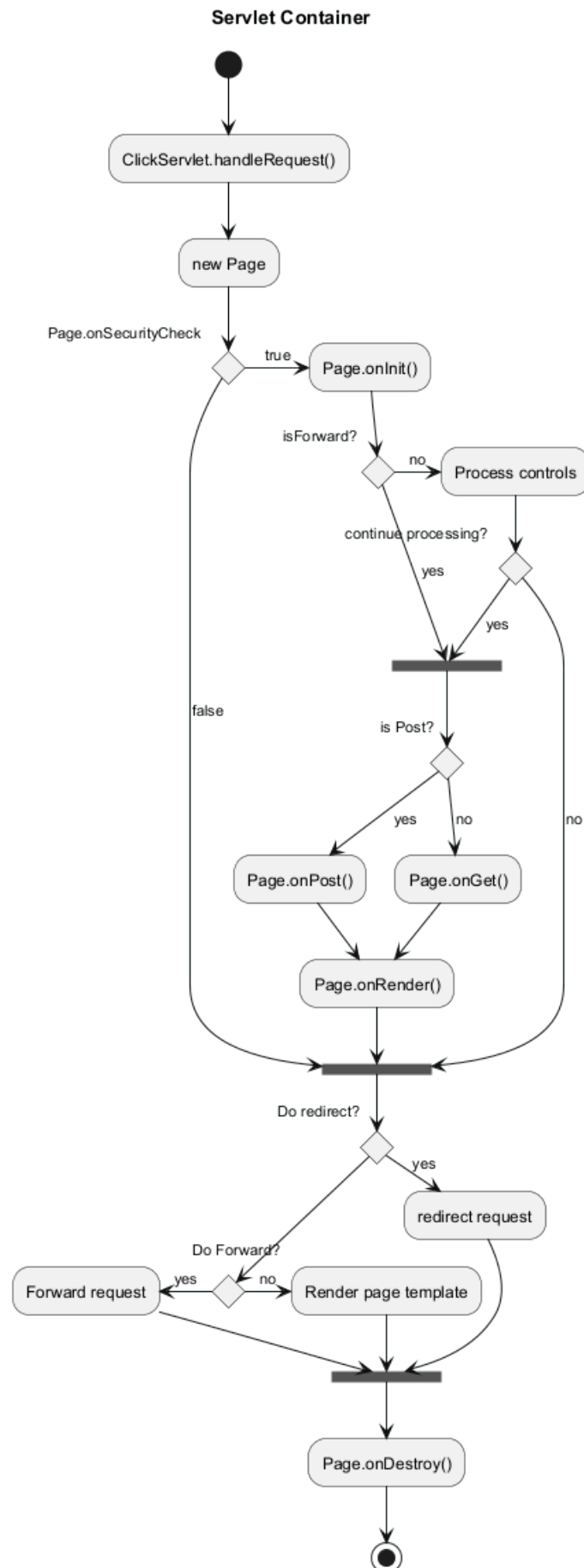
else
  -->[false] ===REDIRECT_CHECK===
endif

if "Do redirect?" then
  ->[yes] "redirect request"
  --> ==BEFORE_DESTROY==
else
  if "Do Forward?" then
    -left->[yes] "Forward request"
    --> ==BEFORE_DESTROY==
  else
    -right->[no] "Render page template"
    --> ==BEFORE_DESTROY==
  endif
endif

--> "Page.onDestroy()"
-->(*)

@enduml

```



6 Activity Diagram (New Syntax)

The previous syntax used for activity diagrams encountered several limitations and maintainability issues. Recognizing these drawbacks, we have introduced a wholly revamped syntax and implementation that is not only user-friendly but also more stable.

6.0.1 Benefits of the New Syntax

- **No Dependency on Graphviz:** Just like with sequence diagrams, the new syntax eliminates the necessity for Graphviz installation, thereby simplifying the setup process.
- **Ease of Maintenance:** The intuitive nature of the new syntax means it is easier to manage and maintain your diagrams.

6.0.2 Transition to the New Syntax

While we will continue to support the old syntax to maintain compatibility, we highly encourage users to migrate to the new syntax to leverage the enhanced features and benefits it offers.

Make the shift today and experience a more streamlined and efficient diagramming process with the new activity diagram syntax.

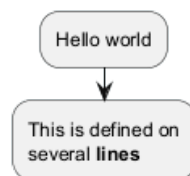
6.1 Simple action

Activities label starts with `:` and ends with `;`.

Text formatting can be done using creole wiki syntax.

They are implicitly linked in their definition order.

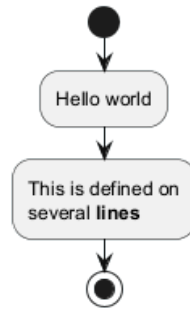
```
@startuml
:Hello world;
:This is defined on
several lines;
@enduml
```



6.2 Start/Stop/End

You can use `start` and `stop` keywords to denote the beginning and the end of a diagram.

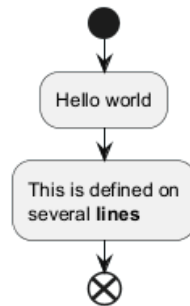
```
@startuml
start
:Hello world;
:This is defined on
several lines;
stop
@enduml
```



You can also use the **end** keyword.

```

@startuml
start
:Hello world;
:This is defined on
several **lines**;
end
@enduml
  
```



6.3 Conditional

You can use **if**, **then**, **else** and **endif** keywords to put tests in your diagram. Labels can be provided using parentheses.

The 3 syntaxes are possible:

- **if** (...) **then** (...) ... [**else** (...) ...] **endif**

```

@startuml

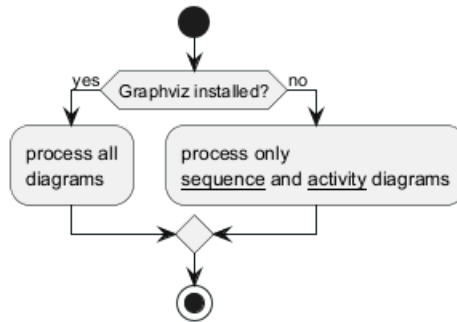
start

if (Graphviz installed?) then (yes)
  :process all\ndiagrams;
else (no)
  :process only
  __sequence__ and __activity__ diagrams;
endif

stop

@enduml
  
```

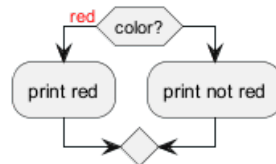




- if (...) is (...) then ... [else (...) ...] endif

```

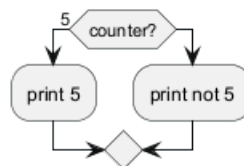
@startuml
if (color?) is (<color:red>red) then
:print red;
else
:print not red;
endif
@enduml
  
```



- if (...) equals (...) then ... [else (...) ...] endif

```

@startuml
if (counter?) equals (5) then
:print 5;
else
:print not 5;
endif
@enduml
  
```



[Ref. QA-301]

6.3.1 Several tests (horizontal mode)

You can use the `elseif` keyword to have several tests (*by default, it is the horizontal mode*):

```

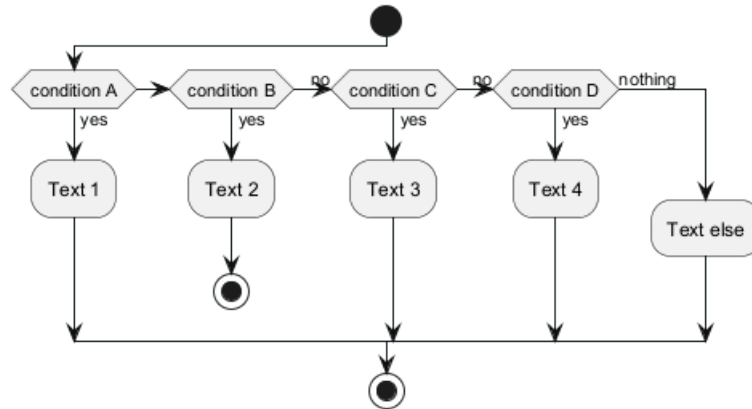
@startuml
start
if (condition A) then (yes)
:Text 1;
elseif (condition B) then (yes)
:Text 2;
stop
(no) elseif (condition C) then (yes)
:Text 3;
(no) elseif (condition D) then (yes)
  
```



```

:Text 4;
else (nothing)
:Text else;
endif
stop
@enduml

```



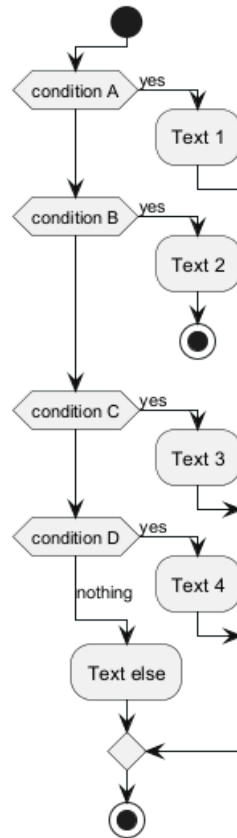
6.3.2 Several tests (vertical mode)

You can use the command `!pragma useVerticalIf on` to have the tests in vertical mode:

```

@startuml
!pragma useVerticalIf on
start
if (condition A) then (yes)
:Text 1;
elseif (condition B) then (yes)
:Text 2;
stop
elseif (condition C) then (yes)
:Text 3;
elseif (condition D) then (yes)
:Text 4;
else (nothing)
:Text else;
endif
stop
@enduml

```



You can use the `-P` command-line option to specify the pragma:

```
java -jar plantuml.jar -PuseVerticalIf=on
```

[Refs. [QA-3931](#), [GH-582](#)]

6.4 Switch and case [switch, case, endswitch]

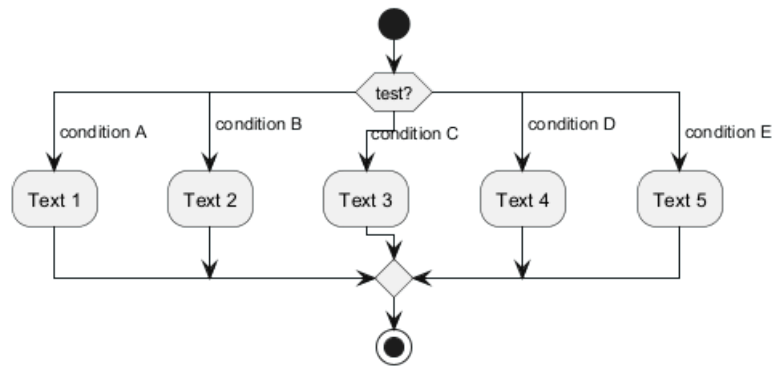
You can use `switch`, `case` and `endswitch` keywords to put switch in your diagram.

Labels can be provided using parentheses.

```

@startuml
start
switch (test?)
case ( condition A )
  :Text 1;
case ( condition B )
  :Text 2;
case ( condition C )
  :Text 3;
case ( condition D )
  :Text 4;
case ( condition E )
  :Text 5;
endswitch
stop
@enduml
  
```



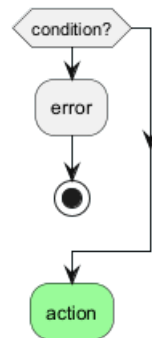


6.5 Conditional with stop on an action [kill, detach]

You can stop action on a if loop.

```

@startuml
if (condition?) then
    :error;
    stop
endif
#palegreen:action;
@enduml
  
```

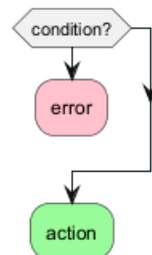


But if you want to stop at the precise action, you can use the **kill** or **detach** keyword:

- **kill**

```

@startuml
if (condition?) then
    #pink:error;
    kill
endif
#palegreen:action;
@enduml
  
```



[Ref. QA-265]

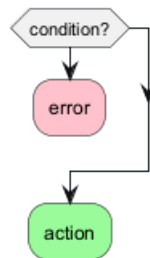
- **detach**




```

@startuml
if (condition?) then
  #pink:error;
  detach
endif
#palegreen:action;
@enduml

```



6.6 Repeat loop

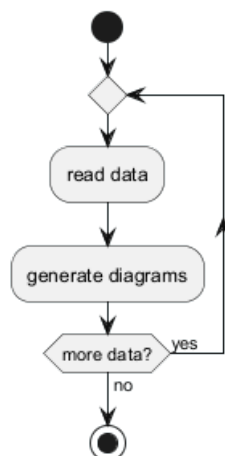
6.6.1 Simple repeat loop

You can use `repeat` and `repeat while` keywords to have repeat loops.

```

@startuml
start
repeat
  :read data;
  :generate diagrams;
repeat while (more data?) is (yes) not (no)
stop
@enduml

```



6.6.2 Repeat loop with repeat action and backward action

It is also possible to use a full action as `repeat` target and insert an action in the return path using the `backward` keyword.

```

@startuml
start

```



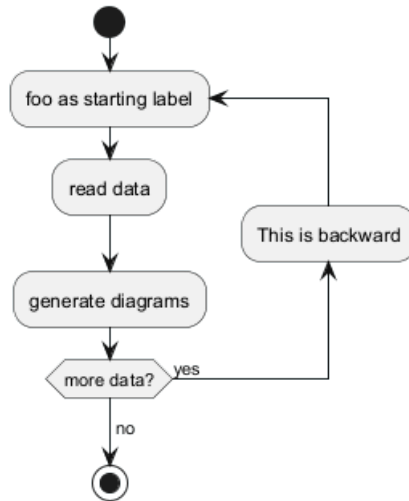
```

repeat :foo as starting label;
  :read data;
  :generate diagrams;
backward:This is backward;
repeat while (more data?) is (yes)
->no;

stop

@enduml

```



[Ref. QA-5826]

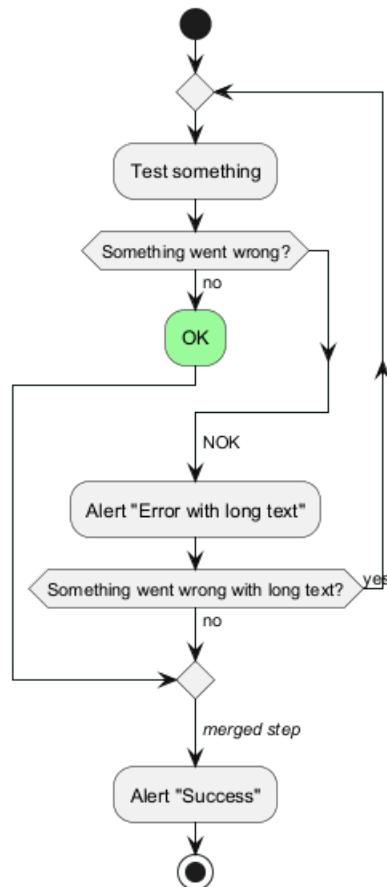
6.7 Break on a repeat loop [break]

You can use the **break** keyword after an action on a loop.

```

@startuml
start
repeat
  :Test something;
  if (Something went wrong?) then (no)
    #palegreen:OK;
    break
  endif
->NOK;
  :Alert "Error with long text";
repeat while (Something went wrong with long text?) is (yes) not (no)
->//merged step//;
:Alert "Success";
stop
@enduml

```



[Ref. QA-6105]

6.8 Goto and Label Processing [label, goto]

It is currently only experimental

You can use `label` and `goto` keywords to denote goto processing, with:

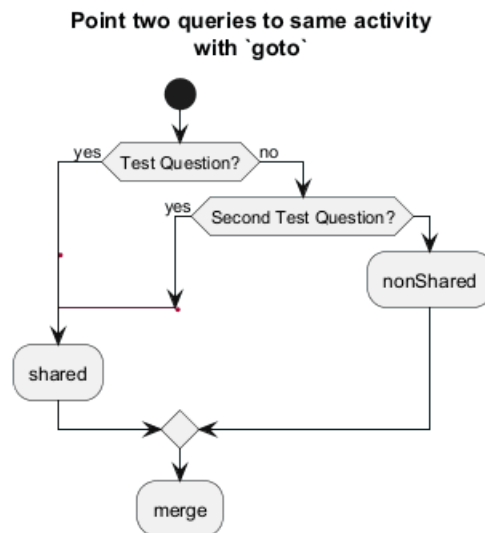
- `label <label_name>`
- `goto <label_name>`

```

@startuml
title Point two queries to same activity\nwith `goto`
start
if (Test Question?) then (yes)
'space label only for alignment
label sp_lab0
label sp_lab1
'real label
label lab
:shared;
else (no)
if (Second Test Question?) then (yes)
label sp_lab2
goto sp_lab1
else
:nonShared;
endif
endif
:merge;
  
```



@enduml



[Ref. QA-15026, QA-12526 and initially QA-1626]

6.9 While loop

6.9.1 Simple while loop

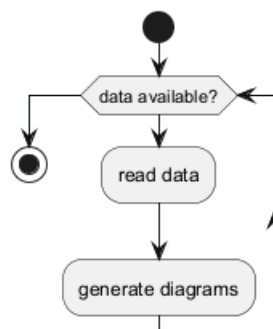
You can use `while` and `endwhile` keywords to have while loop.

```

@startuml
start

while (data available?)
    :read data;
    :generate diagrams;
endwhile

stop
@enduml
  
```



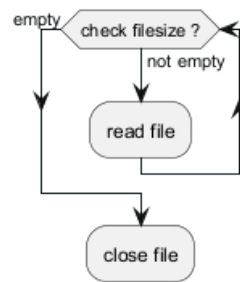
It is possible to provide a label after the `endwhile` keyword, or using the `is` keyword.

```

@startuml
while (check filesize ?) is (not empty)
    :read file;
endwhile (empty)
:close file;
  
```



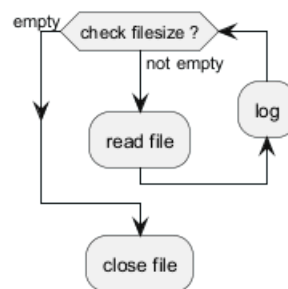
```
@enduml
```



6.9.2 While loop with backward action

It is also possible to insert an action in the return path using the **backward** keyword.

```
@startuml
while (check filesize ?) is (not empty)
  :read file;
  backward:log;
endwhile (empty)
:close file;
@enduml
```



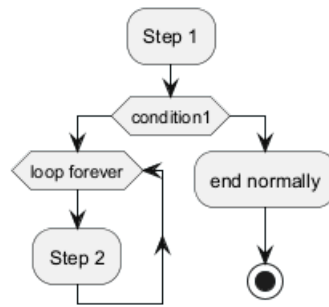
[Ref. QA-11144]

6.9.3 Infinite while loop

If you are using **detach** to form an infinite while loop, then you will want to also hide the partial arrow that results using **-[hidden]->**

```
@startuml
:Step 1;
if (condition1) then
  while (loop forever)
    :Step 2;
  endwhile
  -[hidden]->
  detach
else
  :end normally;
  stop
endif
@enduml
```





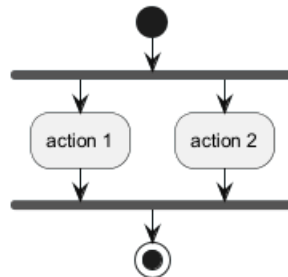
6.10 Parallel processing [fork, fork again, end fork, end merge]

You can use `fork`, `fork again` and `end fork` or `end merge` keywords to denote parallel processing.

6.10.1 Simple fork

```

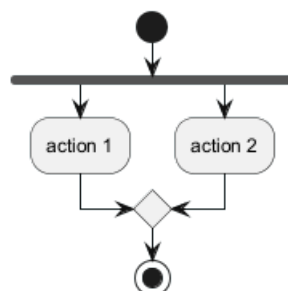
@startuml
start
fork
    :action 1;
fork again
    :action 2;
end fork
stop
@enduml
  
```



6.10.2 fork with end merge

```

@startuml
start
fork
    :action 1;
fork again
    :action 2;
end merge
stop
@enduml
  
```

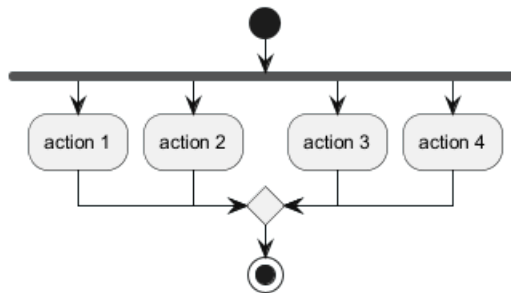


[Ref. QA-5320]

```

@startuml
start
fork
    :action 1;
fork again
    :action 2;
fork again
    :action 3;
fork again
    :action 4;
end merge
stop
@enduml

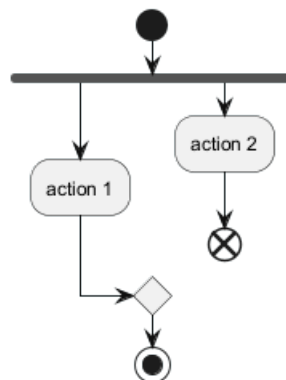
```



```

@startuml
start
fork
    :action 1;
fork again
    :action 2;
end
end merge
stop
@enduml

```



[Ref. QA-13731]

6.10.3 Label on end fork (or UML joinspec):

```

@startuml
start
fork
    :action A;
fork again

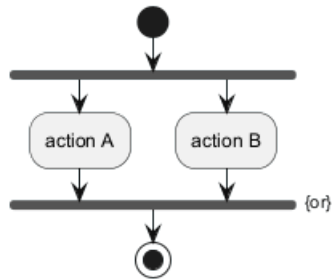
```



```

:action B;
end fork {or}
stop
@enduml

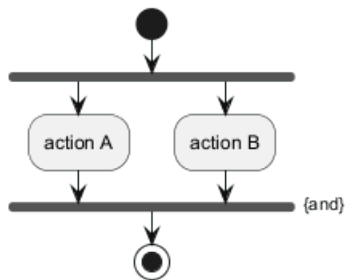
```



```

@startuml
start
fork
:action A;
fork again
:action B;
end fork {and}
stop
@enduml

```



[Ref. QA-5346]

6.10.4 Other example

```

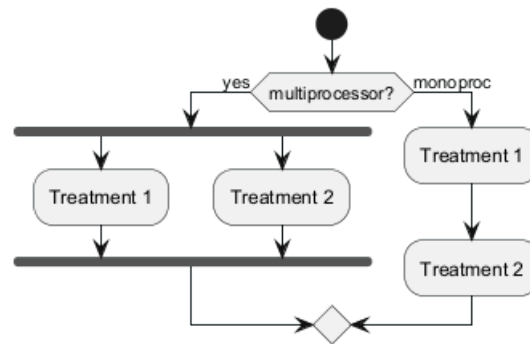
@startuml
start

if (multiprocessor?) then (yes)
    fork
        :Treatment 1;
    fork again
        :Treatment 2;
    end fork
else (monoproc)
    :Treatment 1;
    :Treatment 2;
endif

@enduml

```





6.11 Split processing

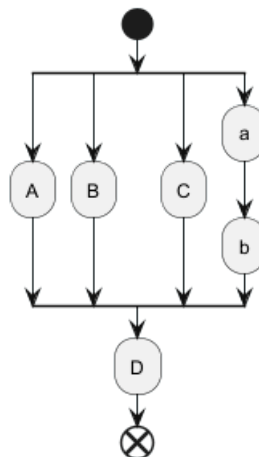
6.11.1 Split

You can use `split`, `split again` and `end split` keywords to denote split processing.

```

@startuml
start
split
:A;
split again
:B;
split again
:C;
split again
:a;
:b;
end split
:D;
end
@enduml

```



6.11.2 Input split (multi-start)

You can use `hidden` arrows to make an input split (multi-start):

```

@startuml
split
-[hidden]->
:A;
split again
-[hidden]->

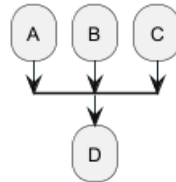
```



```

    :B;
split again
-[hidden]->
    :C;
end split
:D;
@enduml

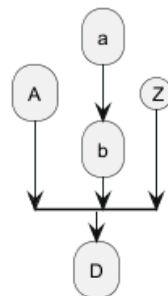
```



```

@startuml
split
-[hidden]->
    :A;
split again
-[hidden]->
    :a;
    :b;
split again
-[hidden]->
    (Z)
end split
:D;
@enduml

```



[Ref. QA-8662]

6.11.3 Output split (multi-end)

You can use `kill` or `detach` to make an output split (multi-end):

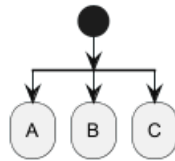
```

@startuml
start
split
    :A;
    kill
split again
    :B;
    detach
split again
    :C;
    kill

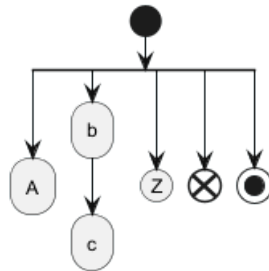
```



```
end split
@enduml
```



```
@startuml
start
split
  :A;
  kill
split again
  :b;
  :c;
  detach
split again
  (Z)
  detach
split again
  end
split again
  stop
end split
@enduml
```



6.12 Notes

Text formatting can be done using creole wiki syntax.

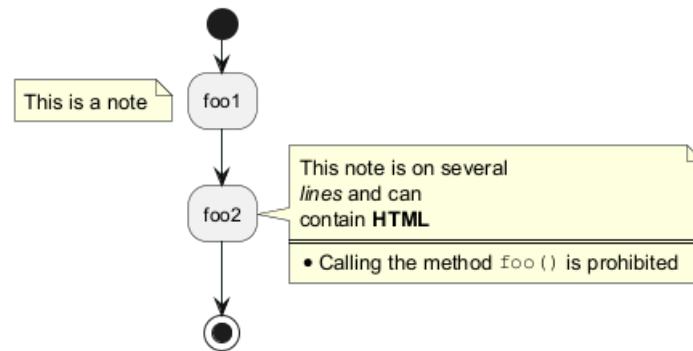
A note can be floating, using `floating` keyword.

```
@startuml

start
:foo1;
floating note left: This is a note
:foo2;
note right
  This note is on several
  //lines// and can
  contain <b>HTML</b>
  ====
  * Calling the method ""foo()"" is prohibited
end note
stop

@enduml
```

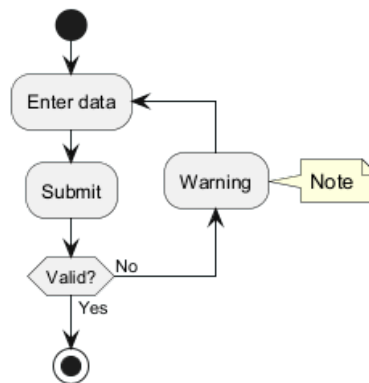




You can add note on backward activity:

```

@startuml
start
repeat :Enter data;
:Submit;
backward :Warning;
note right: Note
repeat while (Valid?) is (No) not (Yes)
stop
@enduml
  
```

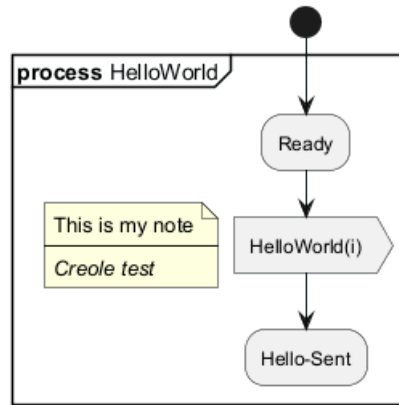


[Ref. QA-11788]

You can add note on partition activity:

```

@startuml
start
partition "**process** HelloWorld" {
    note
        This is my note
        ----
        //Creole test//
    end note
    :Ready;
    :HelloWorld(i)>
    :Hello-Sent;
}
@enduml
  
```



[Ref. QA-2398]

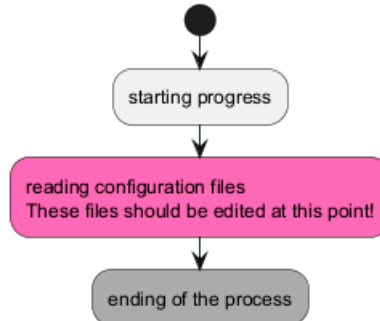
6.13 Colors

You can specify a color for some activities.

```
@startuml
```

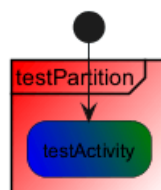
```
start
:starting progress;
#HotPink:reading configuration files
These files should be edited at this point!;
#AAAAAA:ending of the process;
```

```
@enduml
```



You can also use gradient color.

```
@startuml
start
partition #red/white testPartition {
    #blue\green:testActivity;
}
@enduml
```

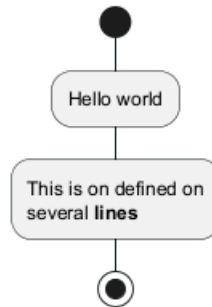


[Ref. QA-4906]

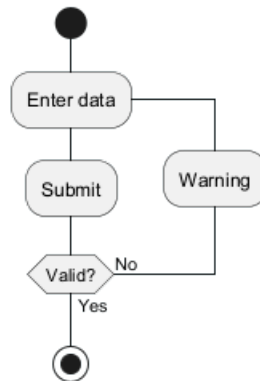
6.14 Lines without arrows

You can use skinparam ArrowHeadColor none in order to connect activities using lines only, without arrows.

```
@startuml
skinparam ArrowHeadColor none
start
:Hello world;
:This is on defined on
several **lines**;
stop
@enduml
```



```
@startuml
skinparam ArrowHeadColor none
start
repeat :Enter data;
:Submit;
backward :Warning;
repeat while (Valid?) is (No) not (Yes)
stop
@enduml
```



6.15 Arrows

Using the -> notation, you can add texts to arrow, and change their color.

It's also possible to have dotted, dashed, bold or hidden arrows.

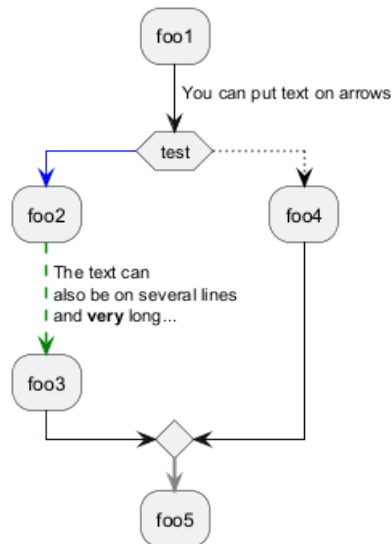
```
@startuml
:foo1;
-> You can put text on arrows;
if (test) then
-[#blue]->
:foo2;
-[#green,dashed]-> The text can
```



```

    also be on several lines
    and very long...;
    :foo3;
else
    -[#black,dotted]->
    :foo4;
endif
-[#gray,bold]->
:foo5;
@enduml

```



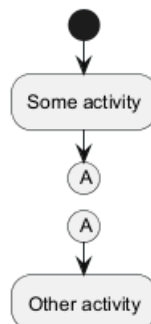
6.16 Connector

You can use parentheses to denote connector.

```

@startuml
start
:Some activity;
(A)
detach
(A)
:Other activity;
@enduml

```



6.17 Color on connector

You can add color on connector.

```

@startuml

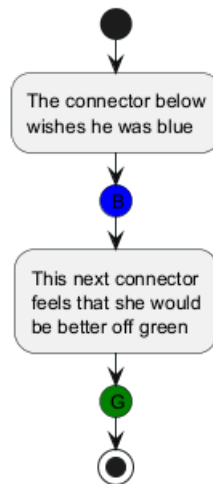
```



```

start
:The connector below
wishes he was blue;
#blue:(B)
:This next connector
feels that she would
be better off green;
#green:(G)
stop
@enduml

```



[Ref. QA-10077]

6.18 Grouping or partition

6.18.1 Group

You can group activity together by defining group:

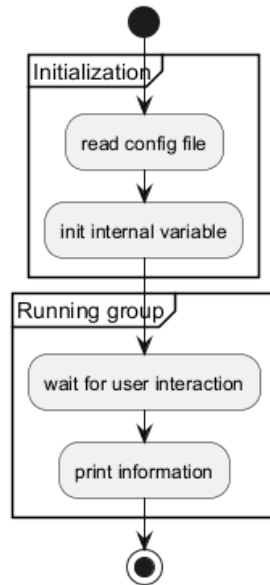
```

@startuml
start
group Initialization
    :read config file;
    :init internal variable;
end group
group Running group
    :wait for user interaction;
    :print information;
end group

stop
@enduml

```



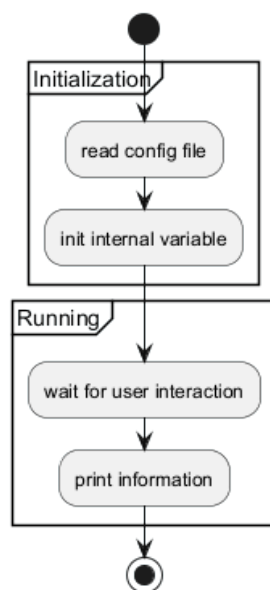


6.18.2 Partition

You can group activity together by defining partition:

```

@startuml
start
partition Initialization {
    :read config file;
    :init internal variable;
}
partition Running {
    :wait for user interaction;
    :print information;
}
stop
@enduml
  
```



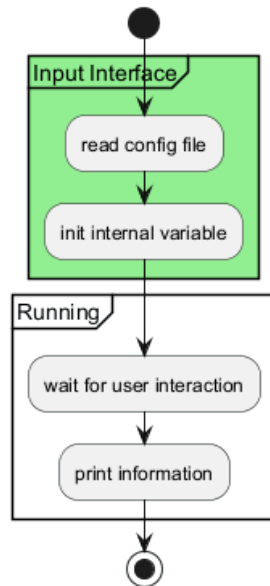
It's also possible to change partition color:



```

@startuml
start
partition #lightGreen "Input Interface" {
    :read config file;
    :init internal variable;
}
partition Running {
    :wait for user interaction;
    :print information;
}
stop
@enduml

```



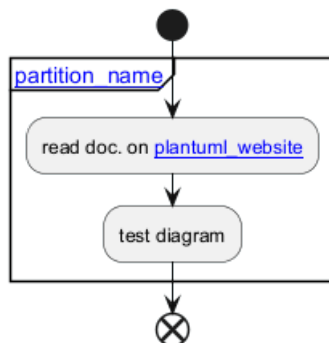
[Ref. QA-2793]

It's also possible to add link to partition:

```

@startuml
start
partition "[[http://plantuml.com partition_name]]" {
    :read doc. on [[http://plantuml.com plantuml_website]];
    :test diagram;
}
end
@enduml

```



[Ref. QA-542]



6.18.3 Group, Partition, Package, Rectangle or Card

You can group activity together by defining:

- group;
- partition;
- package;
- rectangle;
- card.

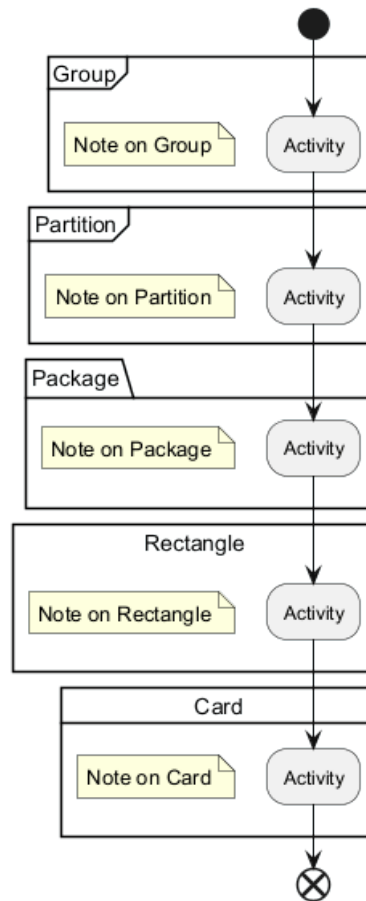
```
@startuml
start
group Group
  :Activity;
end group
floating note: Note on Group

partition Partition {
  :Activity;
}
floating note: Note on Partition

package Package {
  :Activity;
}
floating note: Note on Package

rectangle Rectangle {
  :Activity;
}
floating note: Note on Rectangle

card Card {
  :Activity;
}
floating note: Note on Card
end
@enduml
```

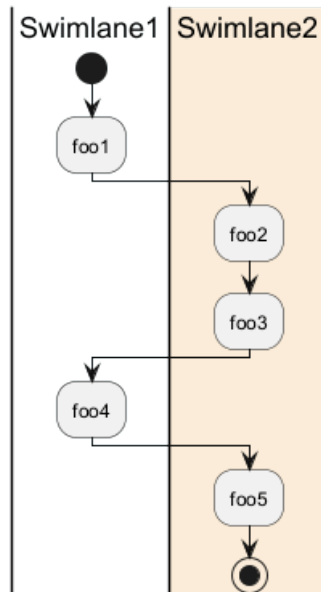


6.19 Swimlanes

Using pipe |, you can define swimlanes.

It's also possible to change swimlanes color.

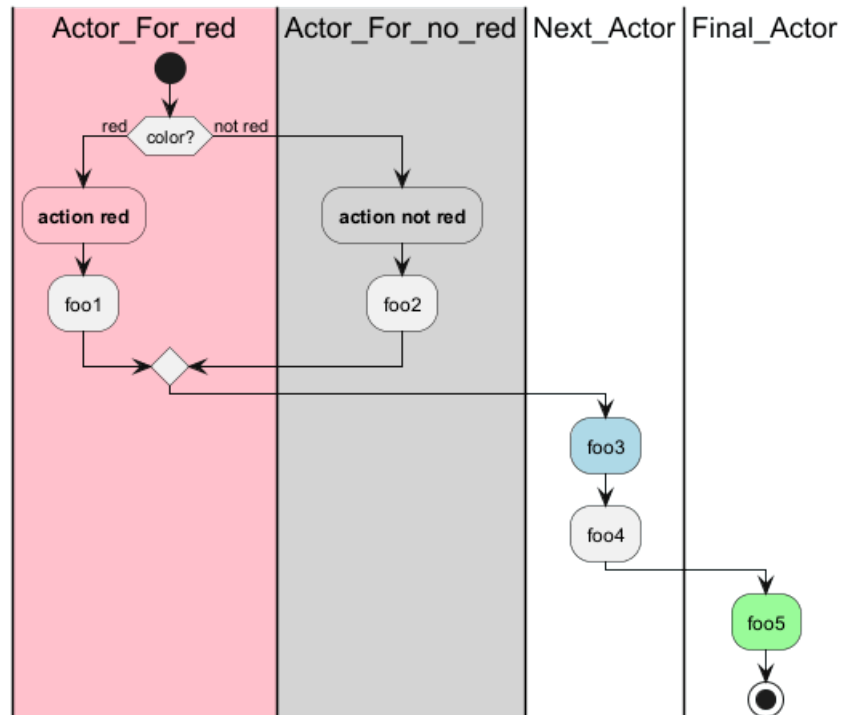
```
@startuml
|Swimlane1|
start
:foo1;
|#AntiqueWhite|Swimlane2|
:foo2;
:foo3;
|Swimlane1|
:foo4;
|Swimlane2|
:foo5;
stop
@enduml
```



You can add **if** conditional or **repeat** or **while** loop within swimlanes.

```

@startuml
|#pink|Actor_For_red|
start
if (color?) is (red) then
#pink:**action red**;
:foo1;
else (not red)
|#lightgray|Actor_For_no_red|
#lightgray:**action not red**;
:foo2;
endif
|Next_Actor|
#lightblue:foo3;
:foo4;
|Final_Actor|
#palegreen:foo5;
stop
@enduml
  
```



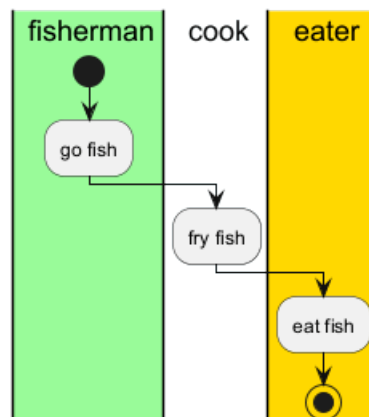
You can also use `alias` with swimlanes, with this syntax:

- `|[#<color>]|<swimlane_alias>| <swimlane_title>`

```

@startuml
|#palegreen|f| fisherman
|c| cook
|#gold|e| eater
|f|
start
:go fish;
|c|
:fry fish;
|e|
:eat fish;
stop
@enduml

```



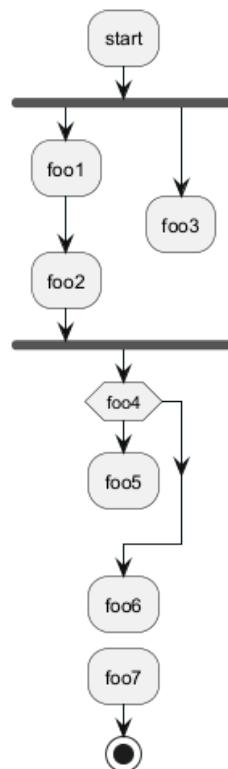
[Ref. QA-2681]

6.20 Detach or kill [detach, kill]

It's possible to remove an arrow using the **detach** or **kill** keyword:

- detach

```
@startuml
: start;
fork
: foo1;
: foo2;
fork again
: foo3;
detach
endfork
if (foo4) then
: foo5;
detach
endif
: foo6;
detach
: foo7;
stop
@enduml
```



- kill

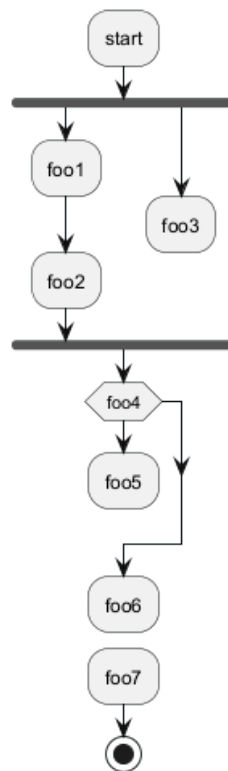
```
@startuml
: start;
fork
: foo1;
: foo2;
fork again
: foo3;
kill
endfork
```



```

endfork
if (foo4) then
    :foo5;
    kill
endif
:foo6;
kill
:foo7;
stop
@enduml

```



6.21 SDL (Specification and Description Language)

6.21.1 Table of SDL Shape Name

Name	Old syntax	Stereotype syntax
Input	<	<<input>>
Output	>	<<output>>
Procedure		<<procedure>>
Load	\	<<load>>
Save	/	<<save>>
Continuous	}	<<continuous>>
Task	]	<<task>>

[Ref. QA-11518, GH-1270]

6.21.2 SDL using final separator (Deprecated form)

By changing the final ; separator, you can set different rendering for the activity:

- |
- <
- >

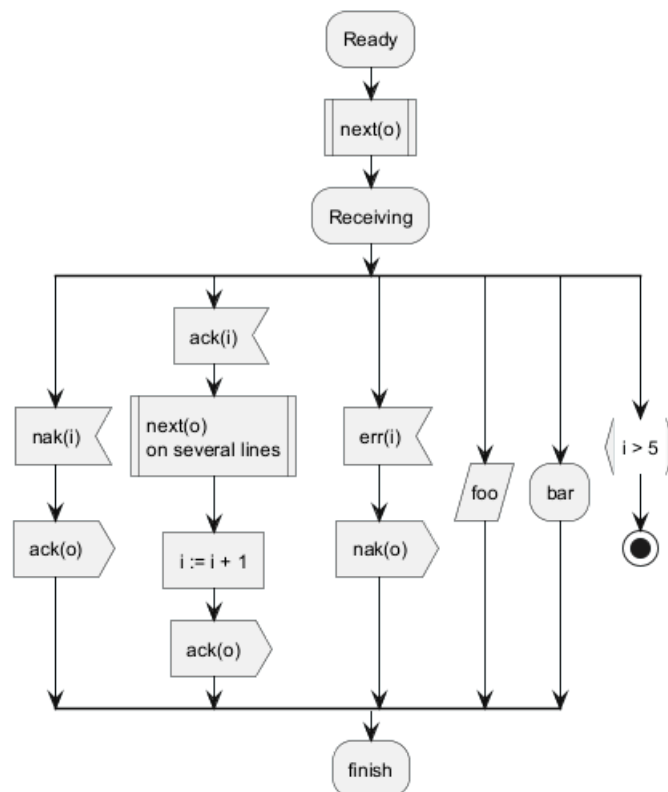


- /
- \\
-]
- }

```

@startuml
:Ready;
:next(o)|
:Receiving;
split
:nak(i)<
:ack(o)>
split again
:ack(i)<
:next(o)
on several lines|
:i := i + 1]
:ack(o)>
split again
:err(i)<
:nak(o)>
split again
:foo/
split again
:bar\\
split again
:i > 5}
stop
end split
:finish;
@enduml

```

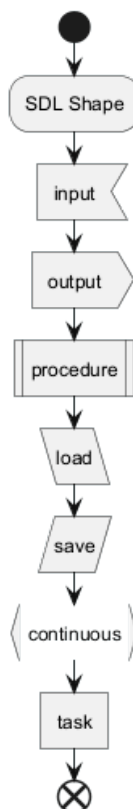


6.21.3 SDL using Normal separator and Stereotype (Current official form)

```

@startuml
start
:SDL Shape;
:input; <<input>>
:output; <<output>>
:procedure; <<procedure>>
:load; <<load>>
:save; <<save>>
:continuous; <<continuous>>
:task; <<task>>
end
@enduml

```



```

@startuml
:Ready;
:next(o); <<procedure>>
:Receiving;
split
    :nak(i); <<input>>
    :ack(o); <<output>>
split again
    :ack(i); <<input>>
    :next(o)
on several lines; <<procedure>>
    :i := i + 1; <<task>>
    :ack(o); <<output>>
split again
    :err(i); <<input>>
    :nak(o); <<output>>
split again
    :foo; <<save>>

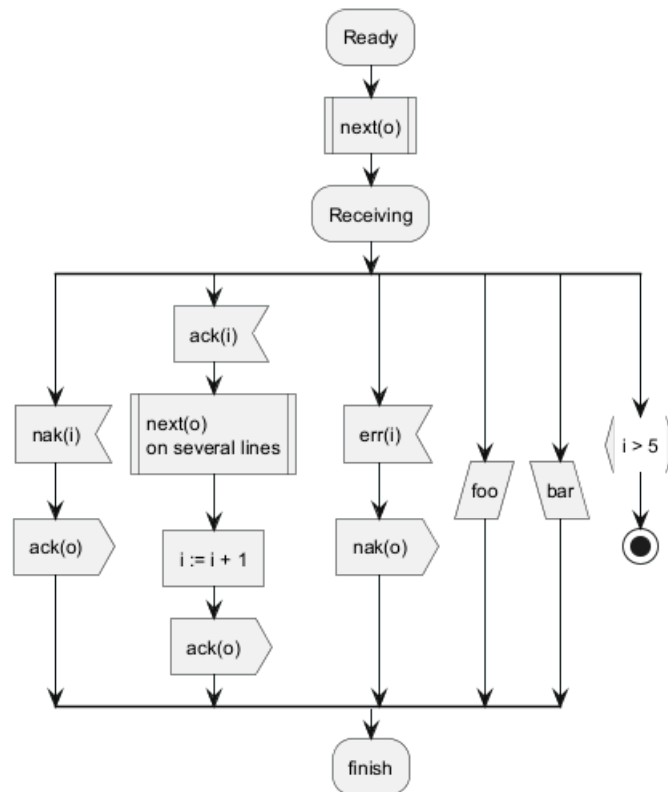
```



```

split again
:bar; <<load>>
split again
:i > 5; <<continuous>>
stop
end split
:finish;
@enduml

```



6.22 Complete example

```

@startuml

start
:ClickServlet.handleRequest();
:new page;
if (Page.onSecurityCheck) then (true)
:Page.onInit();
if (isForward?) then (no)
:Process controls;
if (continue processing?) then (no)
stop
endif
endif

if (isPost?) then (yes)
:Page.onPost();
else (no)
:Page.onGet();
endif
:Page.onRender();
endif
else (false)

```

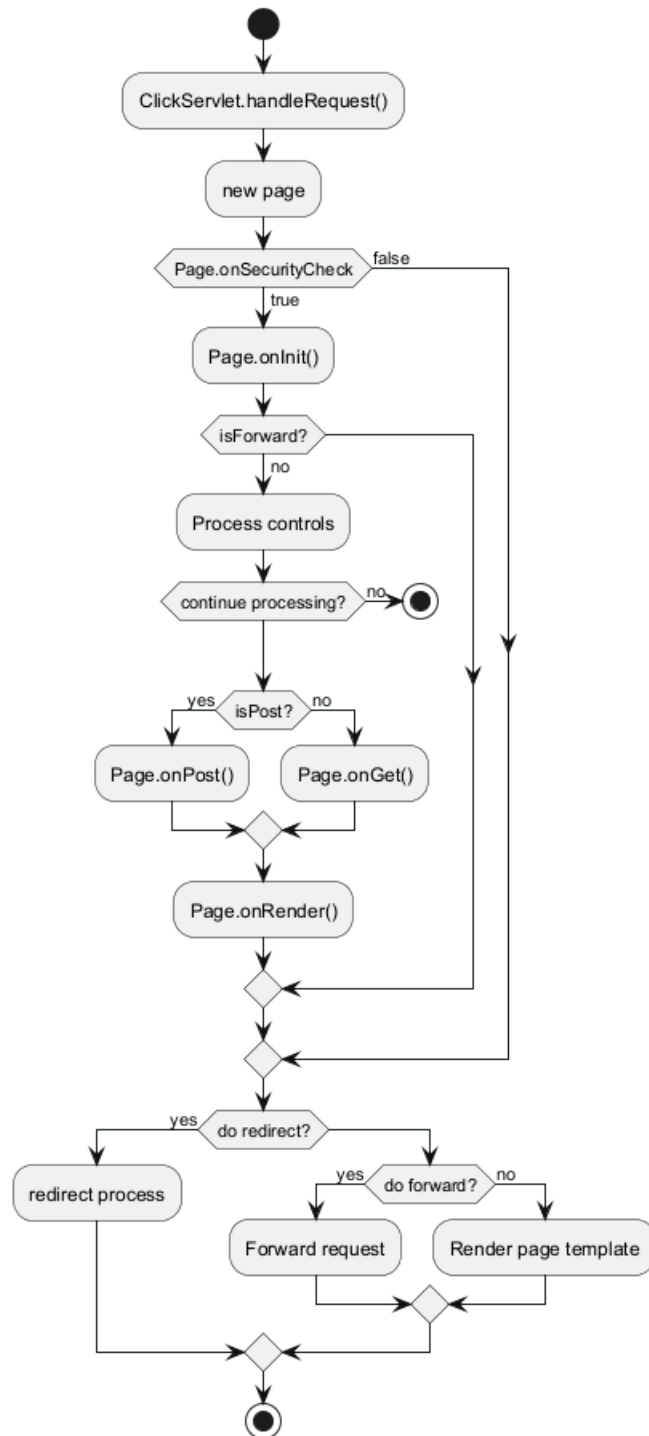


```
endif

if (do redirect?) then (yes)
  :redirect process;
else
  if (do forward?) then (yes)
    :Forward request;
  else (no)
    :Render page template;
  endif
endif

stop

@enduml
```



6.23 Condition Style

6.23.1 Inside style (by default)

```

@startuml
skinparam conditionStyle inside
start
repeat
    :act1;
    :act2;
repeatwhile (<b>end)
:act3;

```



```
@enduml
```



```
@startuml
start
repeat
  :act1;
  :act2;
repeatwhile (<b>end)
:act3;
@enduml
```



6.23.2 Diamond style

```
@startuml
skinparam conditionStyle diamond
start
repeat
  :act1;
  :act2;
repeatwhile (<b>end)
:act3;
@enduml
```

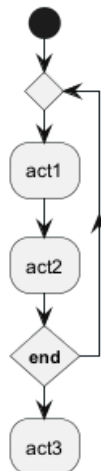




6.23.3 InsideDiamond (or *Foo1*) style

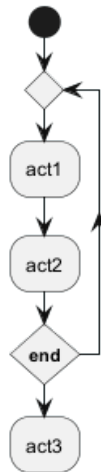
```

@startuml
skinparam conditionStyle InsideDiamond
start
repeat
    :act1;
    :act2;
repeatwhile (<b>end)
:act3;
@enduml
  
```



```

@startuml
skinparam conditionStyle foo1
start
repeat
    :act1;
    :act2;
repeatwhile (<b>end)
:act3;
@enduml
  
```



[Ref. QA-1290 and #400]

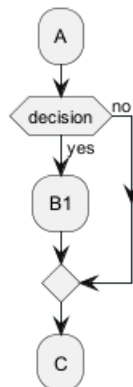
6.24 Condition End Style

6.24.1 Diamond style (by default)

- With one branch

```

@startuml
skinparam ConditionEndStyle diamond
:A;
if (decision) then (yes)
    :B1;
else (no)
endif
:C;
@enduml
  
```

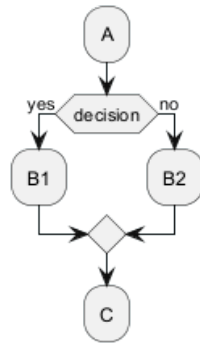


- With two branches (B1, B2)

```

@startuml
skinparam ConditionEndStyle diamond
:A;
if (decision) then (yes)
    :B1;
else (no)
    :B2;
endif
:C;
@enduml
  
```



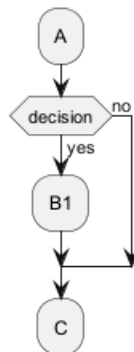


6.24.2 Horizontal line (hline) style

- With one branch

```

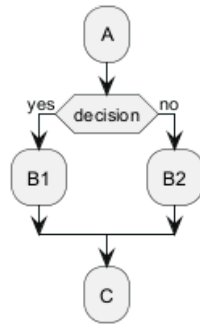
@startuml
skinparam ConditionEndStyle hline
:A;
if (decision) then (yes)
    :B1;
else (no)
endif
:C;
@enduml
  
```



- With two branches (B1, B2)

```

@startuml
skinparam ConditionEndStyle hline
:A;
if (decision) then (yes)
    :B1;
else (no)
    :B2;
endif
:C;
@enduml
  
```



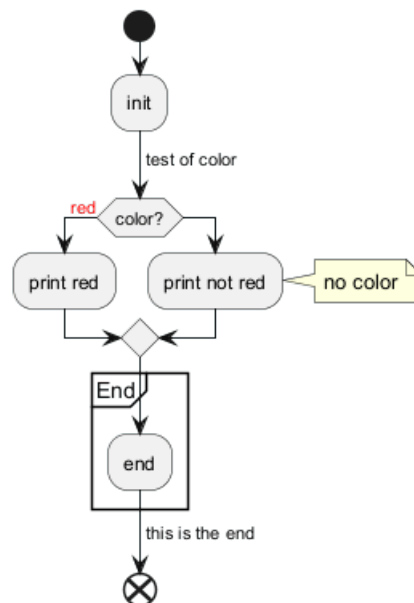
[Ref. QA-4015]

6.25 Using (global) style

6.25.1 Without style (by default)

```

@startuml
start
:init;
-> test of color;
if (color?) is (<color:red>red) then
:print red;
else
:print not red;
note right: no color
endif
partition End {
:end;
}
-> this is the end;
end
@enduml
  
```



6.25.2 With style

You can use style to change rendering of elements.

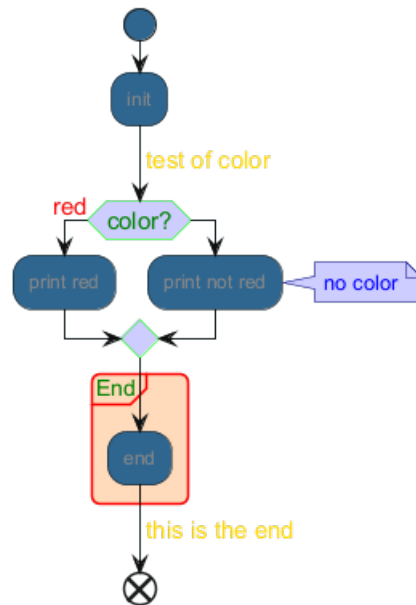
```
@startuml
```



```
<style>
activityDiagram {
    BackgroundColor #33668E
    BorderColor #33668E
    FontColor #888
    FontName arial

    diamond {
        BackgroundColor #ccf
        LineColor #00FF00
        FontColor green
        FontName arial
        FontSize 15
    }
    arrow {
        FontColor gold
        FontName arial
        FontSize 15
    }
    partition {
        LineColor red
        FontColor green
        RoundCorner 10
        BackgroundColor PeachPuff
    }
    note {
        FontColor Blue
        LineColor Navy
        BackgroundColor #ccf
    }
}
document {
    BackgroundColor transparent
}
</style>
start
:init;
-> test of color;
if (color?) is (<color:red>red) then
:print red;
else
:print not red;
note right: no color
endif
partition End {
:end;
}
-> this is the end;
end
@enduml
```





7 Component Diagram

Component Diagram: A component diagram is a type of structural diagram used in UML (Unified Modeling Language) to visualize the organization and relationships of system components. These diagrams help in breaking down complex systems into manageable components, showcasing their interdependencies, and ensuring efficient system design and architecture.

Advantages of PlantUML:

- **Simplicity:** With PlantUML, you can create component diagrams using simple and intuitive text-based descriptions, eliminating the need for complex drawing tools.
- **Integration:** PlantUML seamlessly integrates with various tools and platforms, making it a versatile choice for developers and architects.
- **Collaboration:** The PlantUML forum offers a platform for users to discuss, share, and seek assistance on their diagrams, fostering a collaborative community.

7.1 Components

Components must be bracketed.

You can also use the `component` keyword to define a component. In this case the brackets can be omitted, but only if the component name does not include white-space or special characters.

You can define an alias, using the `as` keyword. This alias will be used later, when defining relations.

```
@startuml
```

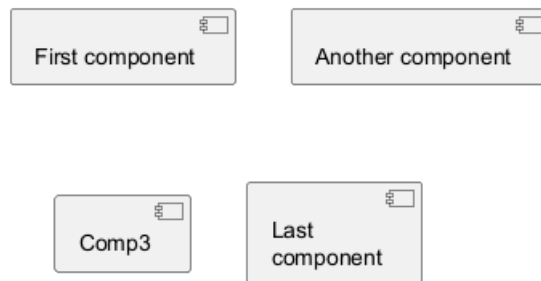
```
[First component]
```

```
[Another component] as Comp2
```

```
component Comp3
```

```
component [Last\ncomponent] as Comp4
```

```
@enduml
```



7.1.1 Naming exceptions

Note that component names starting with `$` cannot be hidden or removed later, because `hide` and `remove` command will consider the name a `$tag` instead of a component name. To later remove such component they must have an alias or must be tagged.

```
@startuml
```

```
component [$C1]
```

```
component [$C2] $C2
```

```
component [$C2] as dollarC2
```

```
remove $C1
```

```
remove $C2
```

```
remove dollarC2
```

```
@enduml
```



7.2 Interfaces

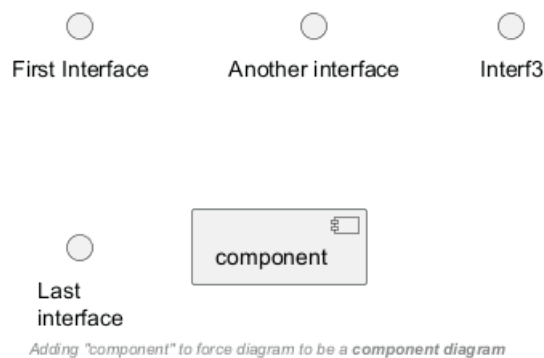
Interface can be defined using the () symbol (because this looks like a circle).

You can also use the `interface` keyword to define an interface. And you can define an alias, using the `as` keyword. This alias will be used latter, when defining relations.

We will see latter that interface definition is optional.

```
@startuml
() "First Interface"
() "Another interface" as Interf2
interface Interf3
interface "Last\ninterface" as Interf4

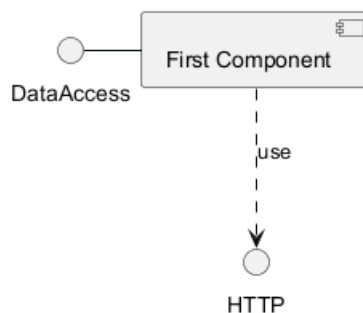
[component]
footer //Adding "component" to force diagram to be a **component diagram**//
@enduml
```



7.3 Basic example

Links between elements are made using combinations of dotted line (.), straight line (--), and arrows (-->) symbols.

```
@startuml
DataAccess - [First Component]
[First Component] ..> HTTP : use
@enduml
```



7.4 Using notes

You can use the `note left of`, `note right of`, `note top of`, `note bottom of` keywords to define notes related to a single object.

```
@startuml
[Component] as C
```



```

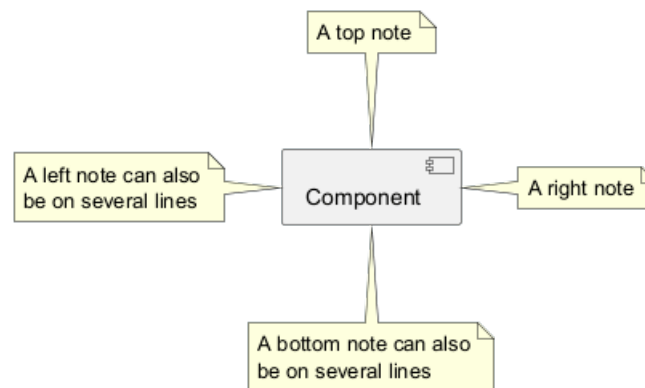
note top of C: A top note

note bottom of C
  A bottom note can also
  be on several lines
end note

note left of C
  A left note can also
  be on several lines
end note

note right of C: A right note
@enduml

```



A note can be also defined alone with the `note` keywords, then linked to other objects using the `..` symbol or whatever arrow symbol (`-`, `--`, `...`).

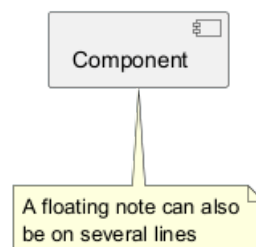
```

@startuml
[Component] as C

note as N
  A floating note can also
  be on several lines
end note

C .. N
@enduml

```



Another note example:

```

@startuml

interface "Data Access" as DA

DA - [First Component]
[First Component] ..> HTTP : use

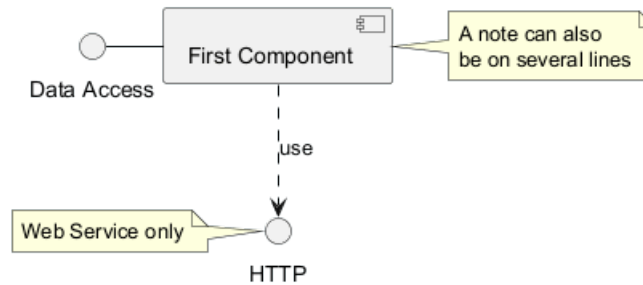
```

```
note left of HTTP : Web Service only
```

```
note right of [First Component]
```

```
  A note can also  
  be on several lines  
end note
```

```
@enduml
```



7.5 Grouping Components

You can use several keywords to group components and interfaces together:

- package
- node
- folder
- frame
- cloud
- database

```
@startuml
```

```
package "Some Group" {
  HTTP - [First Component]
  [Another Component]
}
```

```
node "Other Groups" {
  FTP - [Second Component]
  [First Component] --> FTP
}
```

```
cloud {
  [Example 1]
}
```

```
database "MySQL" {
  folder "This is my folder" {
    [Folder 3]
  }
  frame "Foo" {
    [Frame 4]
  }
}
```

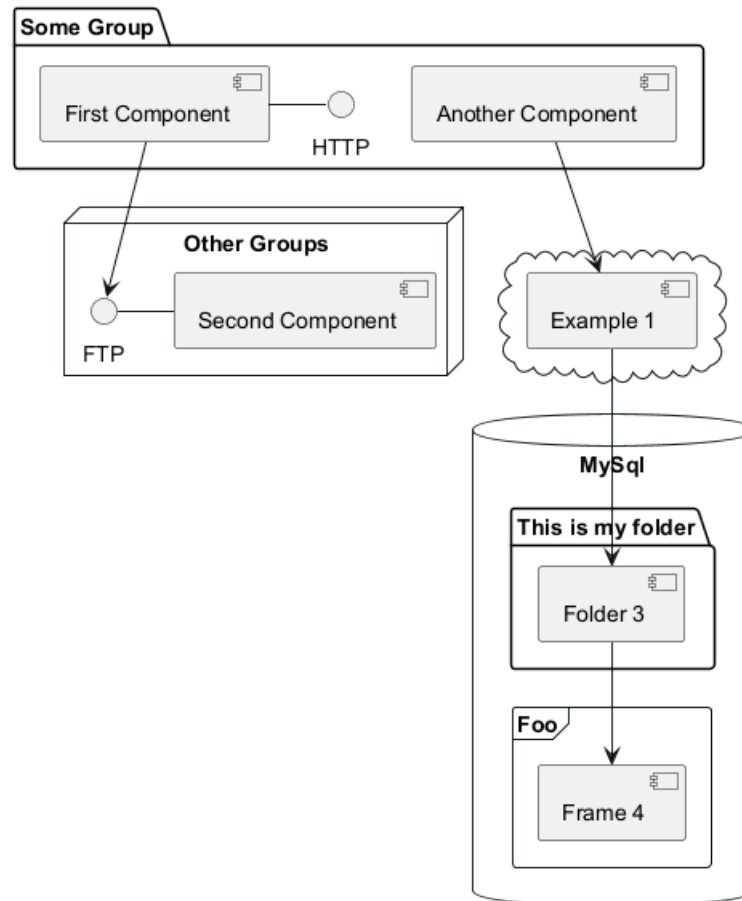



```

[Another Component] --> [Example 1]
[Example 1] --> [Folder 3]
[Folder 3] --> [Frame 4]

```

```
@enduml
```



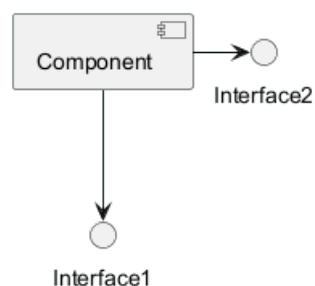
7.6 Changing arrows direction

By default, links between classes have two dashes -- and are vertically oriented. It is possible to use horizontal link by putting a single dash (or dot) like this:

```

@startuml
[Component] --> Interface1
[Component] -> Interface2
@enduml

```



You can also change directions by reversing the link:

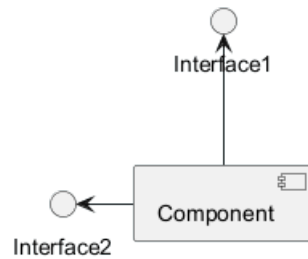
```
@startuml
```



```

Interface1 <-- [Component]
Interface2 <- [Component]
@enduml

```

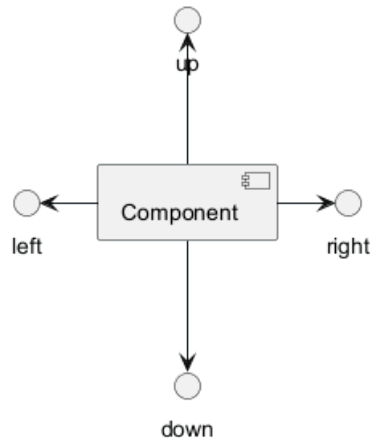


It is also possible to change arrow direction by adding **left**, **right**, **up** or **down** keywords inside the arrow:

```

@startuml
[Component] -left-> left
[Component] -right-> right
[Component] -up-> up
[Component] -down-> down
@enduml

```



You can shorten the arrow by using only the first character of the direction (for example, **-d-** instead of **-down-**) or the two first characters (**-do-**).

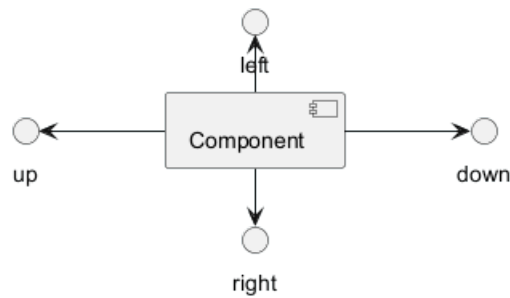
Please note that you should not abuse this functionality : *Graphviz* gives usually good results without tweaking.

And with the **left to right direction** parameter:

```

@startuml
left to right direction
[Component] -left-> left
[Component] -right-> right
[Component] -up-> up
[Component] -down-> down
@enduml

```



See also 'Change diagram orientation' on Deployment diagram page.

7.7 Use UML2 notation

By default (from v1.2020.13-14), UML2 notation is used.

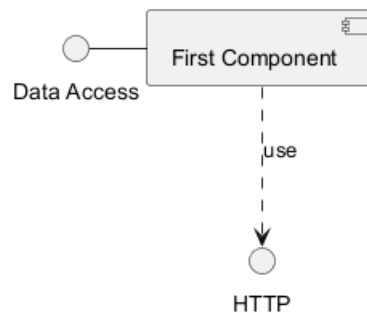
```
@startuml
```

```
interface "Data Access" as DA
```

```
DA - [First Component]
```

```
[First Component] ..> HTTP : use
```

```
@enduml
```



7.8 Use UML1 notation

The skinparam componentStyle uml1 command is used to switch to UML1 notation.

```
@startuml
```

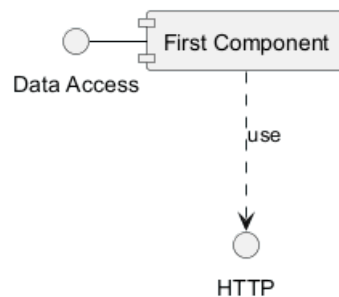
```
skinparam componentStyle uml1
```

```
interface "Data Access" as DA
```

```
DA - [First Component]
```

```
[First Component] ..> HTTP : use
```

```
@enduml
```



7.9 Use rectangle notation (remove UML notation)

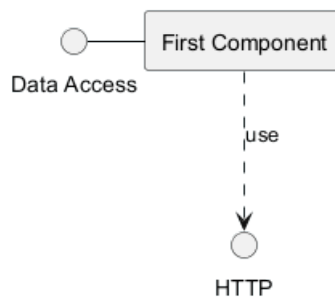
The `skinparam componentStyle rectangle` command is used to switch to rectangle notation (*without any UML notation*).

```
@startuml
skinparam componentStyle rectangle

interface "Data Access" as DA

DA - [First Component]
[First Component] ..> HTTP : use

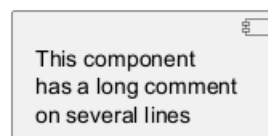
@enduml
```



7.10 Long description

It is possible to put description on several lines using square brackets.

```
@startuml
component comp1 [
This component
has a long comment
on several lines
]
@enduml
```



7.11 Individual colors

You can specify a color after component definition.

```
@startuml
component [Web Server] #Yellow
@enduml
```



7.12 Using Sprite in Stereotype

You can use sprites within stereotype components.

```
@startuml
sprite $businessProcess [16x16/16] {
```



```

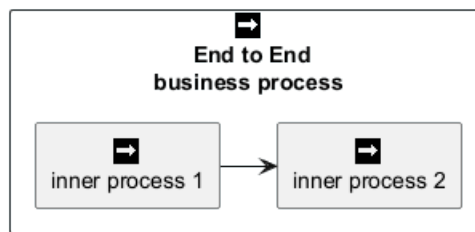
FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF
FFFFFFFFF0FFFF
FFFFFFFFF0FFFF
FF0000000000FF
FF0000000000FF
FF0000000000FF
FFFFFFFFF0FFFF
FFFFFFFFF0FFFF
FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF
}

```

```

rectangle " End to End\nbusiness process" <<$businessProcess>> {
  rectangle "inner process 1" <<$businessProcess>> as src
  rectangle "inner process 2" <<$businessProcess>> as tgt
  src -> tgt
}
@enduml

```



7.13 Skinparam

You can use the skinparam command to change colors and fonts for the drawing.

You can use this command :

- In the diagram definition, like any other commands;
- In an included file;
- In a configuration file, provided in the command line or the Ant task.

You can define specific color and fonts for stereotyped components and interfaces.

```
@startuml
```

```

skinparam interface {
  backgroundColor RosyBrown
  borderColor orange
}

```

```

skinparam component {
  FontSize 13
  BackgroundColor<<Apache>> Pink
  BorderColor<<Apache>> #FF6655
  FontName Courier
  BorderColor black
}

```



```

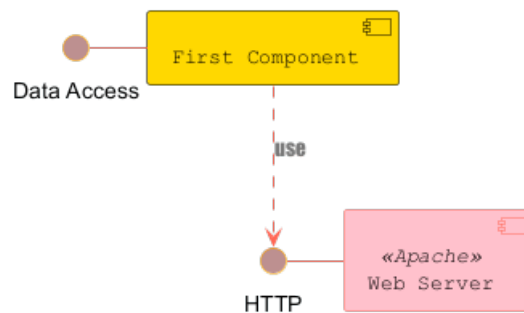
    BackgroundColor gold
    ArrowFontName Impact
    ArrowColor #FF6655
    ArrowFontColor #777777
}

() "Data Access" as DA
Component "Web Server" as WS << Apache >>

DA - [First Component]
[First Component] ..> () HTTP : use
HTTP - WS

@enduml

```



```

@startuml

skinparam component {
    backgroundColor<<static lib>> DarkKhaki
    backgroundColor<<shared lib>> Green
}

skinparam node {
    borderColor Green
    backgroundColor Yellow
    backgroundColor<<shared_node>> Magenta
}

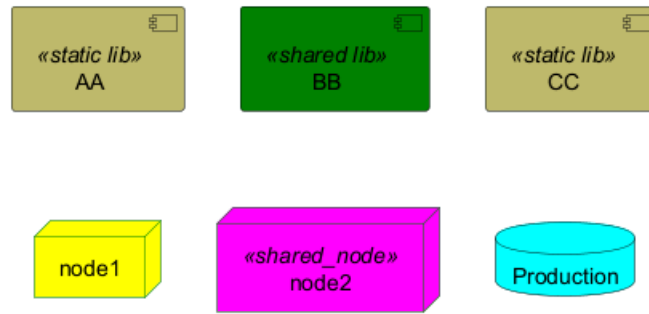
skinparam databaseBackgroundColor Aqua

[AA] <<static lib>>
[BB] <<shared lib>>
[CC] <<static lib>>

node node1
node node2 <<shared_node>>
database Production

@enduml

```

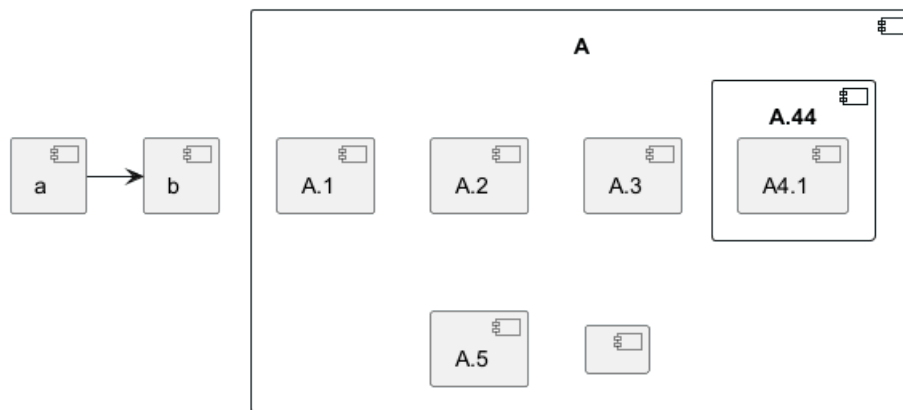


7.14 Specific SkinParameter

7.14.1 componentStyle

- By default (or with `skinparam componentStyle uml2`), you have an icon for component

```
@startuml
skinparam BackgroundColor transparent
skinparam componentStyle uml2
component A {
    component "A.1" {
    }
    component A.44 {
        [A4.1]
    }
    component "A.2"
        [A.3]
    component A.5 [
A.5]
    component A.6 [
]
}
[a]->[b]
@enduml
```



- If you want to suppress it, and to have only the rectangle, you can use `skinparam componentStyle rectangle`

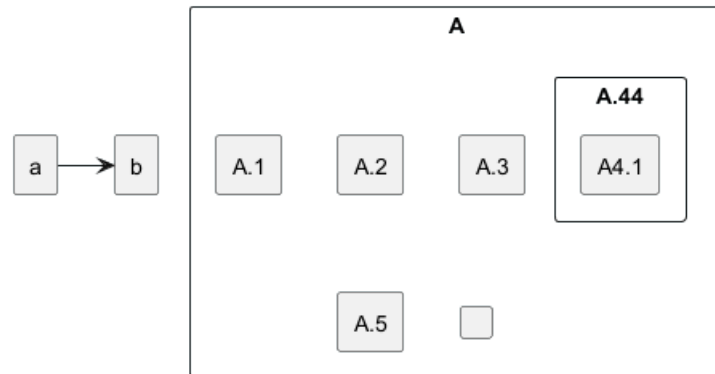
```
@startuml
skinparam BackgroundColor transparent
skinparam componentStyle rectangle
component A {
    component "A.1" {
    }
    component A.44 {
    }
    component "A.2"
        [A.3]
    component A.5 [
A.5]
    component A.6 [
]
}
[a]->[b]
@enduml
```



```

    [A4.1]
}
component "A.2"
[A.3]
component A.5 [
A.5]
component A.6 [
]
}
[a]->[b]
@enduml

```



[Ref. 10798]

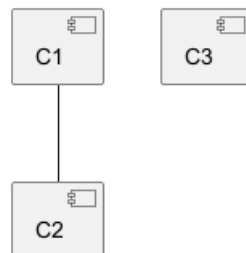
7.15 Hide or Remove unlinked component

By default, all components are displayed:

```

@startuml
component C1
component C2
component C3
C1 -- C2
@enduml

```



But you can:

- **hide @unlinked components:**

```

@startuml
component C1
component C2
component C3
C1 -- C2

hide @unlinked
@enduml

```





- or remove @unlinked components:

```

@startuml
component C1
component C2
component C3
C1 -- C2

remove @unlinked
@enduml
  
```



[Ref. QA-11052]

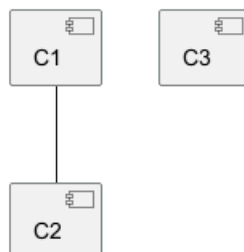
7.16 Hide, Remove or Restore tagged component or wildcard

You can put \$tags (using \$) on components, then remove, hide or restore components either individually or by tags.

By default, all components are displayed:

```

@startuml
component C1 $tag13
component C2
component C3 $tag13
C1 -- C2
@enduml
  
```



But you can:

- hide \$tag13 components:

```

@startuml
component C1 $tag13
component C2
component C3 $tag13
  
```



```
C1 -- C2
```

```
hide $tag13
@enduml
```



- or remove \$tag13 components:

```
@startuml
component C1 $tag13
component C2
component C3 $tag13
C1 -- C2
```

```
remove $tag13
@enduml
```



- or remove \$tag13 and restore \$tag1 components:

```
@startuml
component C1 $tag13 $tag1
component C2
component C3 $tag13
C1 -- C2
```

```
remove $tag13
restore $tag1
@enduml
```



- or remove * and restore \$tag1 components:

```
@startuml
component C1 $tag13 $tag1
component C2
component C3 $tag13
C1 -- C2
```

```
remove *
restore $tag1
@enduml
```





[Ref. QA-7337 and QA-11052]

7.17 Display JSON Data on Component diagram

7.17.1 Simple example

```
@startuml
allowmixing

component Component
()      Interface

json JSON {
    "fruit": "Apple",
    "size": "Large",
    "color": ["Red", "Green"]
}
@enduml
```



JSON	
fruit	Apple
size	Large
color	Red
	Green

[Ref. QA-15481]

For another example, see on JSON page.

7.18 Port [port, portIn, portOut]

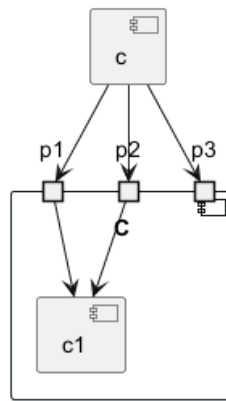
You can add **port** with **port**, **portIn** and **portOut** keywords.

7.18.1 Port

```
@startuml
[c]
component C {
    port p1
    port p2
    port p3
    component c1
}

c --> p1
c --> p2
c --> p3
p1 --> c1
p2 --> c1
@enduml
```





7.18.2 PortIn

```

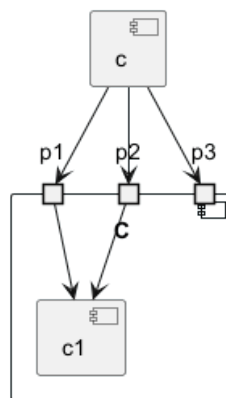
@startuml
[c]
component C {
    portin p1
    portin p2
    portin p3
    component c1
}

```

```

c --> p1
c --> p2
c --> p3
p1 --> c1
p2 --> c1
@enduml

```



7.18.3 PortOut

```

@startuml
component C {
    portout p1
    portout p2
    portout p3
    component c1
}

```

```

[o]
p1 --> o
p2 --> o
p3 --> o

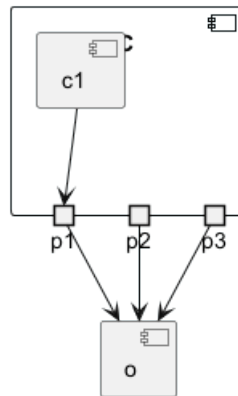
```



```

c1 --> p1
@enduml

```



7.18.4 Mixing PortIn & PortOut

```

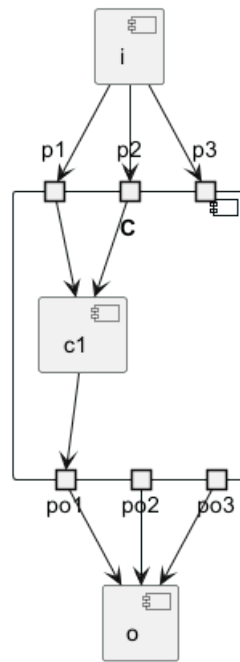
@startuml
[i]
component C {
    portin p1
    portin p2
    portin p3
    portout po1
    portout po2
    portout po3
    component c1
}
[o]

```

```

i --> p1
i --> p2
i --> p3
p1 --> c1
p2 --> c1
po1 --> o
po2 --> o
po3 --> o
c1 --> po1
@enduml

```



8 Deployment Diagram

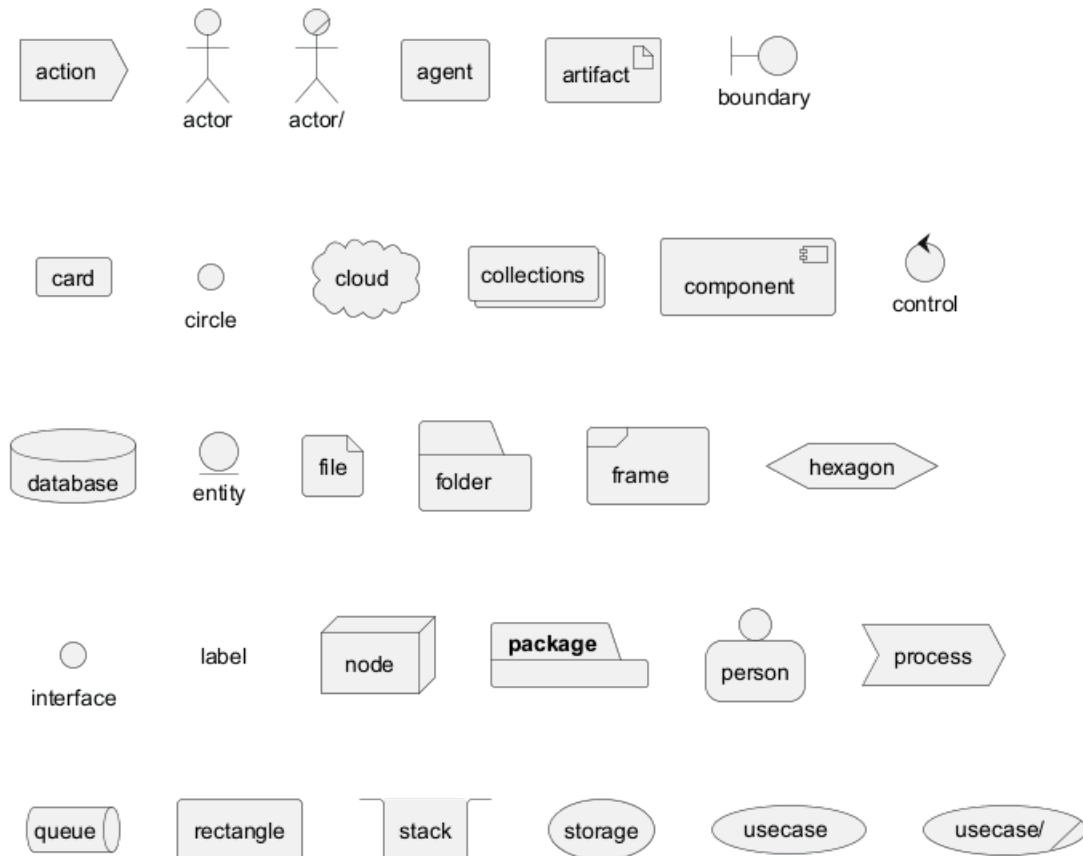
A **Deployment Diagram** is a type of diagram that visualizes the architecture of systems, showcasing how software components are deployed onto hardware. It provides a clear picture of the distribution of components across various nodes, such as servers, workstations, and devices.

With PlantUML, creating deployment diagrams becomes a breeze. The platform offers a simple and intuitive way to design these diagrams using plain text, ensuring rapid iterations and easy version control. Moreover, the PlantUML forum provides a vibrant community where users can seek help, share ideas, and collaborate on diagramming challenges. One of the key advantages of PlantUML is its ability to integrate seamlessly with various tools and platforms, making it a preferred choice for professionals and enthusiasts alike.

8.1 Declaring element

```
@startuml
action action
actor actor
actor/ "actor/"
agent agent
artifact artifact
boundary boundary
card card
circle circle
cloud cloud
collections collections
component component
control control
database database
entity entity
file file
folder folder
frame frame
hexagon hexagon
interface interface
label label
node node
package package
person person
process process
queue queue
rectangle rectangle
stack stack
storage storage
usecase usecase
usecase/ "usecase/"
@enduml
```





You can optionally put text using bracket `[]` for a long description.

```
@startuml
folder folder [
This is a <b>folder
----
You can use separator
====
of different kind
....
and style
]

node node [
This is a <b>node
----
You can use separator
====
of different kind
....
and style
]

database database [
This is a <b>database
----
You can use separator
====
of different kind
....
and style
```



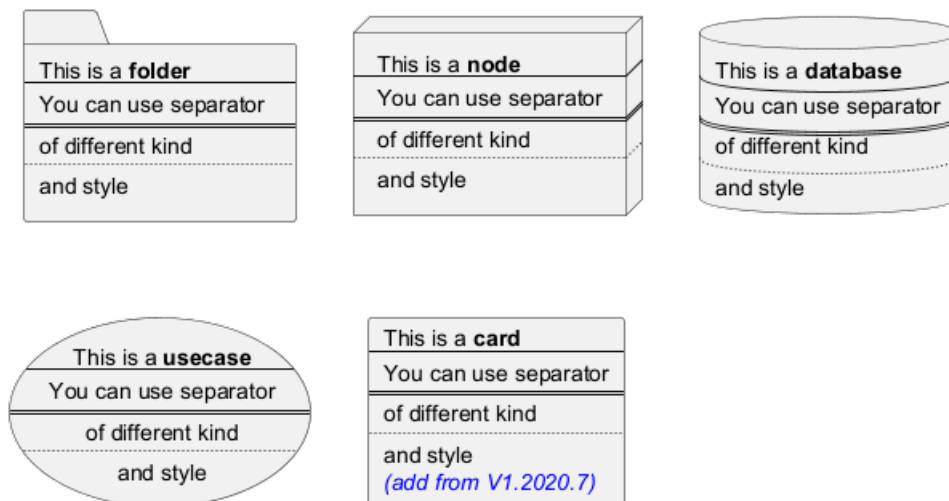

```

]

usecase usecase [
This is a <b>usecase
----
You can use separator
====
of different kind
....
and style
]

card card [
This is a <b>card
----
You can use separator
====
of different kind
....
and style
<i><color:blue>(add from V1.2020.7)</color></i>
]
@enduml

```



8.2 Declaring element (using short form)

We can declare element using some short forms.

Long form Keyword	Short form Keyword	Long form example	Short form example	Ref.
actor	: a :	actor actor1	:actor2:	Actors
component	[c]	component component1	[component2]	Components
interface	() i	interface interface1	() "interface2"	Interfaces
usecase	(u)	usecase usecase1	(usecase2)	Usecases

8.2.1 Actor

```

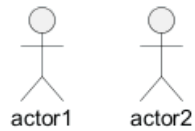
@startuml

actor actor1
:actor2:

@enduml

```





NB: *There is an old syntax for actor with guillemet which is now deprecated and will be removed some days. Please do not use in your diagram.*

8.2.2 Component

```

@startuml

component component1
[component2]

@enduml
  
```



8.2.3 Interface

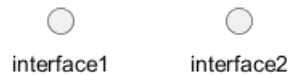
```

@startuml

interface interface1
() "interface2"

label "//interface example//"

@enduml
  
```



interface example

8.2.4 Usecase

```

@startuml

usecase usecase1
(usecase2)

@enduml
  
```



8.3 Linking or arrow

You can create simple links between elements with or without labels:

```

@startuml

node node1
node node2
node node3
node node4
  
```

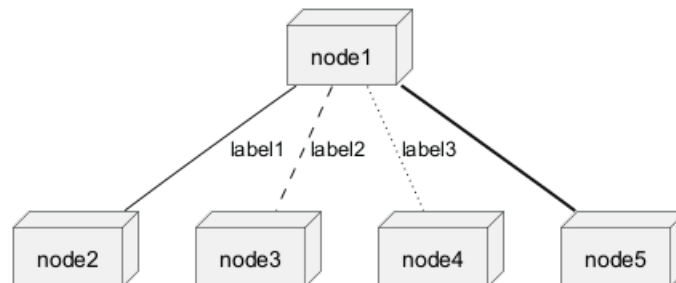


```

node node5
node1 -- node2 : label1
node1 .. node3 : label2
node1 ~~ node4 : label3
node1 == node5

```

```
@enduml
```



It is possible to use several types of links:

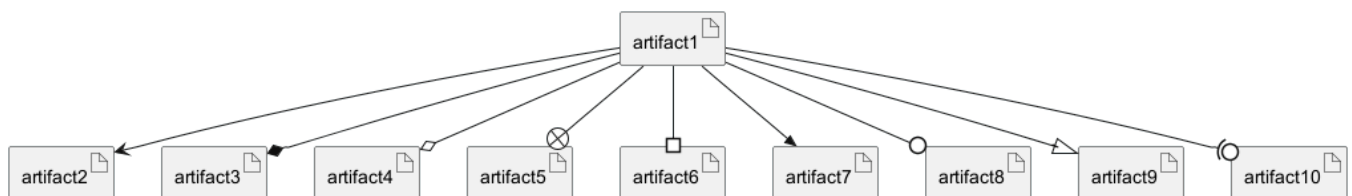
```
@startuml
```

```

artifact artifact1
artifact artifact2
artifact artifact3
artifact artifact4
artifact artifact5
artifact artifact6
artifact artifact7
artifact artifact8
artifact artifact9
artifact artifact10
artifact1 --> artifact2
artifact1 --* artifact3
artifact1 --o artifact4
artifact1 --+ artifact5
artifact1 --# artifact6
artifact1 -->> artifact7
artifact1 --0 artifact8
artifact1 --^ artifact9
artifact1 --(0 artifact10

```

```
@enduml
```



You can also have the following types:

```
@startuml
```

```

cloud cloud1
cloud cloud2
cloud cloud3
cloud cloud4

```

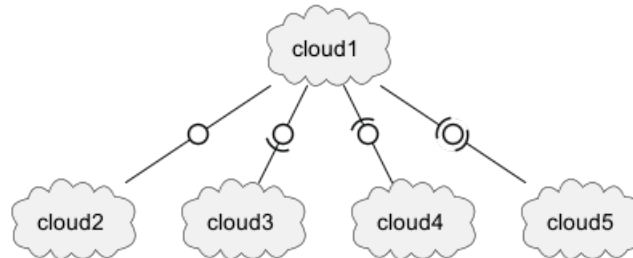


```

cloud cloud5
cloud1 -0- cloud2
cloud1 -0)- cloud3
cloud1 -(0- cloud4
cloud1 -(0)- cloud5

```

```
@enduml
```



or another example:

```

@startuml
actor foo1
actor foo2
foo1 <-0-> foo2
foo1 <-(0)-> foo2

(ac1) -le(0)-> left1
ac1 -ri(0)-> right1
ac1 .up(0).> up1
ac1 ~up(0)~> up2
ac1 -do(0)-> down1
ac1 -do(0)-> down2

actor1 -0)- actor2

component comp1
component comp2
comp1 *-0)-+ comp2
[comp3] <-->> [comp4]

boundary b1
control c1
b1 -(0)- c1

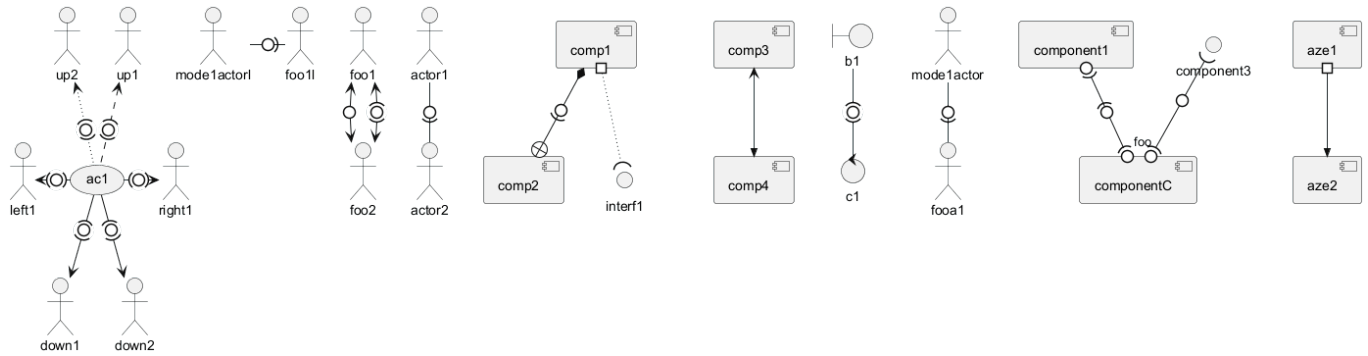
component comp1
interface interf1
comp1 #~~( interf1

:modelactor: -0)- fooa1
:modelactorl: -ri0)- foo1l

[component1] 0)-(0-(0 [componentC]
() component3 )-0-(0 "foo" [componentC]

[aze1] #-->> [aze2]
@enduml

```



[Ref. QA-547 and QA-1736]

See all type on **Appendix**.

8.4 Bracketed arrow style

Similar as Bracketed **class** relations (linking or arrow) style

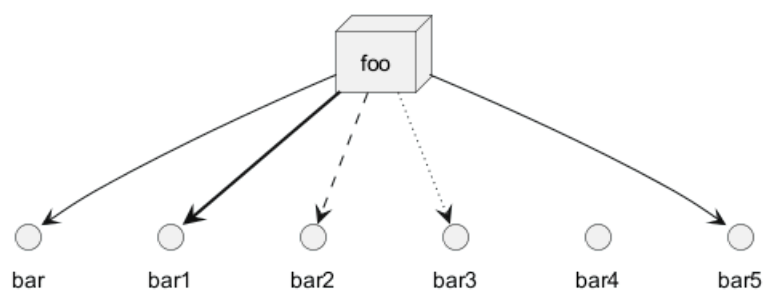
8.4.1 Line style

It's also possible to have explicitly bold, dashed, dotted, hidden or plain arrows:

- without label

```
@startuml
node foo
title Bracketed line style without label
foo --> bar
foo -[bold]-> bar1
foo -[dashed]-> bar2
foo -[dotted]-> bar3
foo -[hidden]-> bar4
foo -[plain]-> bar5
@enduml
```

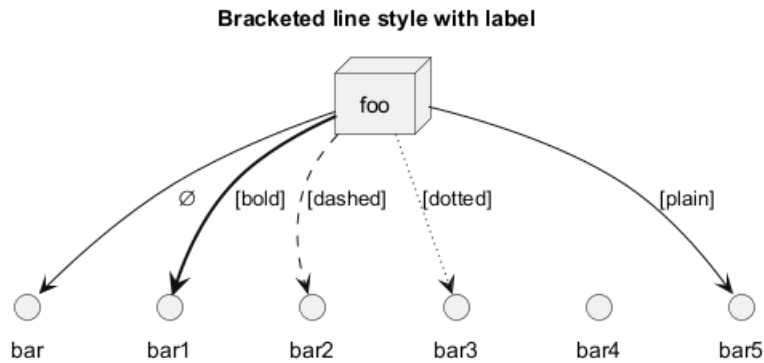
Bracketed line style without label



- with label

```
@startuml
title Bracketed line style with label
node foo
foo --> bar : 
foo -[bold]-> bar1 : [bold]
foo -[dashed]-> bar2 : [dashed]
foo -[dotted]-> bar3 : [dotted]
foo -[hidden]-> bar4 : [hidden]
foo -[plain]-> bar5 : [plain]
@enduml
```



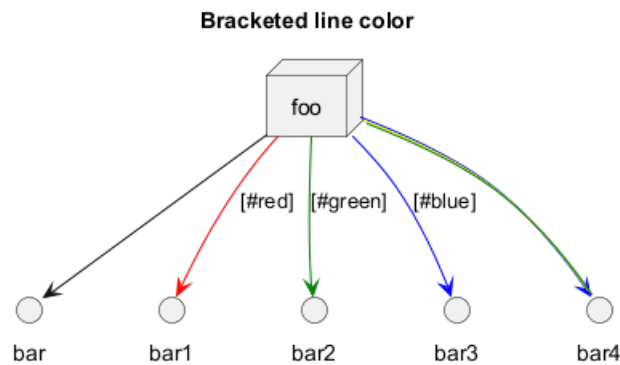


[Adapted from QA-4181]

8.4.2 Line color

```

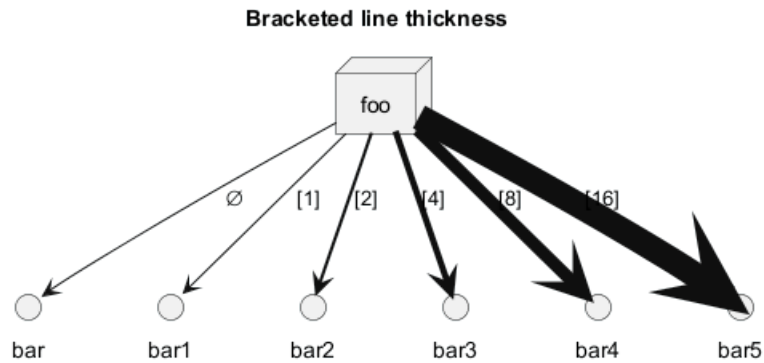
@startuml
title Bracketed line color
node foo
foo --> bar
foo -[#red]-> bar1 : [#red]
foo -[#green]-> bar2 : [#green]
foo -[#blue]-> bar3 : [#blue]
foo -[#blue;#yellow;#green]-> bar4
@enduml
  
```



8.4.3 Line thickness

```

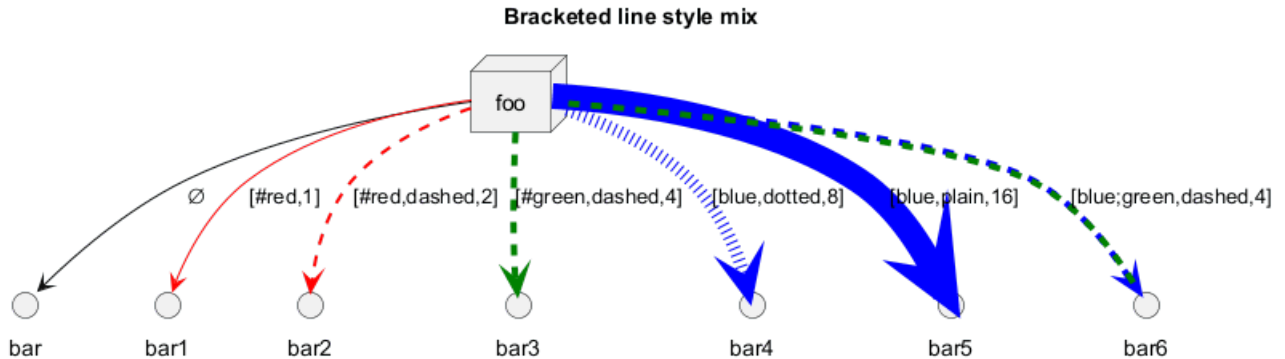
@startuml
title Bracketed line thickness
node foo
foo --> bar : 
foo -[thickness=1]-> bar1 : [1]
foo -[thickness=2]-> bar2 : [2]
foo -[thickness=4]-> bar3 : [4]
foo -[thickness=8]-> bar4 : [8]
foo -[thickness=16]-> bar5 : [16]
@enduml
  
```



[Adapted from QA-4949]

8.4.4 Mix

```
@startuml
title Bracketed line style mix
node foo
foo --> bar : 
foo -[#red,thickness=1]-> bar1 : [#red,1]
foo -[#red,dashed,thickness=2]-> bar2 : [#red,dashed,2]
foo -[#green,dashed,thickness=4]-> bar3 : [#green,dashed,4]
foo -[#blue,dotted,thickness=8]-> bar4 : [blue,dotted,8]
foo -[#blue,plain,thickness=16]-> bar5 : [blue,plain,16]
foo -[#blue;#green,dashed,thickness=4]-> bar6 : [blue;green,dashed,4]
@enduml
```



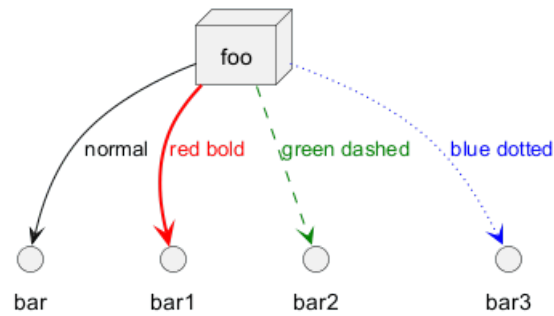
8.5 Change arrow color and style (inline style)

You can change the color or style of individual arrows using the inline following notation:

- `#color;line.[bold|dashed|dotted];text:color`

```
@startuml
node foo
foo --> bar : normal
foo --> bar1 #line:red;line.bold;text:red : red bold
foo --> bar2 #green;line.dashed;text:green : green dashed
foo --> bar3 #blue;line.dotted;text:blue : blue dotted
@enduml
```





[Ref. QA-3770 and QA-3816] [See similar feature on class diagram]

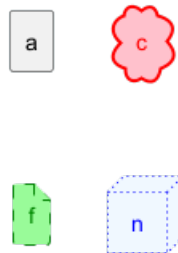
8.6 Change element color and style (inline style)

You can change the color or style of individual element using the following notation:

- `#[color|back:color];line:color;line.[bold|dashed|dotted];text:color`

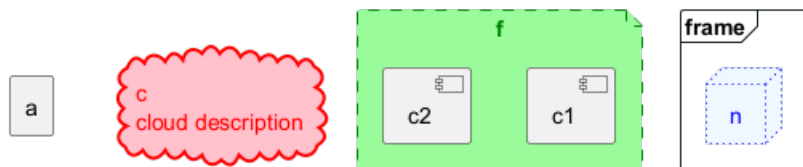
```

@startuml
agent a
cloud c #pink;line:red;line.bold;text:red
file f #palegreen;line:green;line.dashed;text:green
node n #aliceblue;line:blue;line.dotted;text:blue
@enduml
  
```



```

@startuml
agent a
cloud c #pink;line:red;line.bold;text:red [
c
cloud description
]
file f #palegreen;line:green;line.dashed;text:green {
[c1]
[c2]
}
frame frame {
node n #aliceblue;line:blue;line.dotted;text:blue
}
@enduml
  
```



[Ref. QA-6852]



8.7 Nestable elements

Here are the nestable elements:

```
@startuml
action action {
}
artifact artifact {
}
card card {
}
cloud cloud {
}
component component {
}
database database {
}
file file {
}
folder folder {
}
frame frame {
}
hexagon hexagon {
}
node node {
}
package package {
}
process process {
}
queue queue {
}
rectangle rectangle {
}
stack stack {
}
storage storage {
}
@enduml
```



8.8 Packages and nested elements

8.8.1 Example with one level

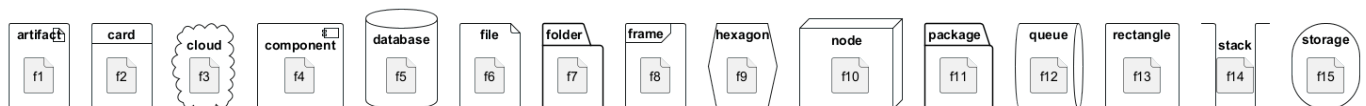
```
@startuml
artifact      artifactVeryL00000000000000000000g      as "artifact" {
file f1
}
card          cardVeryL00000000000000000000g          as "card" {
file f2
}
cloud         cloudVeryL00000000000000000000g         as "cloud" {
file f3
}
component     componentVeryL00000000000000000000g     as "component" {
file f4
}
```



```

}
database      databaseVeryL00000000000000000000g      as "database" {
file f5
}
file          fileVeryL00000000000000000000g          as "file" {
file f6
}
folder        folderVeryL00000000000000000000g        as "folder" {
file f7
}
frame         frameVeryL00000000000000000000g         as "frame" {
file f8
}
hexagon       hexagonVeryL00000000000000000000g       as "hexagon" {
file f9
}
node          nodeVeryL00000000000000000000g          as "node" {
file f10
}
package       packageVeryL00000000000000000000g       as "package" {
file f11
}
queue         queueVeryL00000000000000000000g         as "queue" {
file f12
}
rectangle     rectangleVeryL00000000000000000000g     as "rectangle" {
file f13
}
stack         stackVeryL00000000000000000000g         as "stack" {
file f14
}
storage       storageVeryL00000000000000000000g       as "storage" {
file f15
}
}
@enduml

```



8.8.2 Other example

```

@startuml
artifact Foo1 {
    folder Foo2
}

folder Foo3 {
    artifact Foo4
}

frame Foo5 {
    database Foo6
}

cloud vpc {
    node ec2 {

```

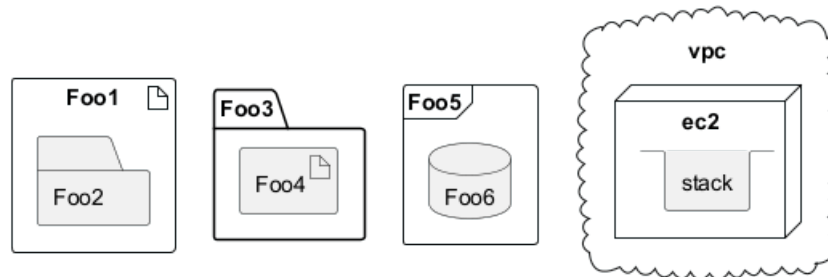


```

    stack stack
  }
}

@enduml

```



```

@startuml
node Foo1 {
  cloud Foo2
}

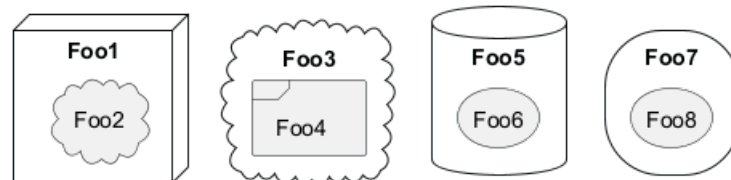
cloud Foo3 {
  frame Foo4
}

database Foo5 {
  storage Foo6
}

storage Foo7 {
  storage Foo8
}

@enduml

```



8.8.3 Full nesting

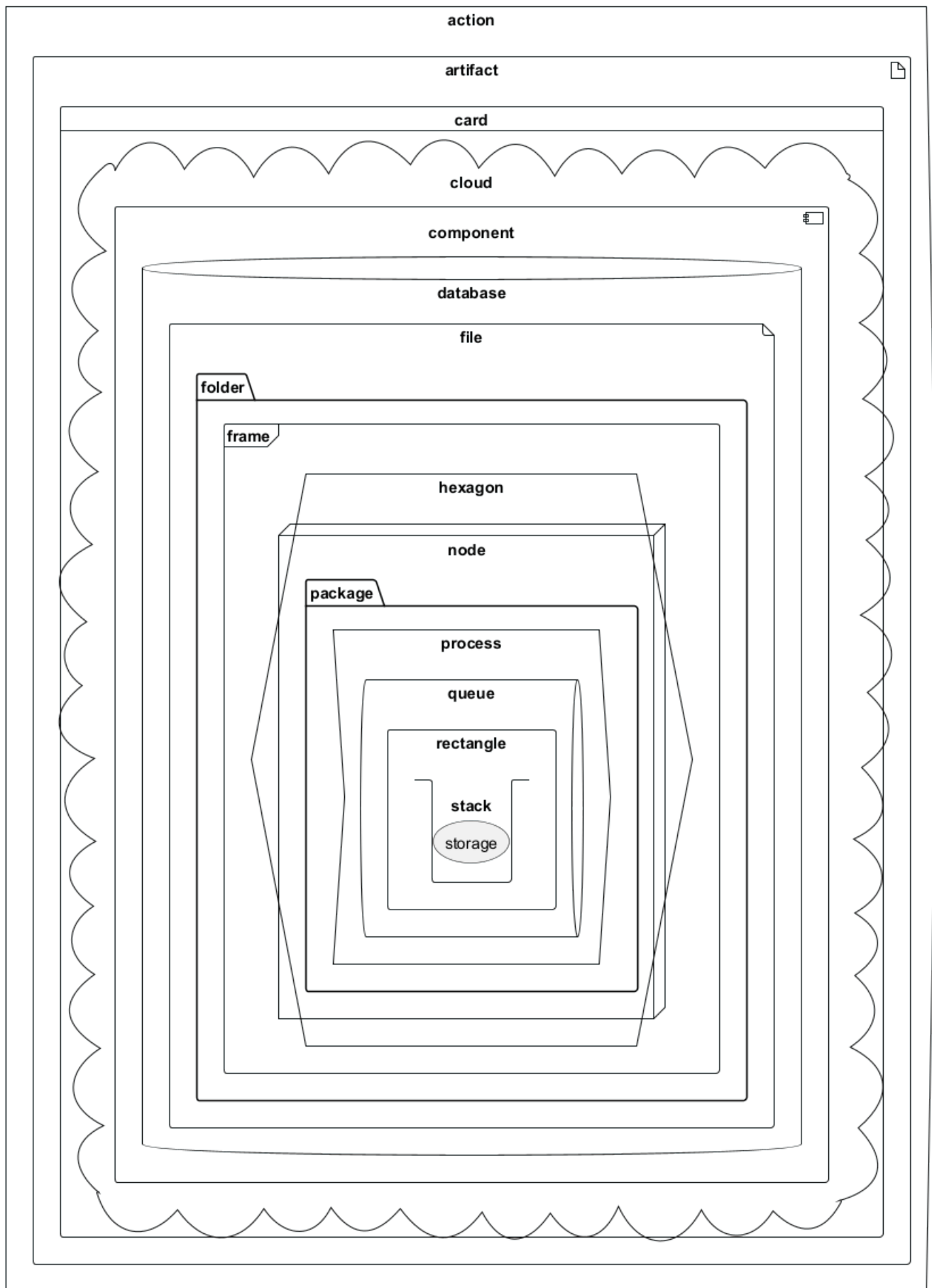
Here is all the nested elements:

- by alphabetical order:

```

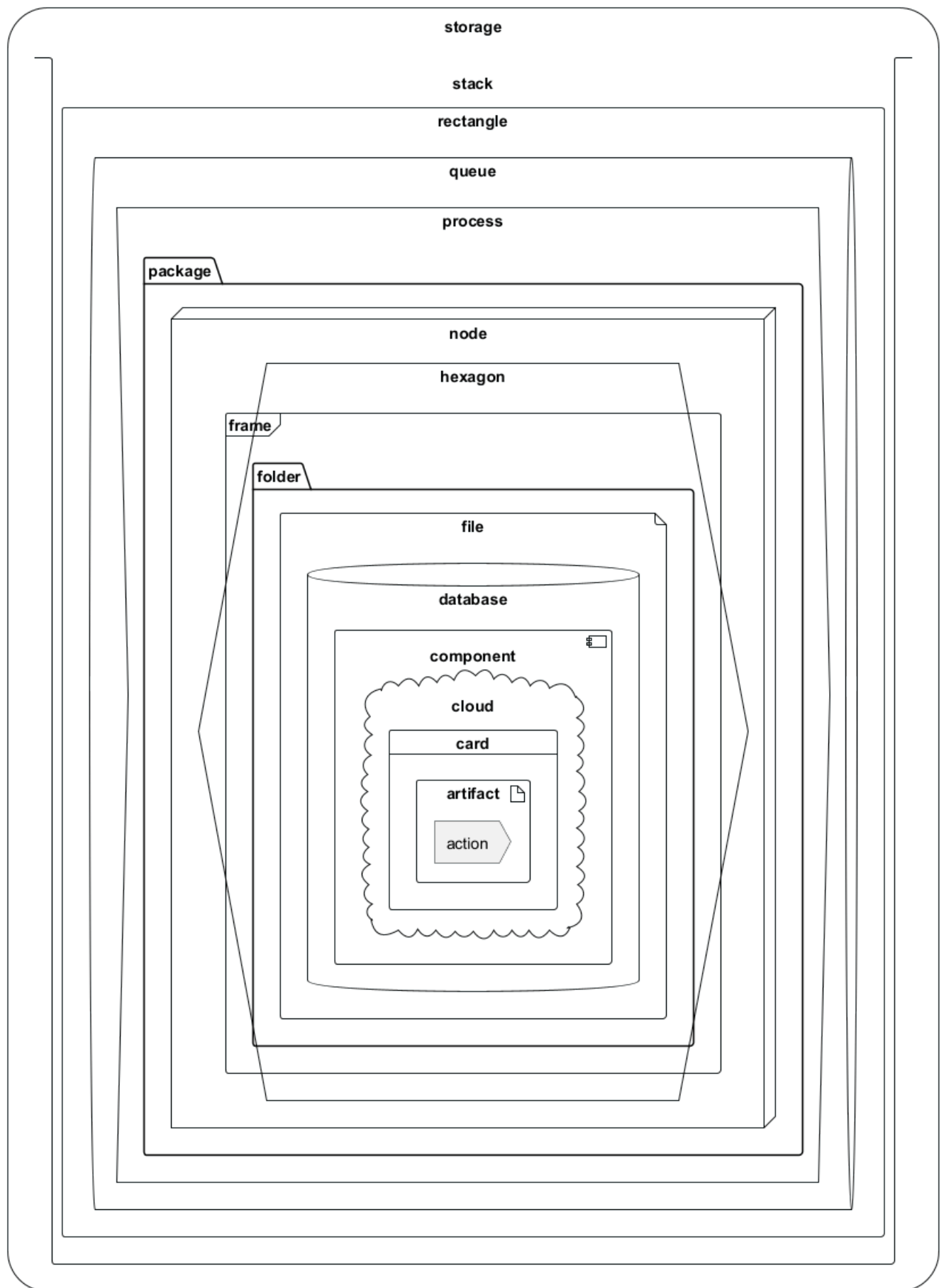
@startuml
action action {
artifact artifact {
card card {
cloud cloud {
component component {
database database {
file file {
folder folder {
frame frame {
hexagon hexagon {
node node {
package package {

```

- or reverse alphabetical order

[illegible]



8.9 Alias

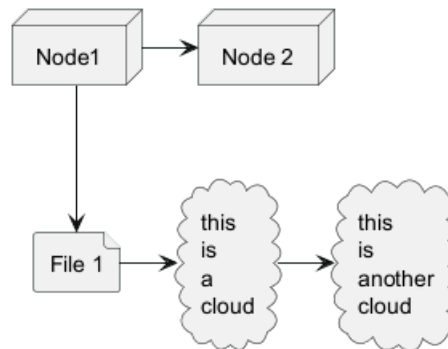
8.9.1 Simple alias with as

```

@startuml
node Node1 as n1
node "Node 2" as n2
file f1 as "File 1"
cloud c1 as "this
is
a
cloud"
cloud c2 [this
is
another
cloud]

n1 -> n2
n1 --> f1
f1 -> c1
c1 -> c2
@enduml

```



8.9.2 Examples of long alias

```

@startuml
actor      "actor"      as actorVeryL00000000000000000000g
agent      "agent"      as agentVeryL00000000000000000000g
artifact   "artifact"   as artifactVeryL00000000000000000000g
boundary   "boundary"   as boundaryVeryL00000000000000000000g
card       "card"       as cardVeryL00000000000000000000g
cloud      "cloud"      as cloudVeryL00000000000000000000g
collections "collections" as collectionsVeryL00000000000000000000g
component  "component"  as componentVeryL00000000000000000000g
control    "control"    as controlVeryL00000000000000000000g
database   "database"   as databaseVeryL00000000000000000000g
entity     "entity"     as entityVeryL00000000000000000000g
file       "file"       as fileVeryL00000000000000000000g
folder     "folder"     as folderVeryL00000000000000000000g
frame      "frame"      as frameVeryL00000000000000000000g
hexagon    "hexagon"    as hexagonVeryL00000000000000000000g
interface  "interface"  as interfaceVeryL00000000000000000000g
label      "label"      as labelVeryL00000000000000000000g
node       "node"       as nodeVeryL00000000000000000000g
package    "package"    as packageVeryL00000000000000000000g
person     "person"     as personVeryL00000000000000000000g
queue      "queue"      as queueVeryL00000000000000000000g

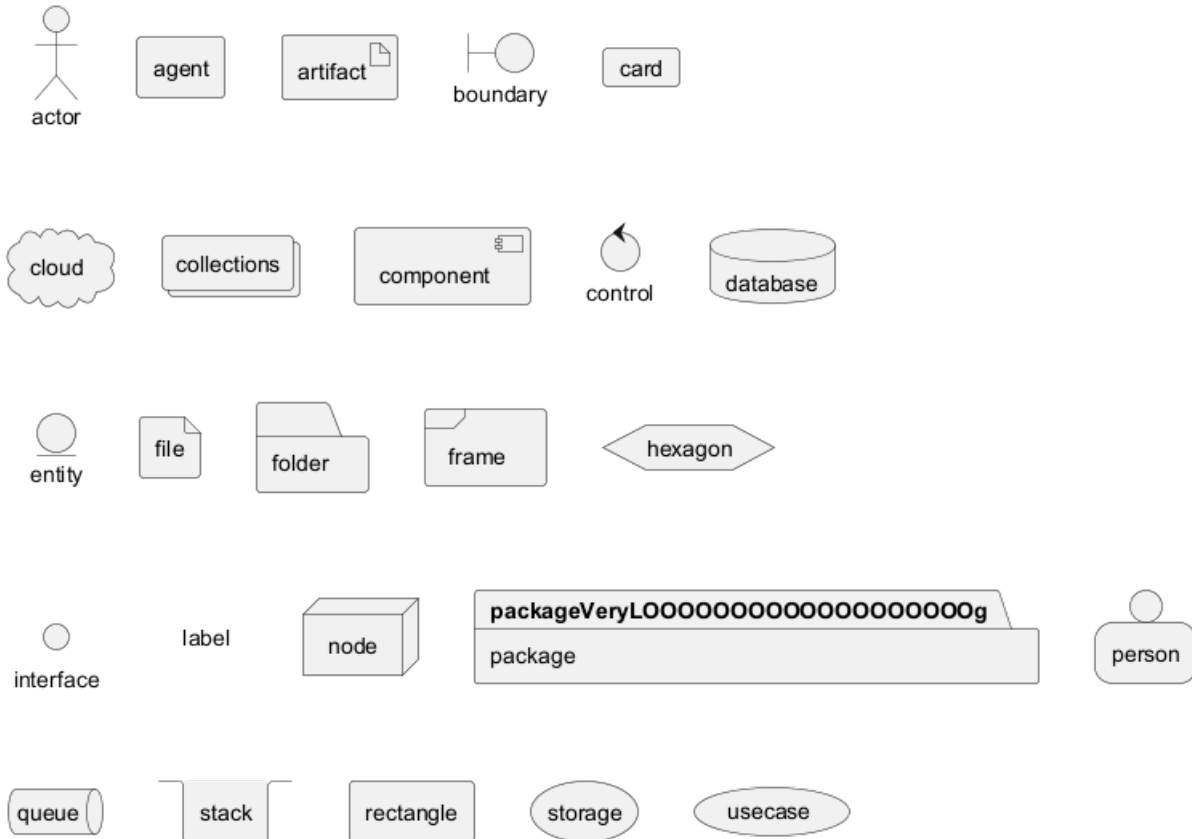
```




```

stack      "stack"      as stackVeryL00000000000000000000g
rectangle  "rectangle"  as rectangleVeryL00000000000000000000g
storage    "storage"    as storageVeryL00000000000000000000g
usecase    "usecase"    as usecaseVeryL00000000000000000000g
@enduml

```



```

@startuml
actor      actorVeryL00000000000000000000g      as "actor"
agent      agentVeryL00000000000000000000g      as "agent"
artifact   artifactVeryL00000000000000000000g   as "artifact"
boundary   boundaryVeryL00000000000000000000g   as "boundary"
card       cardVeryL00000000000000000000g       as "card"
cloud      cloudVeryL00000000000000000000g      as "cloud"
collections collectionsVeryL00000000000000000000g as "collections"
component  componentVeryL00000000000000000000g  as "component"
control    controlVeryL00000000000000000000g    as "control"
database   databaseVeryL00000000000000000000g   as "database"
entity     entityVeryL00000000000000000000g     as "entity"
file       fileVeryL00000000000000000000g       as "file"
folder     folderVeryL00000000000000000000g     as "folder"
frame      frameVeryL00000000000000000000g      as "frame"
hexagon    hexagonVeryL00000000000000000000g    as "hexagon"
interface  interfaceVeryL00000000000000000000g  as "interface"
label      labelVeryL00000000000000000000g      as "label"
node       nodeVeryL00000000000000000000g       as "node"
package    packageVeryL00000000000000000000g    as "package"
person     personVeryL00000000000000000000g     as "person"
queue      queueVeryL00000000000000000000g      as "queue"
stack      stackVeryL00000000000000000000g      as "stack"
rectangle  rectangleVeryL00000000000000000000g  as "rectangle"
storage    storageVeryL00000000000000000000g    as "storage"

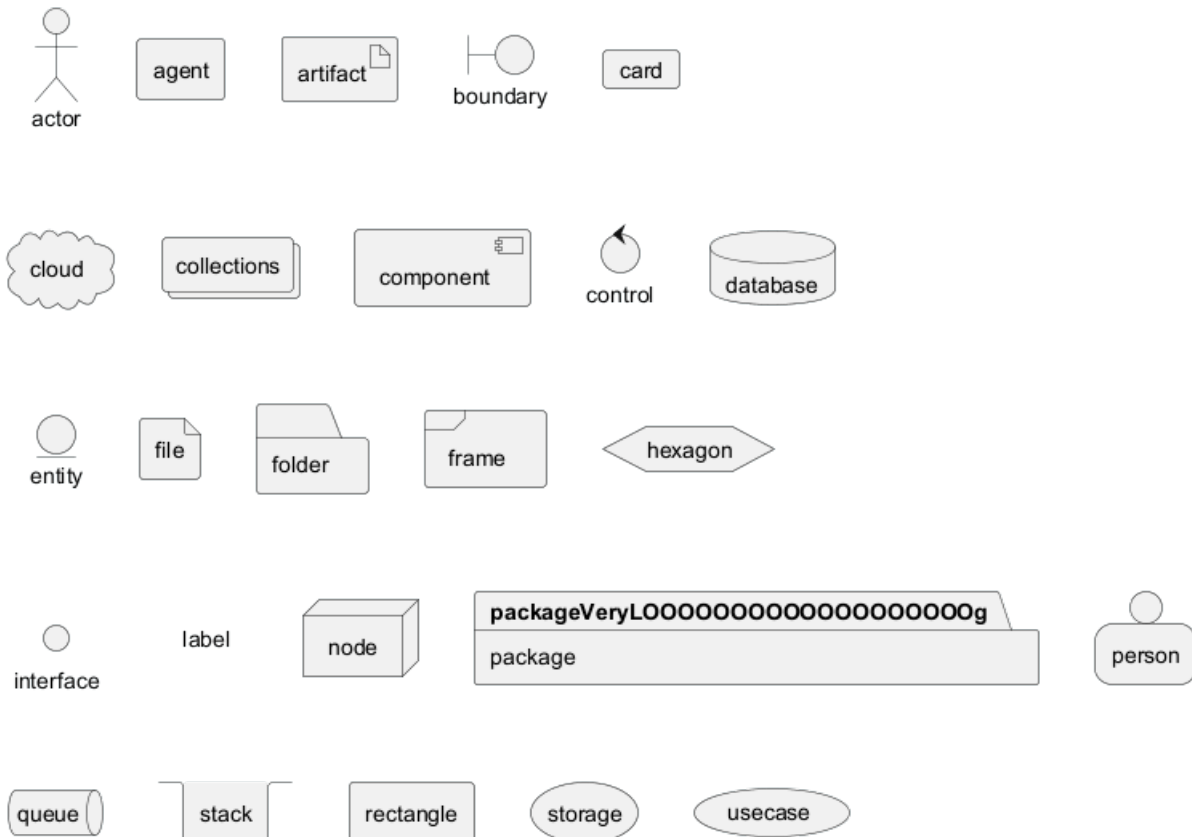
```



```

usecase
usecaseVeryLO00000000000000000000g as "usecase"
@enduml

```



[Ref. QA-12082]

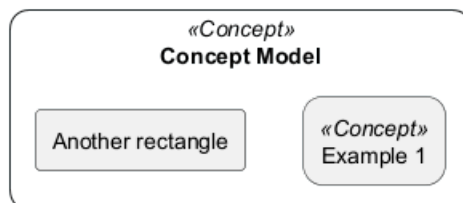
8.10 Round corner

```

@startuml
skinparam rectangle {
    roundCorner<<Concept>> 25
}

rectangle "Concept Model" <<Concept>> {
    rectangle "Example 1" <<Concept>> as ex1
    rectangle "Another rectangle"
}
@enduml

```



8.11 Specific SkinParameter

8.11.1 roundCorner

```

@startuml
skinparam roundCorner 15
actor actor

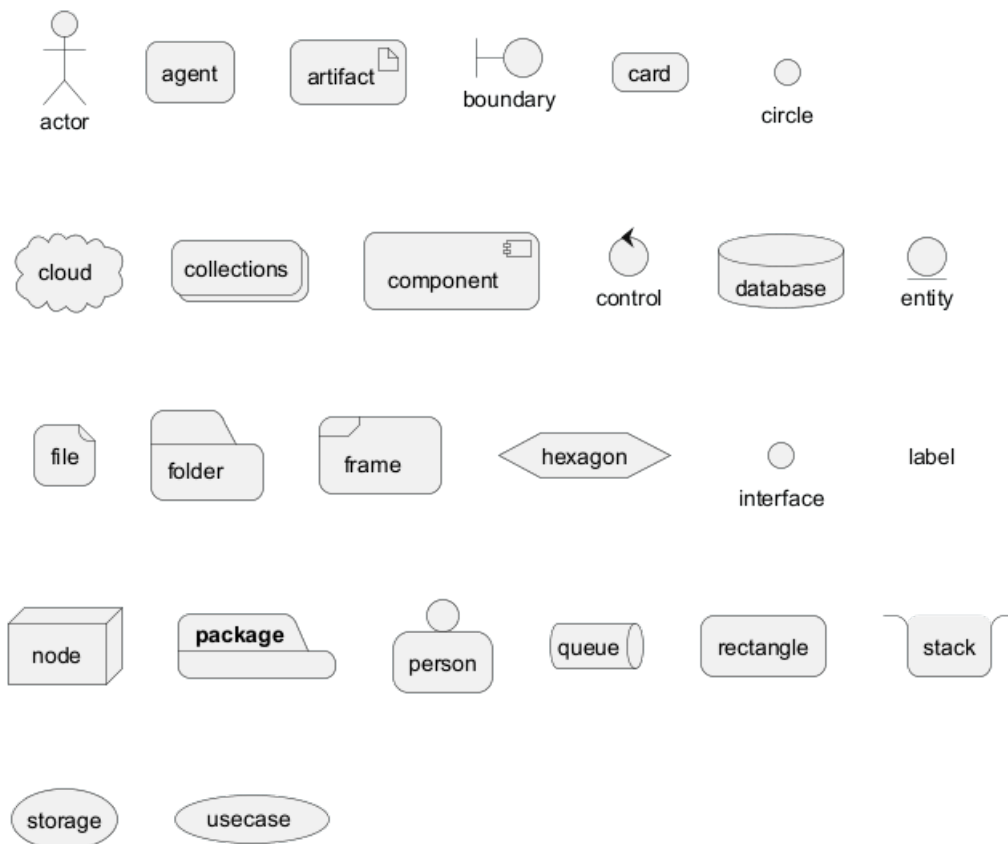
```



```

agent agent
artifact artifact
boundary boundary
card card
circle circle
cloud cloud
collections collections
component component
control control
database database
entity entity
file file
folder folder
frame frame
hexagon hexagon
interface interface
label label
node node
package package
person person
queue queue
rectangle rectangle
stack stack
storage storage
usecase usecase
@enduml

```



[Ref. QA-5299, QA-6915, QA-11943]

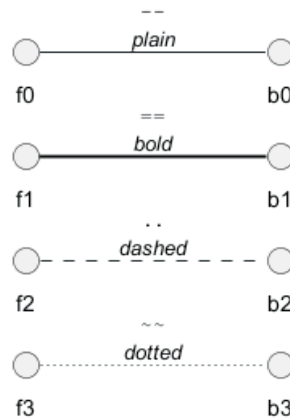
8.12 Appendix: All type of arrow line

```
@startuml
```



left to right direction
skinparam nodesep 5

```
f3 ~~ b3 : ""~~""\n//dotted//
f2 .. b2 : ""..""\n//dashed//
f1 == b1 : ""==""\n//bold//
f0 -- b0 : ""--""\n//plain//
@enduml
```



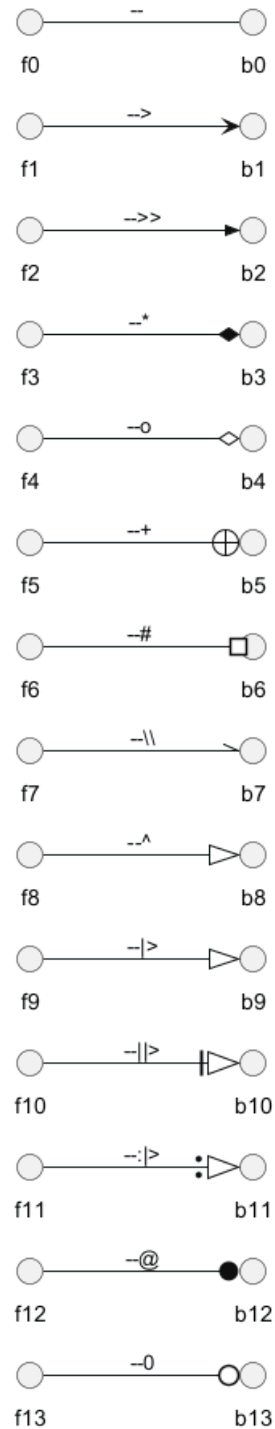
8.13 Appendix: All type of arrow head or '0' arrow

8.13.1 Type of arrow head

@startuml
left to right direction
skinparam nodesep 5

```
f13 --0 b13 : ""--0""
f12 --@ b12 : ""--@""
f11 --:|> b11 : ""--:|>""
f10 --||> b10 : ""--||>""
f9 --|> b9 : ""--|>""
f8 --^ b8 : ""--^ ""
f7 --\\ b7 : ""--\\\\ ""
f6 --# b6 : ""--# ""
f5 --+ b5 : ""--+ ""
f4 --o b4 : ""--o ""
f3 --* b3 : ""--* ""
f2 -->> b2 : ""-->>""
f1 --> b1 : ""--> ""
f0 -- b0 : ""-- ""
@enduml
```





8.13.2 Type of '0' arrow or circle arrow

```
@startuml
left to right direction
skinparam nodesep 5

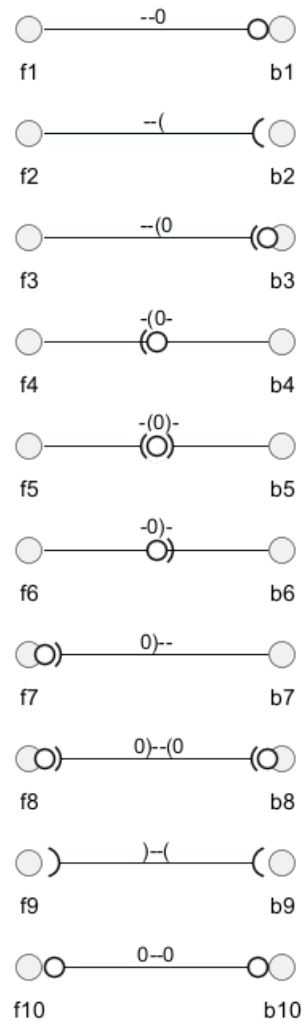
f10 0--0 b10 : "" 0--0 ""
f9 )--( b9 : "" )--( ""
f8 0)--(0 b8 : "" 0)--(0""
f7 0)-- b7 : "" 0)-- ""
f6 -0)- b6 : "" -0)- ""
f5 -(0)- b5 : "" -(0)-""
```



```

f4 -(0- b4 : "" -(0- ""
f3 --(0 b3 : "" --(0 ""
f2 --( b2 : "" --( ""
f1 --0 b1 : "" --0 ""
@enduml

```



8.14 Appendix: Test of inline style on all element

8.14.1 Simple element

```

@startuml
action action           #aliceblue;line:blue;line.dotted;text:blue
actor actor             #aliceblue;line:blue;line.dotted;text:blue
actor/ "actor/"         #aliceblue;line:blue;line.dotted;text:blue
agent agent             #aliceblue;line:blue;line.dotted;text:blue
artifact artifact       #aliceblue;line:blue;line.dotted;text:blue
boundary boundary       #aliceblue;line:blue;line.dotted;text:blue
card card               #aliceblue;line:blue;line.dotted;text:blue
circle circle           #aliceblue;line:blue;line.dotted;text:blue
cloud cloud             #aliceblue;line:blue;line.dotted;text:blue
collections collections #aliceblue;line:blue;line.dotted;text:blue
component component     #aliceblue;line:blue;line.dotted;text:blue
control control         #aliceblue;line:blue;line.dotted;text:blue
database database       #aliceblue;line:blue;line.dotted;text:blue
entity entity           #aliceblue;line:blue;line.dotted;text:blue
file file               #aliceblue;line:blue;line.dotted;text:blue

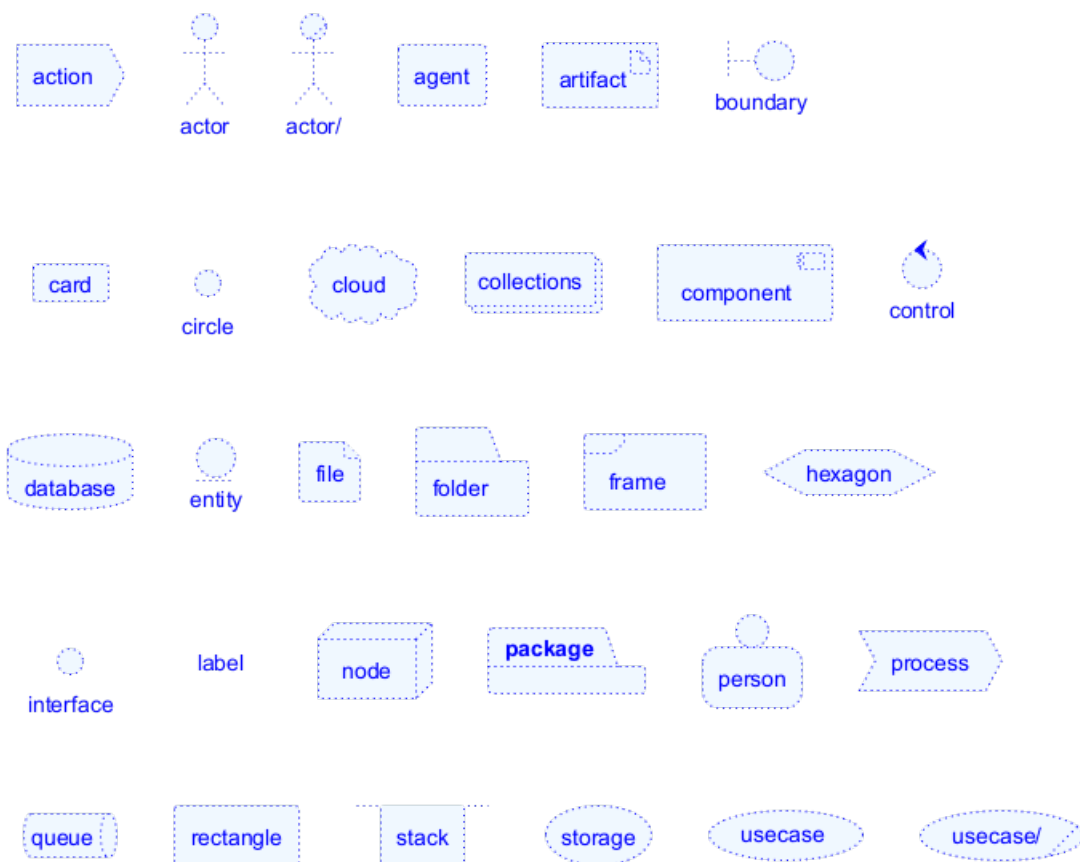
```



```

folder folder      #aliceblue;line:blue;line.dotted;text:blue
frame frame        #aliceblue;line:blue;line.dotted;text:blue
hexagon hexagon    #aliceblue;line:blue;line.dotted;text:blue
interface interface #aliceblue;line:blue;line.dotted;text:blue
label label        #aliceblue;line:blue;line.dotted;text:blue
node node          #aliceblue;line:blue;line.dotted;text:blue
package package    #aliceblue;line:blue;line.dotted;text:blue
person person      #aliceblue;line:blue;line.dotted;text:blue
process process    #aliceblue;line:blue;line.dotted;text:blue
queue queue        #aliceblue;line:blue;line.dotted;text:blue
rectangle rectangle #aliceblue;line:blue;line.dotted;text:blue
stack stack        #aliceblue;line:blue;line.dotted;text:blue
storage storage    #aliceblue;line:blue;line.dotted;text:blue
usecase usecase    #aliceblue;line:blue;line.dotted;text:blue
usecase/ "usecase/" #aliceblue;line:blue;line.dotted;text:blue
@enduml

```



8.14.2 Nested element

8.14.3 Without sub-element

```

@startuml
action action #aliceblue;line:blue;line.dotted;text:blue {
}
artifact artifact #aliceblue;line:blue;line.dotted;text:blue {
}
card card #aliceblue;line:blue;line.dotted;text:blue {
}
cloud cloud #aliceblue;line:blue;line.dotted;text:blue {
}
component component #aliceblue;line:blue;line.dotted;text:blue {
}

```



```

}
database database #aliceblue;line:blue;line.dotted;text:blue {
}
file file #aliceblue;line:blue;line.dotted;text:blue {
}
folder folder #aliceblue;line:blue;line.dotted;text:blue {
}
frame frame #aliceblue;line:blue;line.dotted;text:blue {
}
hexagon hexagon #aliceblue;line:blue;line.dotted;text:blue {
}
node node #aliceblue;line:blue;line.dotted;text:blue {
}
package package #aliceblue;line:blue;line.dotted;text:blue {
}
process process #aliceblue;line:blue;line.dotted;text:blue {
}
queue queue #aliceblue;line:blue;line.dotted;text:blue {
}
rectangle rectangle #aliceblue;line:blue;line.dotted;text:blue {
}
stack stack #aliceblue;line:blue;line.dotted;text:blue {
}
storage storage #aliceblue;line:blue;line.dotted;text:blue {
}
@enduml

```



8.14.4 With sub-element

```

@startuml
action      actionVeryL00000000000000000000g      as "action" #aliceblue;line:blue;line.dotted;text:blue {
file f1
}
artifact    artifactVeryL00000000000000000000g    as "artifact" #aliceblue;line:blue;line.dotted;text:blue {
file f1
}
card        cardVeryL00000000000000000000g        as "card" #aliceblue;line:blue;line.dotted;text:blue {
file f2
}
cloud       cloudVeryL00000000000000000000g       as "cloud" #aliceblue;line:blue;line.dotted;text:blue {
file f3
}
component   componentVeryL00000000000000000000g   as "component" #aliceblue;line:blue;line.dotted;text:blue {
file f4
}
database    databaseVeryL00000000000000000000g    as "database" #aliceblue;line:blue;line.dotted;text:blue {
file f5
}
file        fileVeryL00000000000000000000g        as "file" #aliceblue;line:blue;line.dotted;text:blue {
file f6
}
folder      folderVeryL00000000000000000000g      as "folder" #aliceblue;line:blue;line.dotted;text:blue {
file f7
}
frame       frameVeryL00000000000000000000g       as "frame" #aliceblue;line:blue;line.dotted;text:blue {
}

```




```

file f8
}
hexagon    hexagonVeryL00000000000000000000g    as "hexagon" #aliceblue;line:blue;line.dotted;text:bl
file f9
}
node       nodeVeryL00000000000000000000g       as "node" #aliceblue;line:blue;line.dotted;text:blu
file f10
}
package    packageVeryL00000000000000000000g    as "package" #aliceblue;line:blue;line.dotted;text:bl
file f11
}
process     processVeryL00000000000000000000g    as "process" #aliceblue;line:blue;line.dotted;text:bl
file f11
}
queue      queueVeryL00000000000000000000g      as "queue" #aliceblue;line:blue;line.dotted;text:bl
file f12
}
rectangle  rectangleVeryL00000000000000000000g  as "rectangle" #aliceblue;line:blue;line.dotted;text:bl
file f13
}
stack      stackVeryL00000000000000000000g      as "stack" #aliceblue;line:blue;line.dotted;text:bl
file f14
}
storage    storageVeryL00000000000000000000g    as "storage" #aliceblue;line:blue;line.dotted;text:bl
file f15
}
@enduml

```



8.15 Appendix: Test of style on all element

8.15.1 Simple element

8.15.2 Global style (on componentDiagram)

```

@startuml
<style>
componentDiagram {
    BackgroundColor palegreen
    LineThickness 1
    LineColor red
}
document {
    BackgroundColor white
}
</style>
actor actor
actor/ "actor/"
agent agent
artifact artifact
boundary boundary
card card
circle circle
cloud cloud
collections collections
component component

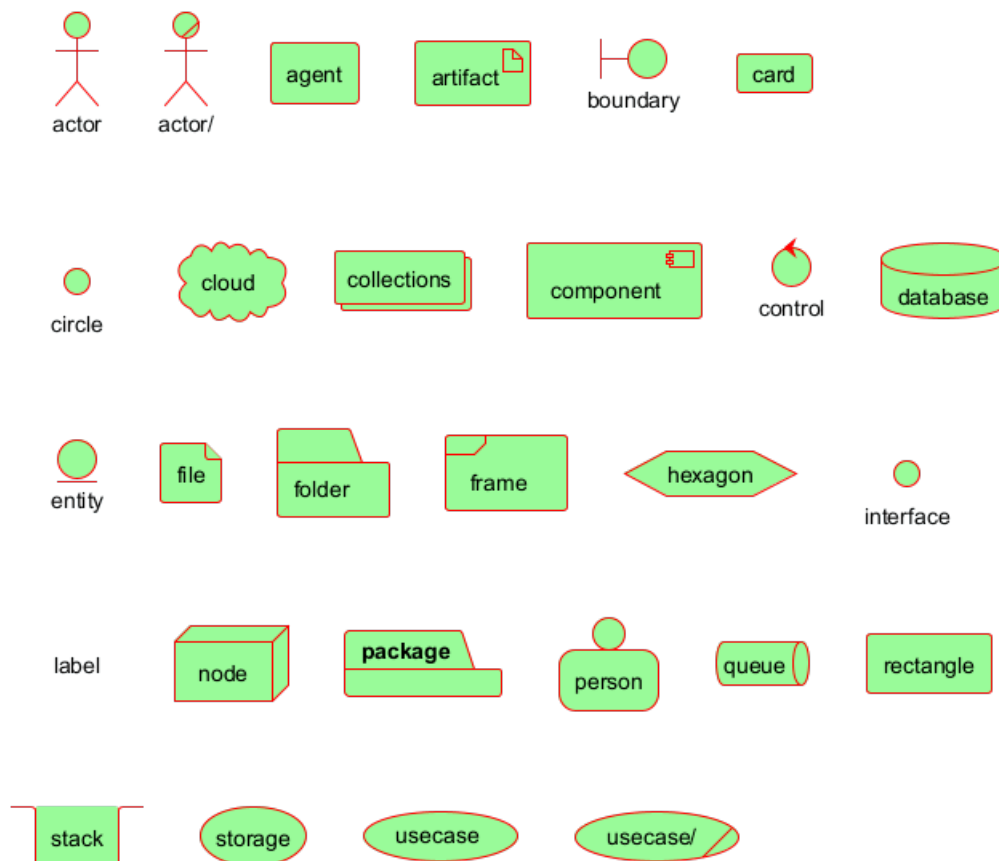
```



```

control control
database database
entity entity
file file
folder folder
frame frame
hexagon hexagon
interface interface
label label
node node
package package
person person
queue queue
rectangle rectangle
stack stack
storage storage
usecase usecase
usecase/ "usecase/"
@enduml

```



8.15.3 Style for each element

```

@startuml
<style>
actor {
  BackgroundColor #f80c12
  LineThickness 1
  LineColor black
}
agent {
  BackgroundColor #f80c12

```



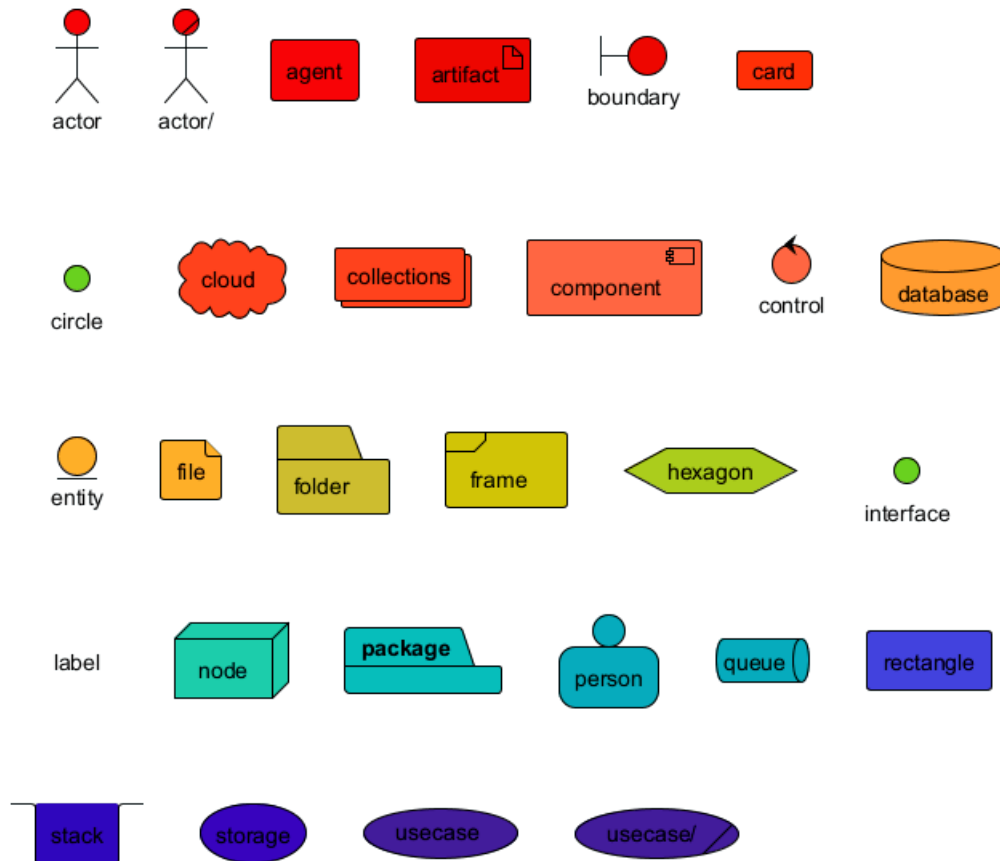
```
    LineThickness 1
    LineColor black
}
artifact {
    BackGroundColor #ee1100
    LineThickness 1
    LineColor black
}
boundary {
    BackGroundColor #ee1100
    LineThickness 1
    LineColor black
}
card {
    BackGroundColor #ff3311
    LineThickness 1
    LineColor black
}
circle {
    BackGroundColor #ff3311
    LineThickness 1
    LineColor black
}
cloud {
    BackGroundColor #ff4422
    LineThickness 1
    LineColor black
}
collections {
    BackGroundColor #ff4422
    LineThickness 1
    LineColor black
}
component {
    BackGroundColor #ff6644
    LineThickness 1
    LineColor black
}
control {
    BackGroundColor #ff6644
    LineThickness 1
    LineColor black
}
database {
    BackGroundColor #ff9933
    LineThickness 1
    LineColor black
}
entity {
    BackGroundColor #feae2d
    LineThickness 1
    LineColor black
}
file {
    BackGroundColor #feae2d
    LineThickness 1
    LineColor black
}
```



```
folder {
  BackGroundColor #ccbb33
  LineThickness 1
  LineColor black
}
frame {
  BackGroundColor #d0c310
  LineThickness 1
  LineColor black
}
hexagon {
  BackGroundColor #aacc22
  LineThickness 1
  LineColor black
}
interface {
  BackGroundColor #69d025
  LineThickness 1
  LineColor black
}
label {
  BackGroundColor black
  LineThickness 1
  LineColor black
}
node {
  BackGroundColor #22ccaa
  LineThickness 1
  LineColor black
}
package {
  BackGroundColor #12bdb9
  LineThickness 1
  LineColor black
}
person {
  BackGroundColor #11aabb
  LineThickness 1
  LineColor black
}
queue {
  BackGroundColor #11aabb
  LineThickness 1
  LineColor black
}
rectangle {
  BackGroundColor #4444dd
  LineThickness 1
  LineColor black
}
stack {
  BackGroundColor #3311bb
  LineThickness 1
  LineColor black
}
storage {
  BackGroundColor #3b0cbd
  LineThickness 1
}
```



```
    LineColor black
}
usecase {
    BackGroundColor #442299
    LineThickness 1
    LineColor black
}
</style>
actor actor
actor/ "actor/"
agent agent
artifact artifact
boundary boundary
card card
circle circle
cloud cloud
collections collections
component component
control control
database database
entity entity
file file
folder folder
frame frame
hexagon hexagon
interface interface
label label
node node
package package
person person
queue queue
rectangle rectangle
stack stack
storage storage
usecase usecase
usecase/ "usecase/"
@enduml
```



[Ref. QA-13261]

8.15.4 Nested element (without level)

8.15.5 Global style (on componentDiagram)

```
@startuml
<style>
componentDiagram {
  BackGroundColor palegreen
  LineThickness 2
  LineColor red
}
</style>
artifact artifact {
}
card card {
}
cloud cloud {
}
component component {
}
database database {
}
file file {
}
folder folder {
}
frame frame {
}
hexagon hexagon {
```



```

}
node node {
}
package package {
}
queue queue {
}
rectangle rectangle {
}
stack stack {
}
storage storage {
}
@enduml

```



8.15.6 Style for each nested element

```

@startuml
<style>
artifact {
    BackGroundColor #ee1100
    LineThickness 1
    LineColor black
}
card {
    BackGroundColor #ff3311
    LineThickness 1
    LineColor black
}
cloud {
    BackGroundColor #ff4422
    LineThickness 1
    LineColor black
}
component {
    BackGroundColor #ff6644
    LineThickness 1
    LineColor black
}
database {
    BackGroundColor #ff9933
    LineThickness 1
    LineColor black
}
file {
    BackGroundColor #feae2d
    LineThickness 1
    LineColor black
}
folder {
    BackGroundColor #ccbb33
    LineThickness 1
    LineColor black
}
frame {

```



```

    BackGroundColor #d0c310
    LineThickness 1
    LineColor black
}
hexagon {
    BackGroundColor #aacc22
    LineThickness 1
    LineColor black
}
node {
    BackGroundColor #22ccaa
    LineThickness 1
    LineColor black
}
package {
    BackGroundColor #12bdb9
    LineThickness 1
    LineColor black
}
queue {
    BackGroundColor #11aabb
    LineThickness 1
    LineColor black
}
rectangle {
    BackGroundColor #4444dd
    LineThickness 1
    LineColor black
}
stack {
    BackGroundColor #3311bb
    LineThickness 1
    LineColor black
}
storage {
    BackGroundColor #3b0cbd
    LineThickness 1
    LineColor black
}

</style>
artifact artifact {
}
card card {
}
cloud cloud {
}
component component {
}
database database {
}
file file {
}
folder folder {
}
frame frame {
}
hexagon hexagon {

```




```

}
node node {
}
package package {
}
queue queue {
}
rectangle rectangle {
}
stack stack {
}
storage storage {
}
@enduml

```



8.15.7 Nested element (with one level)

8.15.8 Global style (on componentDiagram)

```

@startuml
<style>
componentDiagram {
  BackgroundColor palegreen
  LineThickness 1
  LineColor red
}
document {
  BackgroundColor white
}
</style>
artifact e1 as "artifact" {
file f1
}
card e2 as "card" {
file f2
}
cloud e3 as "cloud" {
file f3
}
component e4 as "component" {
file f4
}
database e5 as "database" {
file f5
}
file e6 as "file" {
file f6
}
folder e7 as "folder" {
file f7
}
frame e8 as "frame" {
file f8
}
hexagon e9 as "hexagon" {
file f9
}

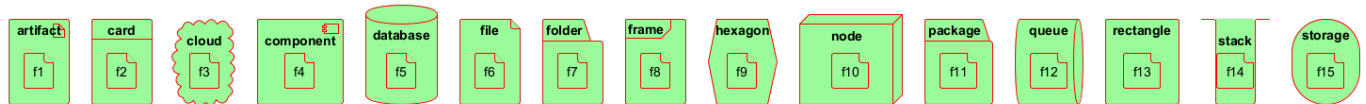
```



```

}
node e10 as "node" {
file f10
}
package e11 as "package" {
file f11
}
queue e12 as "queue" {
file f12
}
rectangle e13 as "rectangle" {
file f13
}
stack e14 as "stack" {
file f14
}
storage e15 as "storage" {
file f15
}
@enduml

```



8.15.9 Style for each nested element

```

@startuml
<style>
artifact {
  BackGroundColor #ee1100
  LineThickness 1
  LineColor black
}
card {
  BackGroundColor #ff3311
  LineThickness 1
  LineColor black
}
cloud {
  BackGroundColor #ff4422
  LineThickness 1
  LineColor black
}
component {
  BackGroundColor #ff6644
  LineThickness 1
  LineColor black
}
database {
  BackGroundColor #ff9933
  LineThickness 1
  LineColor black
}
file {
  BackGroundColor #feae2d
  LineThickness 1
}

```



```

    LineColor black
}
folder {
    BackGroundColor #ccbb33
    LineThickness 1
    LineColor black
}
frame {
    BackGroundColor #d0c310
    LineThickness 1
    LineColor black
}
hexagon {
    BackGroundColor #aacc22
    LineThickness 1
    LineColor black
}
node {
    BackGroundColor #22ccaa
    LineThickness 1
    LineColor black
}
package {
    BackGroundColor #12bdb9
    LineThickness 1
    LineColor black
}
queue {
    BackGroundColor #11aabb
    LineThickness 1
    LineColor black
}
rectangle {
    BackGroundColor #4444dd
    LineThickness 1
    LineColor black
}
stack {
    BackGroundColor #3311bb
    LineThickness 1
    LineColor black
}
storage {
    BackGroundColor #3b0cbd
    LineThickness 1
    LineColor black
}
</style>
artifact e1 as "artifact" {
file f1
}
card e2 as "card" {
file f2
}
cloud e3 as "cloud" {
file f3
}
component e4 as "component" {

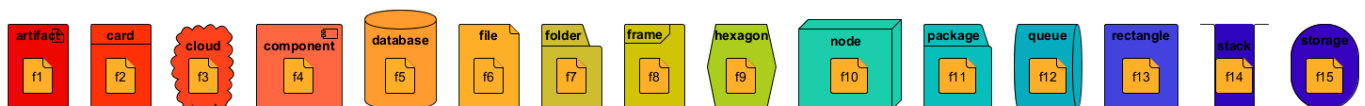
```



```

file f4
}
database e5 as "database" {
file f5
}
file e6 as "file" {
file f6
}
folder e7 as "folder" {
file f7
}
frame e8 as "frame" {
file f8
}
hexagon e9 as "hexagon" {
file f9
}
node e10 as "node" {
file f10
}
package e11 as "package" {
file f11
}
queue e12 as "queue" {
file f12
}
rectangle e13 as "rectangle" {
file f13
}
stack e14 as "stack" {
file f14
}
storage e15 as "storage" {
file f15
}
@enduml

```



8.16 Appendix: Test of stereotype with style on all element

8.16.1 Simple element

```

@startuml
<style>
.stereo {
  BackgroundColor palegreen
}
</style>
actor actor << stereo >>
actor/ "actor/" << stereo >>
agent agent << stereo >>
artifact artifact << stereo >>
boundary boundary << stereo >>
card card << stereo >>
circle circle << stereo >>

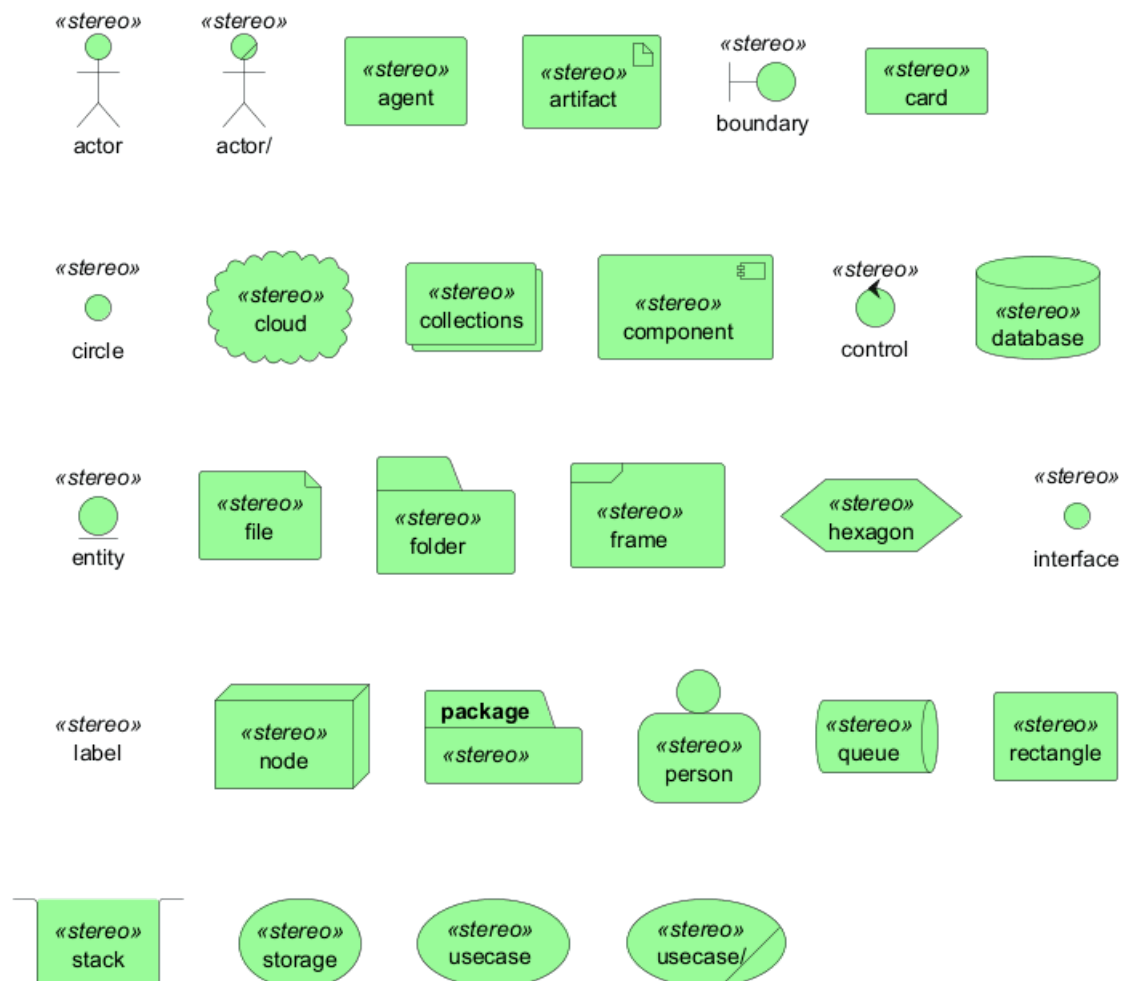
```



```

cloud cloud << stereo >>
collections collections << stereo >>
component component << stereo >>
control control << stereo >>
database database << stereo >>
entity entity << stereo >>
file file << stereo >>
folder folder << stereo >>
frame frame << stereo >>
hexagon hexagon << stereo >>
interface interface << stereo >>
label label << stereo >>
node node << stereo >>
package package << stereo >>
person person << stereo >>
queue queue << stereo >>
rectangle rectangle << stereo >>
stack stack << stereo >>
storage storage << stereo >>
usecase usecase << stereo >>
usecase/ "usecase/" << stereo >>
@enduml

```



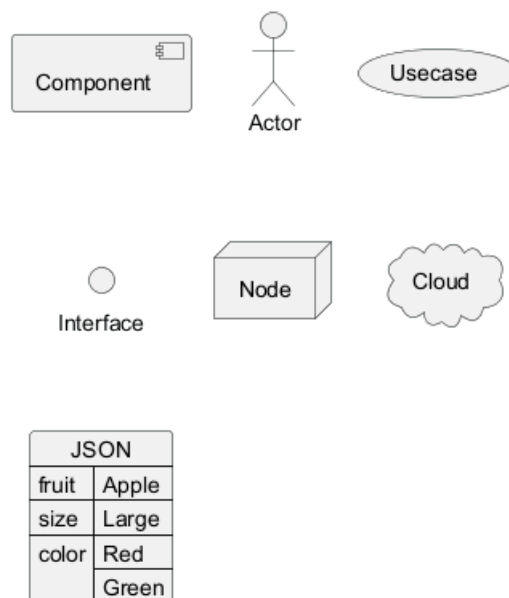
8.17 Display JSON Data on Deployment diagram

8.17.1 Simple example

```
@startuml
allowmixing

component Component
actor Actor
usecase Usecase
() Interface
node Node
cloud Cloud

json JSON {
    "fruit": "Apple",
    "size": "Large",
    "color": ["Red", "Green"]
}
@enduml
```



[Ref. QA-15481]

For another example, see on JSON page.

8.18 Mixing Deployment (Usecase, Component, Deployment) element within a Class or Object diagram

In order to add a Deployment element or a State element within a Class or Object diagram, you can use the `allowmixing` or `allow_mixing` directive.

8.18.1 Mixing all elements

```
@startuml
allowmixing

skinparam nodesep 10
abstract abstract
abstract class "abstract class"
annotation annotation
```

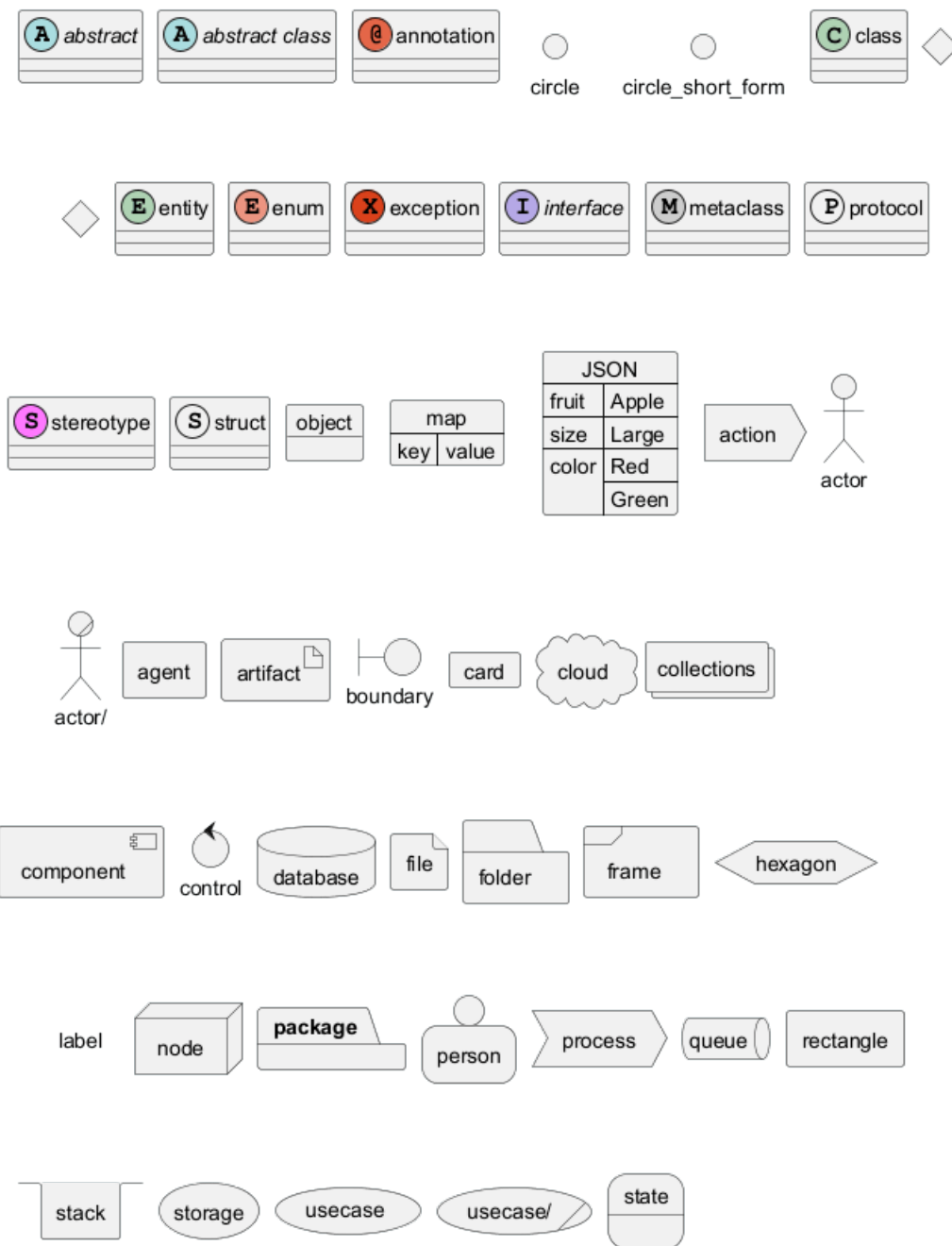


```

circle          circle
()              circle_short_form
class           class
diamond         diamond
<>             diamond_short_form
entity          entity
enum            enum
exception       exception
interface       interface
metaclass       metaclass
protocol        protocol
stereotype      stereotype
struct          struct
object          object
map map {
  key => value
}
json JSON {
  "fruit": "Apple",
  "size": "Large",
  "color": ["Red", "Green"]
}
action action
actor actor
actor/ "actor/"
agent agent
artifact artifact
boundary boundary
card card
circle circle
cloud cloud
collections collections
component component
control control
database database
entity entity
file file
folder folder
frame frame
hexagon hexagon
interface interface
label label
node node
package package
person person
process process
queue queue
rectangle rectangle
stack stack
storage storage
usecase usecase
usecase/ "usecase/"
state state
@enduml

```





[Ref. QA-2335 and QA-5329]

8.19 Port [port, portIn, portOut]

You can add **port** with **port**, **portIn** and **portOut** keywords.

8.19.1 Port

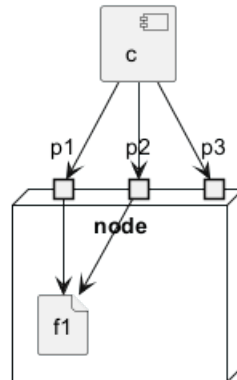
```
@startuml
[c]
node node {
    port p1
    port p2
    port p3
    file f1
}
```




```

c --> p1
c --> p2
c --> p3
p1 --> f1
p2 --> f1
@enduml

```



8.19.2 PortIn

```

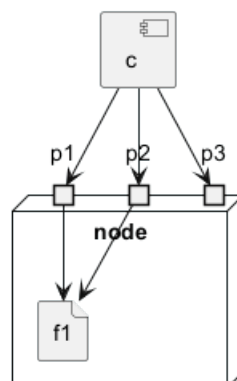
@startuml
[c]
node node {
    portin p1
    portin p2
    portin p3
    file f1
}

```

```

c --> p1
c --> p2
c --> p3
p1 --> f1
p2 --> f1
@enduml

```



8.19.3 PortOut

```

@startuml
node node {
    portout p1
    portout p2
}

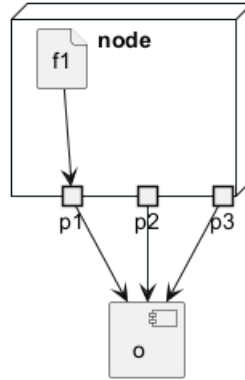
```



```

    portout p3
    file f1
}
[o]
p1 --> o
p2 --> o
p3 --> o
f1 --> p1
@enduml

```



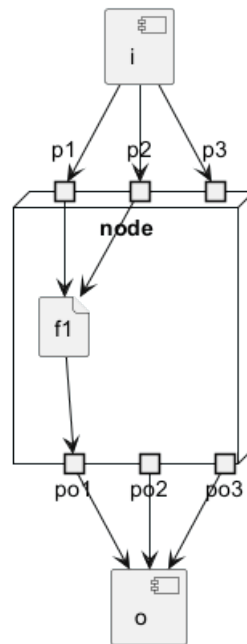
8.19.4 Mixing PortIn & PortOut

```

@startuml
[i]
node node {
    portin p1
    portin p2
    portin p3
    portout po1
    portout po2
    portout po3
    file f1
}
[o]

i --> p1
i --> p2
i --> p3
p1 --> f1
p2 --> f1
po1 --> o
po2 --> o
po3 --> o
f1 --> po1
@enduml

```



8.20 Change diagram orientation

You can change (whole) diagram orientation with:

- top to bottom direction (*by default*)
- left to right direction

8.20.1 Top to bottom (*by default*)

8.20.2 With Graphviz (*layout engine by default*)

The main rule is: **Nested element first, then simple element.**

```
@startuml
card a
card b
package A {
  card a1
  card a2
  card a3
  card a4
  card a5
  package sub_a {
    card sa1
    card sa2
    card sa3
  }
}

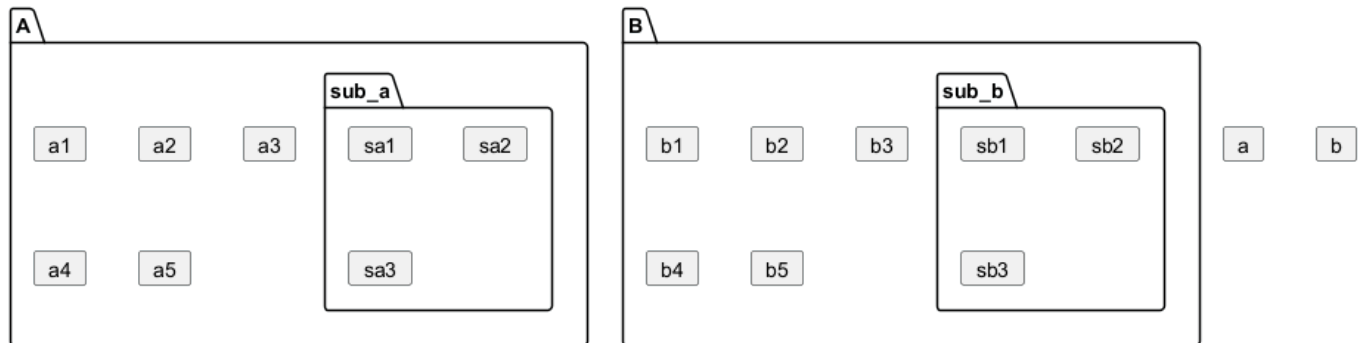
package B {
  card b1
  card b2
  card b3
  card b4
  card b5
  package sub_b {
    card sb1
    card sb2
  }
}
```



```

    card sb3
  }
}
@enduml

```



8.20.3 With Smetana (*internal layout engine*)

The main rule is the opposite: **Simple element first, then nested element.**

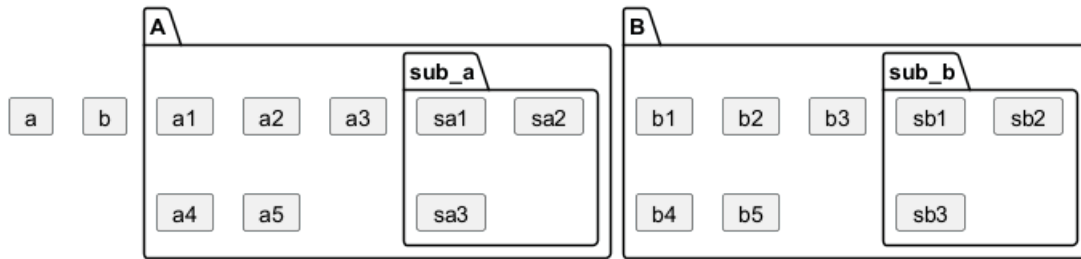
```

@startuml
!pragma layout smetana
card a
card b
package A {
  card a1
  card a2
  card a3
  card a4
  card a5
  package sub_a {
    card sa1
    card sa2
    card sa3
  }
}

package B {
  card b1
  card b2
  card b3
  card b4
  card b5
  package sub_b {
    card sb1
    card sb2
    card sb3
  }
}
@enduml

```



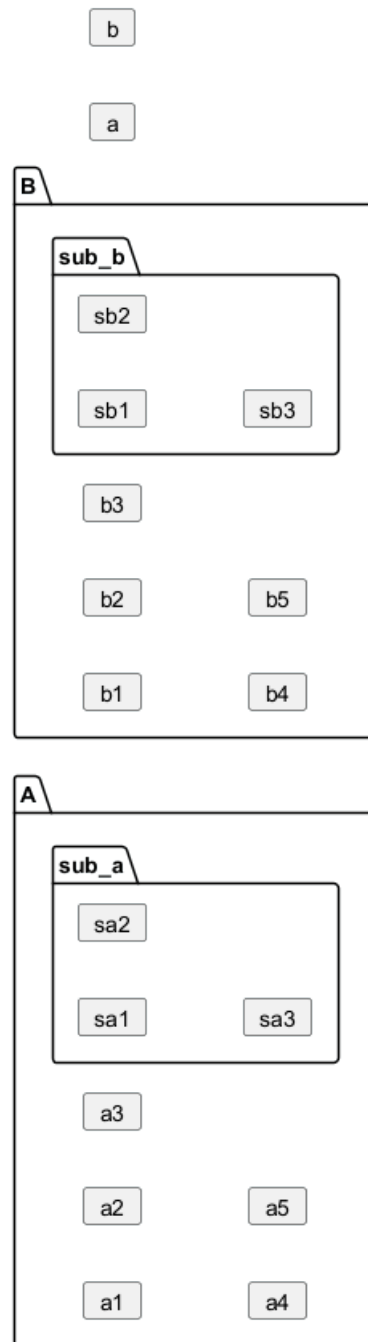


8.20.4 Left to right

8.20.5 With Graphviz (*layout engine by default*)

```
@startuml
left to right direction
card a
card b
package A {
  card a1
  card a2
  card a3
  card a4
  card a5
  package sub_a {
    card sa1
    card sa2
    card sa3
  }
}

package B {
  card b1
  card b2
  card b3
  card b4
  card b5
  package sub_b {
    card sb1
    card sb2
    card sb3
  }
}
@enduml
```



8.20.6 With Smetana (*internal layout engine*)

```

@startuml
!pragma layout smetana
left to right direction
card a
card b
package A {
  card a1
  card a2
  card a3
  card a4
  card a5
  package sub_a {
    card sa1
  }
}
  
```

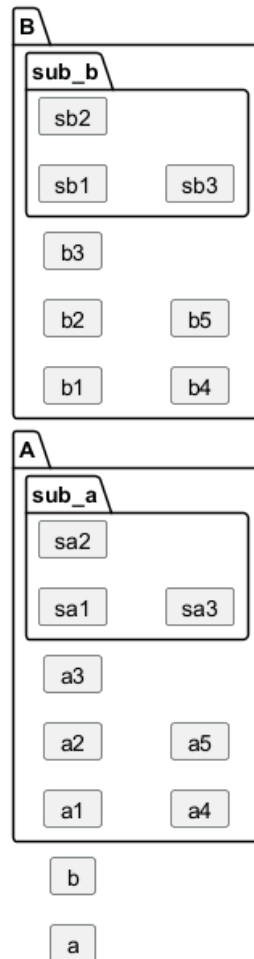


```

    card sa2
    card sa3
  }
}

package B {
  card b1
  card b2
  card b3
  card b4
  card b5
  package sub_b {
    card sb1
    card sb2
    card sb3
  }
}
}
@enduml

```



9 State Diagram

State diagrams provide a visual representation of the various states a system or an object can be in, as well as the transitions between those states. They are essential in modeling the dynamic behavior of systems, capturing how they respond to different events over time. State diagrams depict the system's life cycle, making it easier to understand, design, and optimize its behavior.

Using **PlantUML** to create state diagrams offers several advantages:

- **Text-Based Language:** Quickly define and visualize the states and transitions without the hassle of manual drawing.
- **Efficiency and Consistency:** Ensure streamlined diagram creation and easy version control.
- **Versatility:** Integrate with various documentation platforms and support multiple output formats.
- **Open-Source & Community Support:** Backed by a **strong community** that continuously contributes to its enhancements and offers invaluable resources.

9.1 Simple State

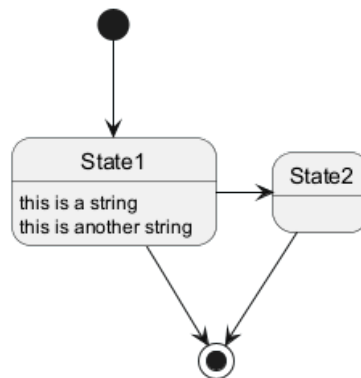
You can use `[*]` for the starting point and ending point of the state diagram.

Use `-->` for arrows.

```
@startuml
[*] --> State1
State1 --> [*]
State1 : this is a string
State1 : this is another string

State1 -> State2
State2 --> [*]

@enduml
```



9.2 Change state rendering

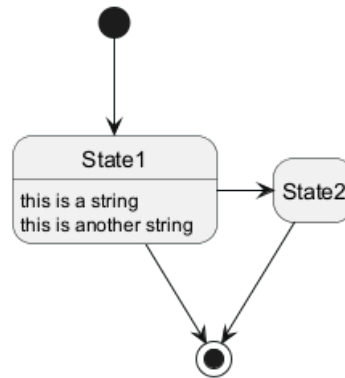
You can use `hide empty description` to render state as simple box.

```
@startuml
hide empty description
[*] --> State1
State1 --> [*]
State1 : this is a string
State1 : this is another string

State1 -> State2
State2 --> [*]
```



@enduml



9.3 Composite state

A state can also be composite. You have to define it using the **state** keywords and brackets.

9.3.1 Internal sub-state

```

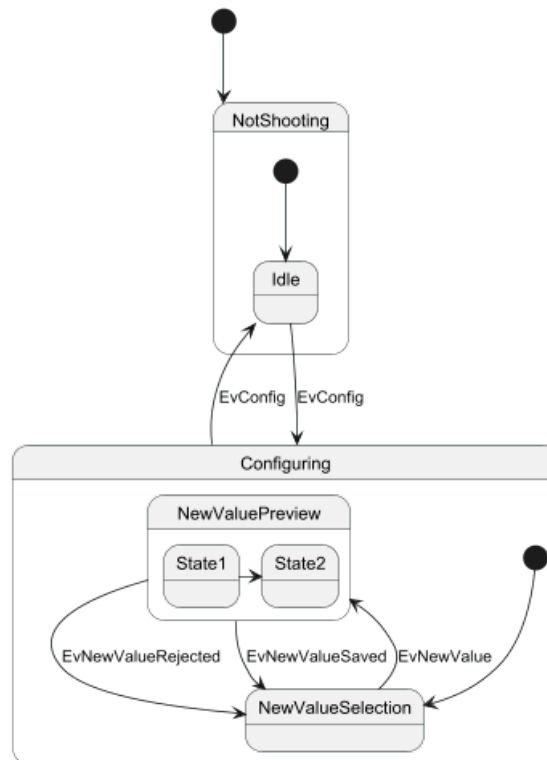
@startuml
scale 350 width
[*] --> NotShooting

state NotShooting {
    [*] --> Idle
    Idle --> Configuring : EvConfig
    Configuring --> Idle : EvConfig
}

state Configuring {
    [*] --> NewValueSelection
    NewValueSelection --> NewValuePreview : EvNewValue
    NewValuePreview --> NewValueSelection : EvNewValueRejected
    NewValuePreview --> NewValueSelection : EvNewValueSaved

    state NewValuePreview {
        State1 -> State2
    }
}

@enduml
  
```



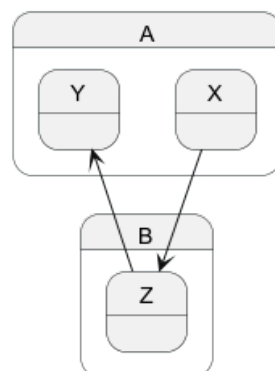
9.3.2 Sub-state to sub-state

```

@startuml
state A {
    state X {
    }
    state Y {
    }
}

state B {
    state Z {
    }
}

X --> Z
Z --> Y
@enduml
  
```



[Ref. QA-3300]

```

@startuml
  
```



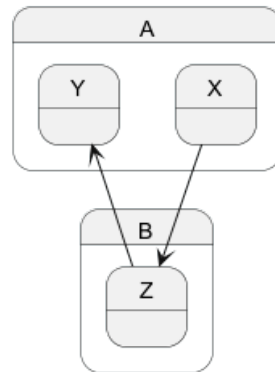
```

state A.X
state A.Y

state B.Z

X --> Z
Z --> Y
@enduml

```



9.4 Long name

You can also use the `state` keyword to use long description for states.

```

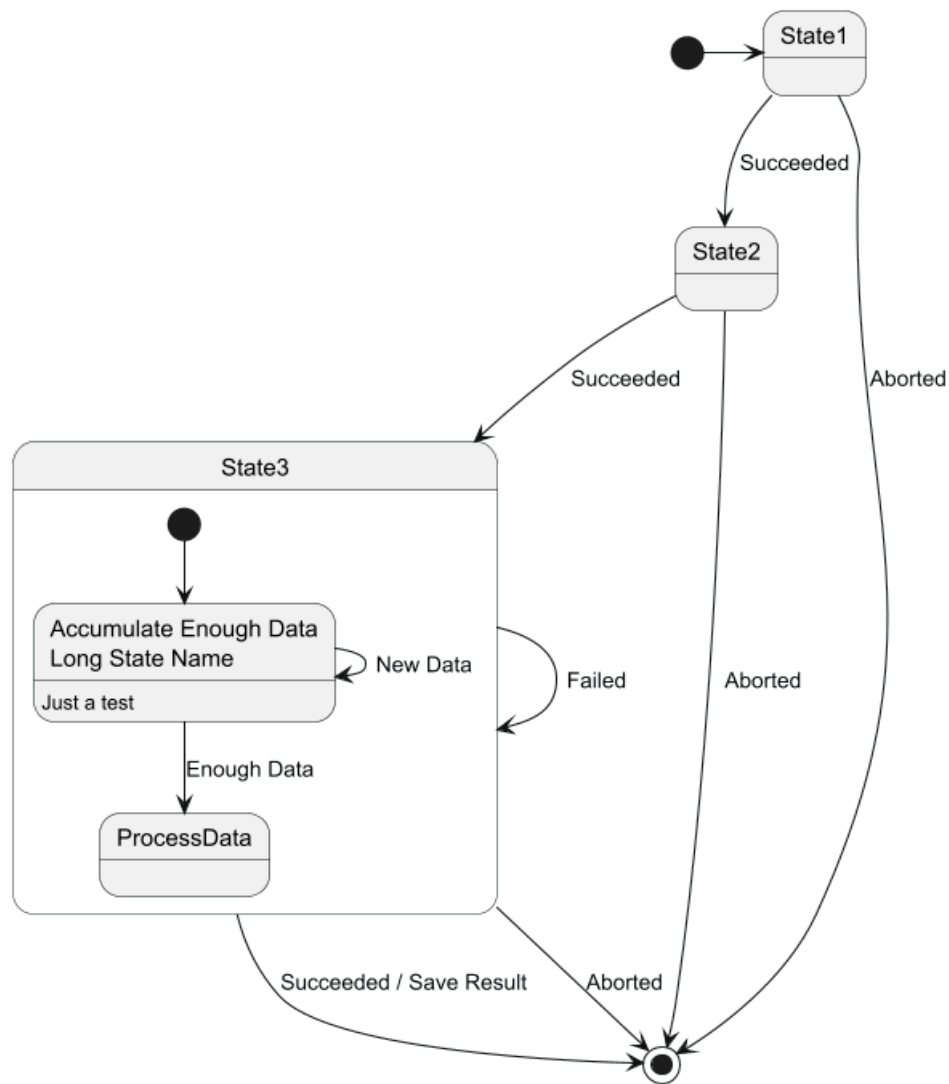
@startuml
scale 600 width

[*] -> State1
State1 --> State2 : Succeeded
State1 --> [*] : Aborted
State2 --> State3 : Succeeded
State2 --> [*] : Aborted
state State3 {
    state "Accumulate Enough Data\nLong State Name" as long1
    long1 : Just a test
    [*] --> long1
    long1 --> long1 : New Data
    long1 --> ProcessData : Enough Data
}
State3 --> State3 : Failed
State3 --> [*] : Succeeded / Save Result
State3 --> [*] : Aborted

@enduml

```





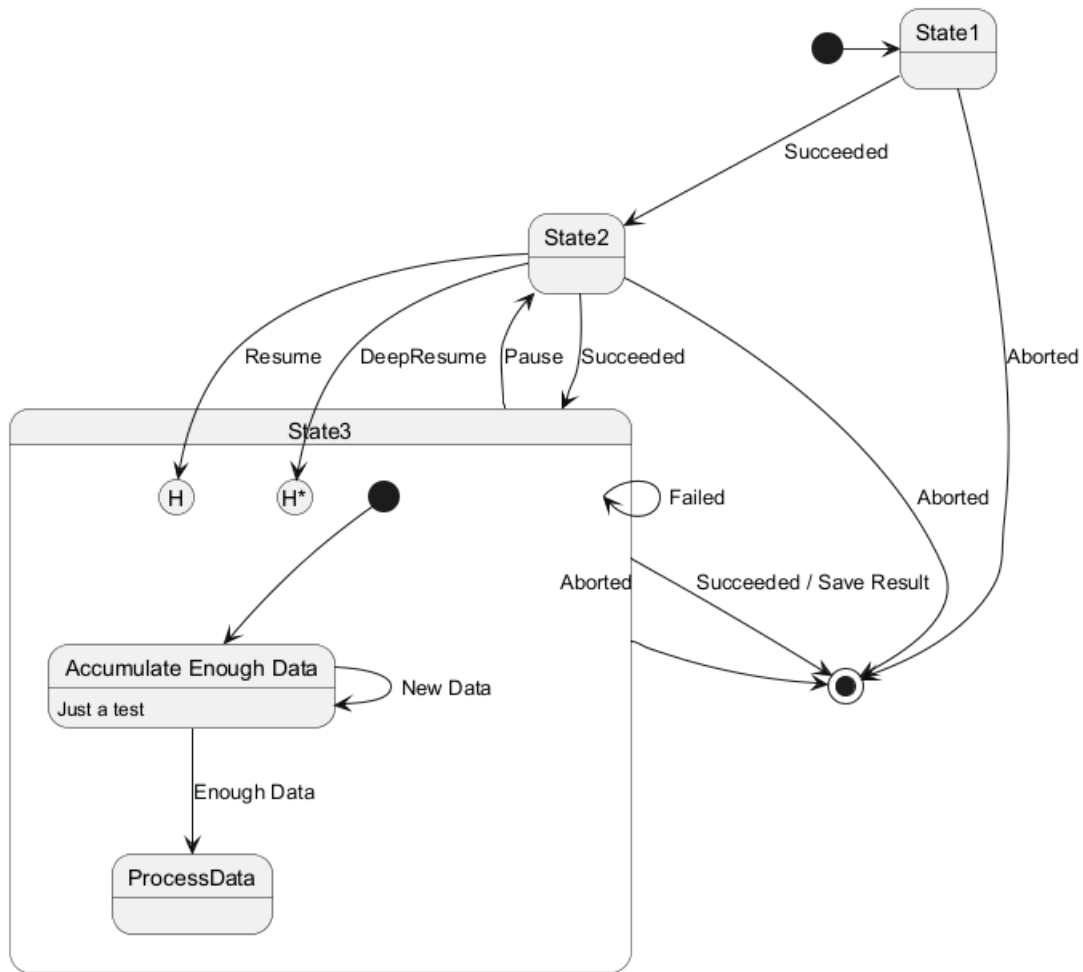
9.5 History $[[H], [H^*]]$

You can use $[H]$ for the history and $[H^*]$ for the deep history of a substate.

```

@startuml
[*] -> State1
State1 --> State2 : Succeeded
State1 --> [*] : Aborted
State2 --> State3 : Succeeded
State2 --> [*] : Aborted
state State3 {
    state "Accumulate Enough Data" as long1
    long1 : Just a test
    [*] --> long1
    long1 --> long1 : New Data
    long1 --> ProcessData : Enough Data
    State2 --> [H]: Resume
}
State3 --> State2 : Pause
State2 --> State3[H*]: DeepResume
State3 --> State3 : Failed
State3 --> [*] : Succeeded / Save Result
State3 --> [*] : Aborted
@enduml
  
```





9.6 Fork [fork, join]

You can also fork and join using the `<<fork>>` and `<<join>>` stereotypes.

```

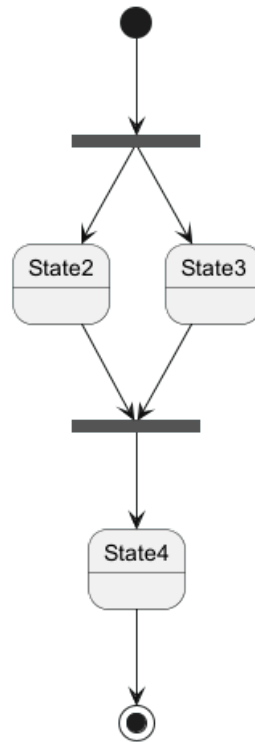
@startuml

state fork_state <<fork>>
[*] --> fork_state
fork_state --> State2
fork_state --> State3

state join_state <<join>>
State2 --> join_state
State3 --> join_state
join_state --> State4
State4 --> [*]

@enduml

```



9.7 Concurrent state [–, ||]

You can define concurrent state into a composite state using either -- or || symbol as separator.

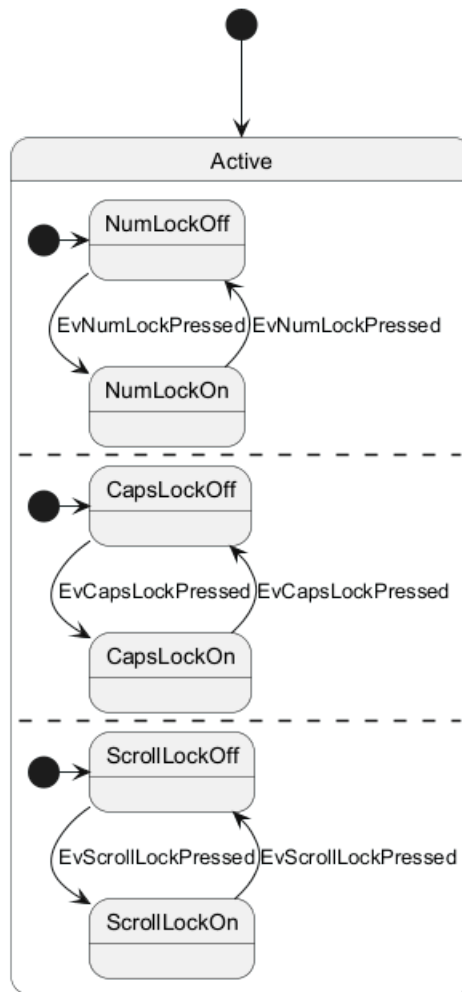
9.7.1 Horizontal separator --

```

@startuml
[*] --> Active

state Active {
    [*] -> NumLockOff
    NumLockOff --> NumLockOn : EvNumLockPressed
    NumLockOn --> NumLockOff : EvNumLockPressed
    --
    [*] -> CapsLockOff
    CapsLockOff --> CapsLockOn : EvCapsLockPressed
    CapsLockOn --> CapsLockOff : EvCapsLockPressed
    --
    [*] -> ScrollLockOff
    ScrollLockOff --> ScrollLockOn : EvScrollLockPressed
    ScrollLockOn --> ScrollLockOff : EvScrollLockPressed
}

@enduml
  
```



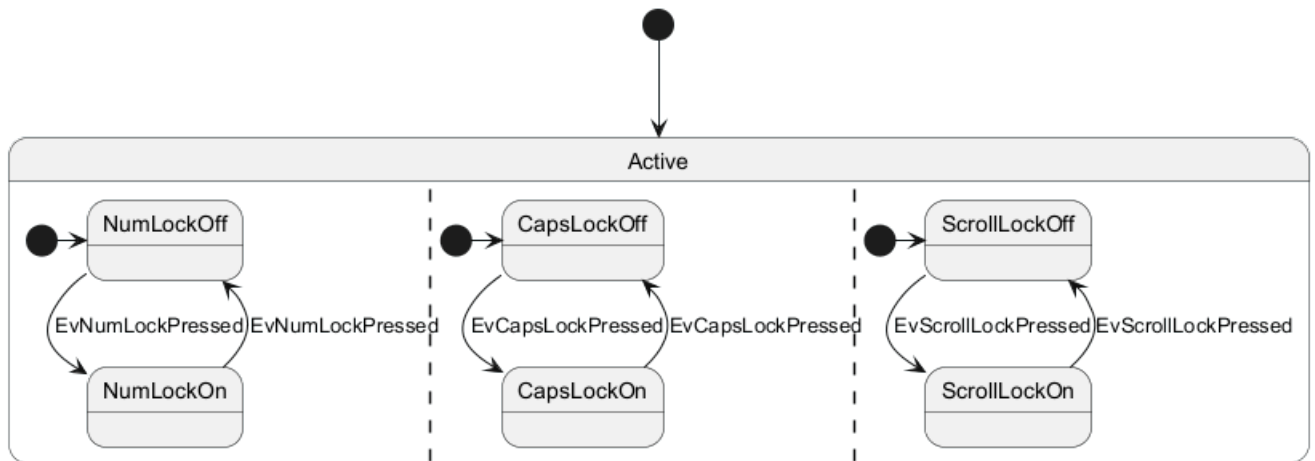
9.7.2 Vertical separator ||

```

@startuml
[*] --> Active

state Active {
    [*] -> NumLockOff
    NumLockOff --> NumLockOn : EvNumLockPressed
    NumLockOn --> NumLockOff : EvNumLockPressed
    ||
    [*] -> CapsLockOff
    CapsLockOff --> CapsLockOn : EvCapsLockPressed
    CapsLockOn --> CapsLockOff : EvCapsLockPressed
    ||
    [*] -> ScrollLockOff
    ScrollLockOff --> ScrollLockOn : EvScrollLockPressed
    ScrollLockOn --> ScrollLockOff : EvScrollLockPressed
}

@enduml
  
```



[Ref. QA-3086]

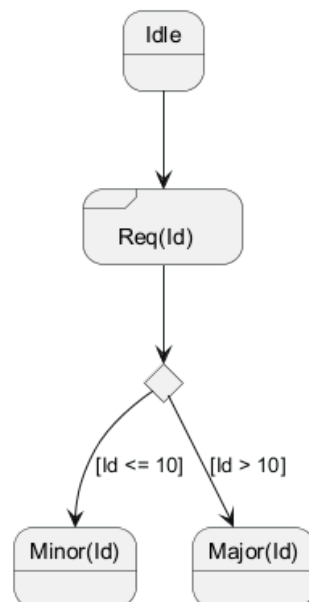
9.8 Conditional [choice]

The stereotype <<choice>> can be used to use conditional state.

```
@startuml
state "Req(Id)" as ReqId <<sdlreceive>>
state "Minor(Id)" as MinorId
state "Major(Id)" as MajorId

state c <<choice>>

Idle --> ReqId
ReqId --> c
c --> MinorId : [Id <= 10]
c --> MajorId : [Id > 10]
@enduml
```



9.9 Stereotypes full example [start, choice, fork, join, end, history, history*]

9.9.1 Start, choice, fork, join, end

```
@startuml
```




```

state start1 <<start>>
state choice1 <<choice>>
state fork1 <<fork>>
state join2 <<join>>
state end3 <<end>>

[*] --> choice1 : from start\nto choice
start1 --> choice1 : from start stereo\nto choice

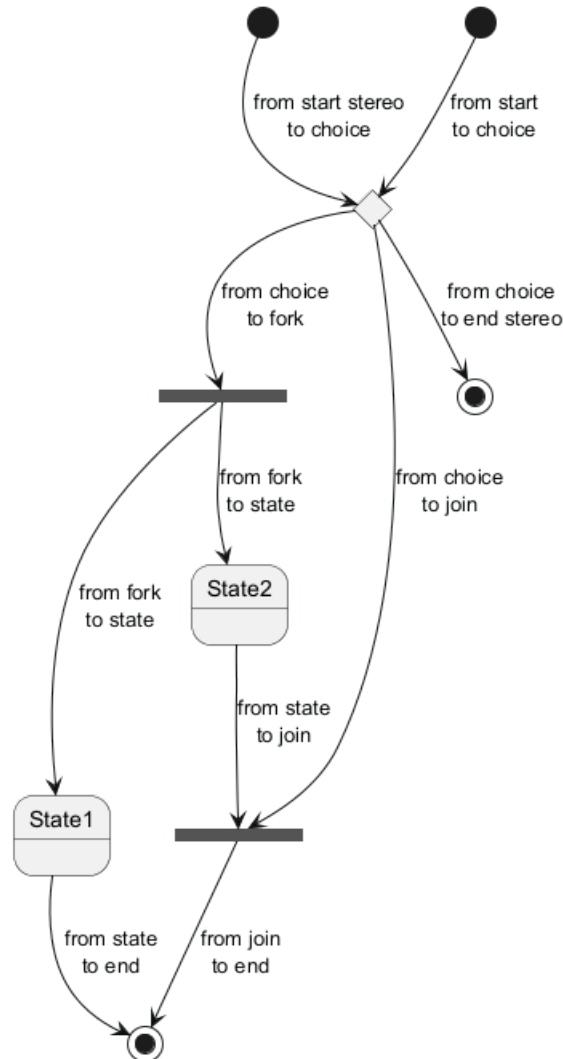
choice1 --> fork1 : from choice\nto fork
choice1 --> join2 : from choice\nto join
choice1 --> end3 : from choice\nto end stereo

fork1 ---> State1 : from fork\nto state
fork1 --> State2 : from fork\nto state

State2 --> join2 : from state\nto join
State1 --> [*] : from state\nto end

join2 --> [*] : from join\nto end
@enduml

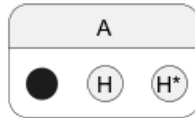
```



[Ref. QA-404, QA-1159 and GH-887]

9.9.2 History, history*

```
@startuml
state A {
    state s1 as "Start 1" <<start>>
    state s2 as "H 2" <<history>>
    state s3 as "H 3" <<history*>>
}
@enduml
```



[Ref. QA-16824]

9.9.3 Minimal example with all stereotypes

```
@startuml
state start1 <<start>>
state choice1 <<choice>>
state fork1 <<fork>>
state join2 <<join>>
state end3 <<end>>
state sdlreceive <<sdlreceive>>
state history <<history>>
state history2 <<history*>>
@enduml
```



[Ref. QA-19174]

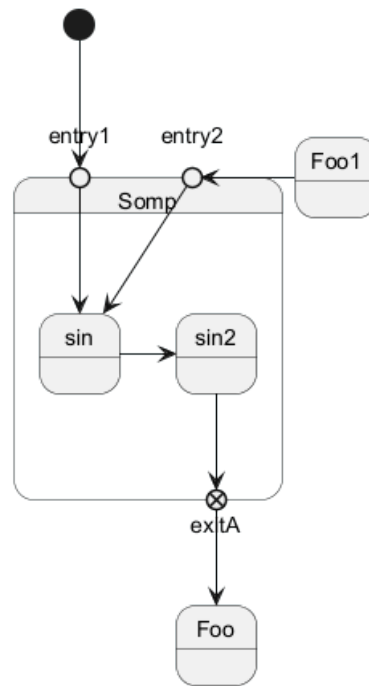
9.10 Point [entryPoint, exitPoint]

You can add **point** with <<entryPoint>> and <<exitPoint>> stereotypes:

```
@startuml
state Somp {
    state entry1 <<entryPoint>>
    state entry2 <<entryPoint>>
    state sin
    entry1 --> sin
    entry2 -> sin
    sin -> sin2
    sin2 --> exitA <<exitPoint>>
}

[*] --> entry1
exitA --> Foo
Foo1 -> entry2
@enduml
```





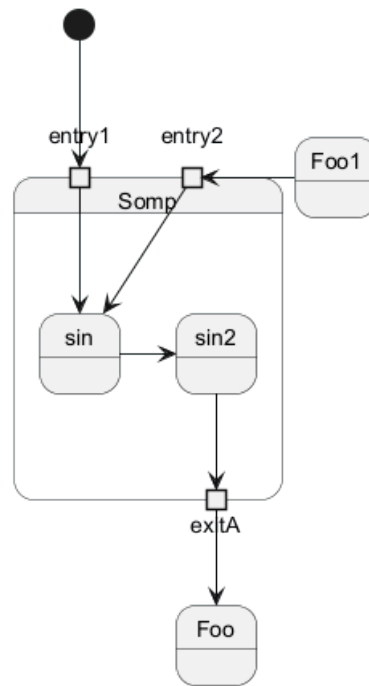
9.11 Pin [inputPin, outputPin]

You can add **pin** with `<<inputPin>>` and `<<outputPin>>` stereotypes:

```

@startuml
state Somp {
    state entry1 <<inputPin>>
    state entry2 <<inputPin>>
    state sin
    entry1 --> sin
    entry2 -> sin2
    sin -> sin2
    sin2 --> exitA <<outputPin>>
}

[*] --> entry1
exitA --> Foo
Foo1 -> entry2
@enduml
  
```



[Ref. QA-4309]

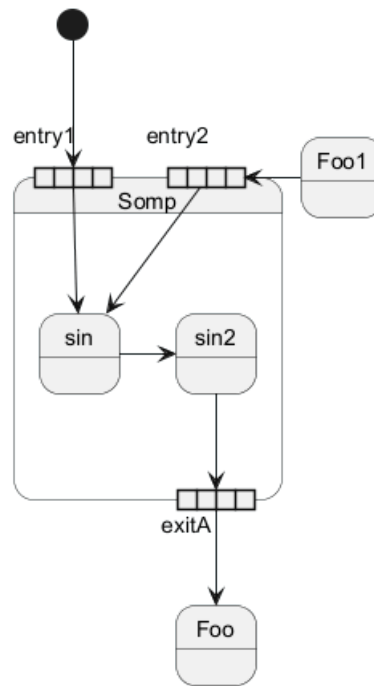
9.12 Expansion [expansionInput, expansionOutput]

You can add **expansion** with `<<expansionInput>>` and `<<expansionOutput>>` stereotypes:

```

@startuml
state Somp {
    state entry1 <<expansionInput>>
    state entry2 <<expansionInput>>
    state sin
    state sin2
    entry1 --> sin
    entry2 --> sin
    sin --> sin2
    sin2 --> exitA <<expansionOutput>>
}

[*] --> entry1
exitA --> Foo
Foo1 --> entry2
@enduml
  
```



[Ref. QA-4309]

9.13 Arrow direction

You can use `->` for horizontal arrows. It is possible to force arrow's direction using the following syntax:

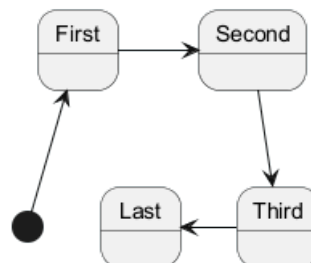
- `-down->` or `-->`
- `-right->` or `->` (*default arrow*)
- `-left->`
- `-up->`

@startuml

```

[*] -up-> First
First -right-> Second
Second --> Third
Third -left-> Last
  
```

@enduml



You can shorten the arrow definition by using only the first character of the direction (for example, `-d-` instead of `-down-`) or the two first characters (`-do-`).

Please note that you should not abuse this functionality : *Graphviz* gives usually good results without tweaking.

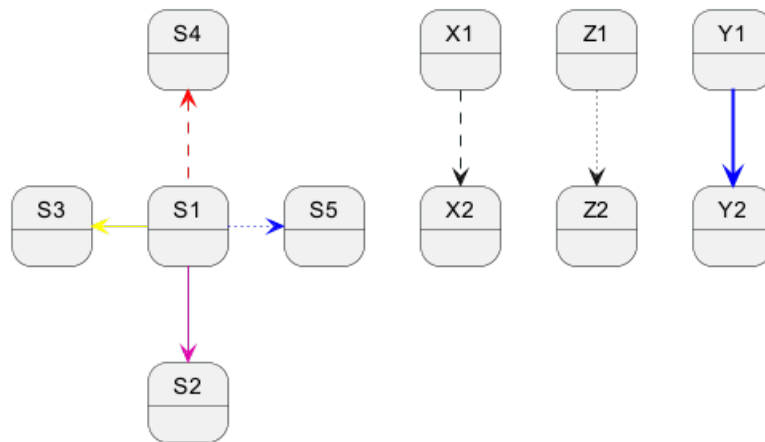


9.14 Change line color and style

You can change line color and/or line style.

```
@startuml
State S1
State S2
S1 -[#DD00AA]-> S2
S1 -left[#yellow]-> S3
S1 -up[#red,dashed]-> S4
S1 -right[dotted,#blue]-> S5

X1 -[dashed]-> X2
Z1 -[dotted]-> Z2
Y1 -[#blue,bold]-> Y2
@enduml
```



[Ref. Incubation: Change line color in state diagrams]

9.15 Note

You can also define notes using `note left of`, `note right of`, `note top of`, `note bottom of` keywords.

You can also define notes on several lines.

```
@startuml

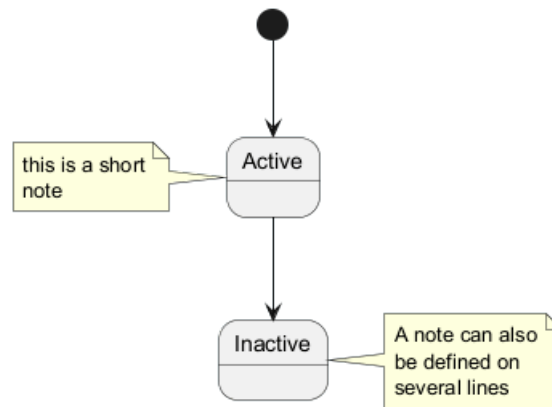
[*] --> Active
Active --> Inactive

note left of Active : this is a short\nnote

note right of Inactive
  A note can also
  be defined on
  several lines
end note

@enduml
```





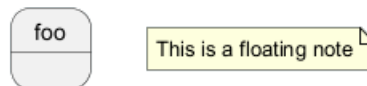
You can also have floating notes.

```

@startuml

state foo
note "This is a floating note" as N1

@enduml
  
```

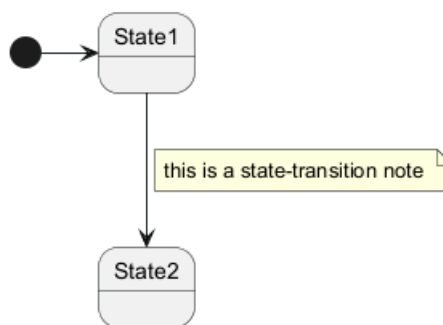


9.16 Note on link

You can put notes on state-transition or link, with `note on link` keyword.

```

@startuml
[*] -> State1
State1 --> State2
note on link
    this is a state-transition note
end note
@enduml
  
```



9.17 More in notes

You can put notes on composite states.

```

@startuml

[*] --> NotShooting

state "Not Shooting State" as NotShooting {
    state "Idle mode" as Idle
    state "Configuring mode" as Configuring
  }
  
```



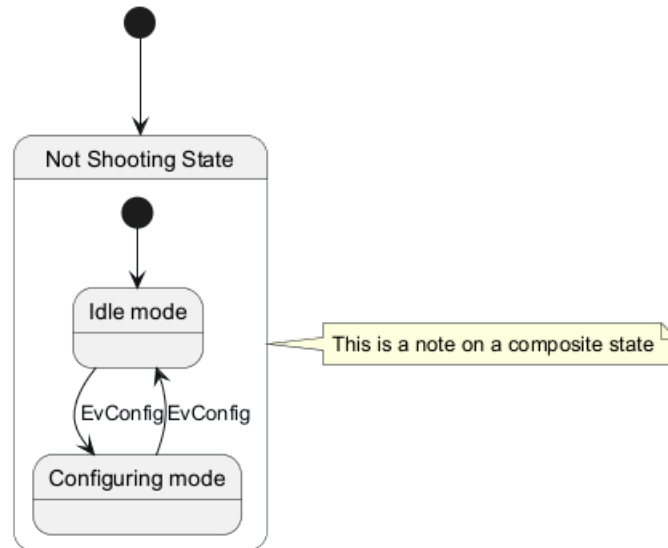
```

[*] --> Idle
Idle --> Configuring : EvConfig
Configuring --> Idle : EvConfig
}

note right of NotShooting : This is a note on a composite state

@enduml

```

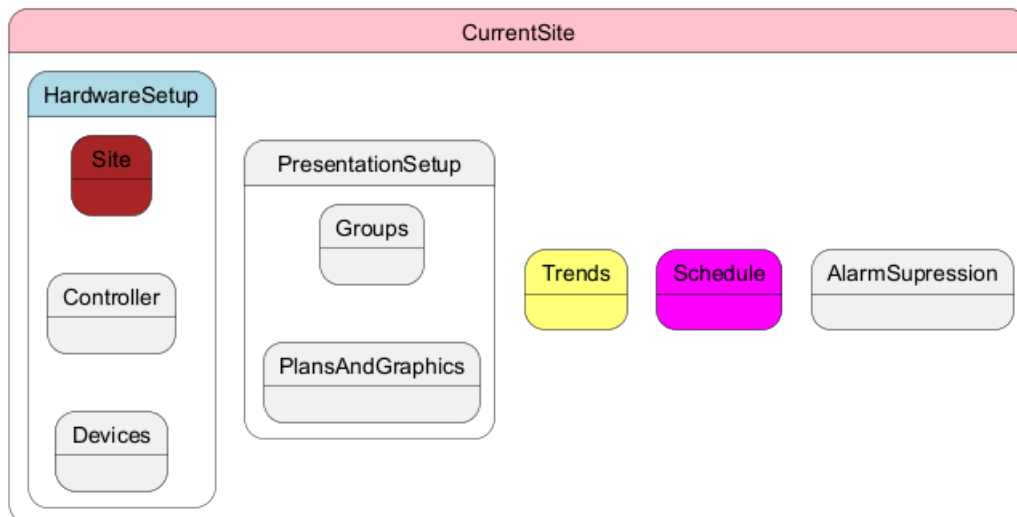


9.18 Inline color

```

@startuml
state CurrentSite #pink {
    state HardwareSetup #lightblue {
        state Site #brown
        Site -[hidden]-> Controller
        Controller -[hidden]-> Devices
    }
    state PresentationSetup{
        Groups -[hidden]-> PlansAndGraphics
    }
    state Trends #FFFF77
    state Schedule #magenta
    state AlarmSupression
}
@enduml

```

[Ref. QA-1812]

9.19 Skinparam

You can use the skinparam command to change colors and fonts for the drawing.

You can use this command :

- In the diagram definition, like any other commands,
- In an included file,
- In a configuration file, provided in the command line or the Ant task.

You can define specific color and fonts for stereotyped states.

```

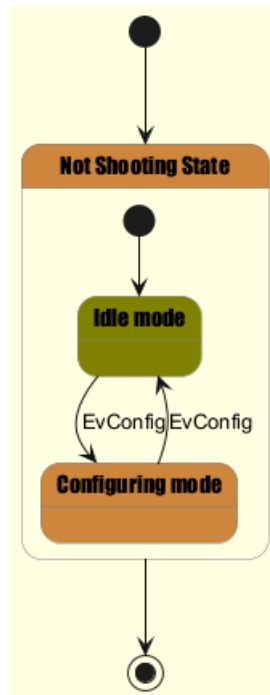
@startuml
skinparam backgroundColor LightYellow
skinparam state {
  StartColor MediumBlue
  EndColor Red
  BackgroundColor Peru
  BackgroundColor<<Warning>> Olive
  BorderColor Gray
  FontName Impact
}

[*] --> NotShooting

state "Not Shooting State" as NotShooting {
  state "Idle mode" as Idle <<Warning>>
  state "Configuring mode" as Configuring
  [*] --> Idle
  Idle --> Configuring : EvConfig
  Configuring --> Idle : EvConfig
}

NotShooting --> [*]
@enduml
  
```





9.19.1 Test of all specific skinparam to State Diagrams

```

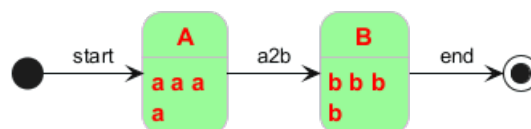
@startuml
skinparam State {
  AttributeFontColor blue
  AttributeFontName serif
  AttributeFontSize 9
  AttributeFontStyle italic
  BackgroundColor palegreen
  BorderColor violet
  EndColor gold
  FontColor red
  FontName Sanserif
  FontSize 15
  FontStyle bold
  StartColor silver
}
  
```

```

state A : a a a\na
state B : b b b\nb
  
```

```

[*] -> A : start
A -> B : a2b
B -> [*] : end
@enduml
  
```



9.20 Changing style

You can change style.

```

@startuml
  
```



```

<style>
stateDiagram {
  BackgroundColor Peru
  'LineColor Gray
  FontName Impact
  FontColor Red
  arrow {
    FontSize 13
    LineColor Blue
  }
}
</style>

```

```

[*] --> NotShooting

```

```

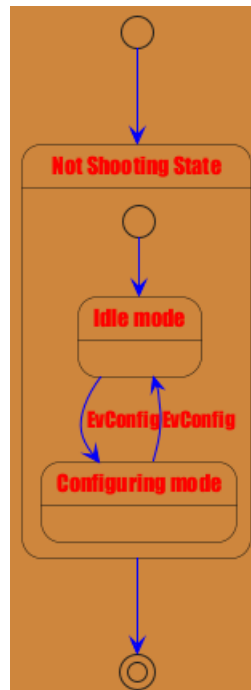
state "Not Shooting State" as NotShooting {
  state "Idle mode" as Idle <<Warning>>
  state "Configuring mode" as Configuring
  [*] --> Idle
  Idle --> Configuring : EvConfig
  Configuring --> Idle : EvConfig
}

```

```

NotShooting --> [*]
@enduml

```



```

@startuml
<style>
  diamond {
    BackgroundColor #palegreen
    LineColor #green
    LineThickness 2.5
  }
</style>
state state1

```

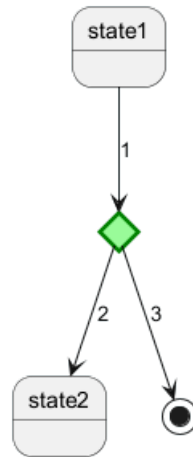


```

state state2
state choice1 <<choice>>
state end3    <<end>>

state1 --> choice1 : 1
choice1 --> state2 : 2
choice1 --> end3   : 3
@enduml

```



[Ref. GH-880]

9.21 Change state color and style (inline style)

You can change the color or style of individual state using the following notation:

- #color ##[style]color

With background color first (#color), then line style and line color (##[style]color).

```

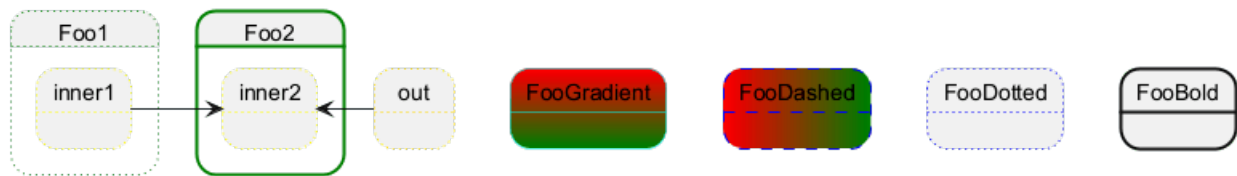
@startuml
state FooGradient #red-green ##00FFFF
state FooDashed #red|green ##[dashed]blue {
}
state FooDotted ##[dotted]blue {
}
state FooBold ##[bold] {
}
state Foo1 ##[dotted]green {
state inner1 ##[dotted]yellow
}

state out ##[dotted]gold

state Foo2 ##[bold]green {
state inner2 ##[dotted]yellow
}
inner1 -> inner2
out -> inner2
@enduml

```





[Ref. QA-1487]

- #color;line:color;line.[bold|dashed|dotted];text:color

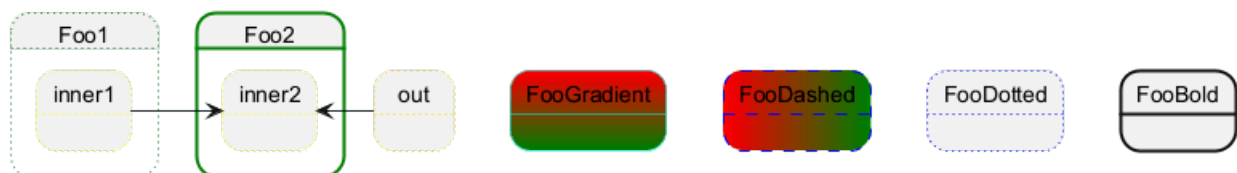
TODO: FIXME text:color seems not to be taken into account **TODO: FIXME**

```

@startuml
@startuml
state FooGradient #red-green;line:00FFFF
state FooDashed #red|green;line.dashed;line:blue {
}
state FooDotted #line.dotted;line:blue {
}
state FooBold #line.bold {
}
state Foo1 #line.dotted;line:green {
state inner1 #line.dotted;line:yellow
}

state out #line.dotted;line:gold

state Foo2 #line.bold;line:green {
state inner2 #line.dotted;line:yellow
}
inner1 -> inner2
out -> inner2
@enduml
@enduml
  
```



```

@startuml
state s1 : s1 description
state s2 #pink;line:red;line.bold;text:red : s2 description
state s3 #palegreen;line:green;line.dashed;text:green : s3 description
state s4 #aliceblue;line:blue;line.dotted;text:blue : s4 description
@enduml
  
```



[Adapted from QA-3770]

9.22 Alias

With State you can use alias, like:

```

@startuml
state alias1
  
```



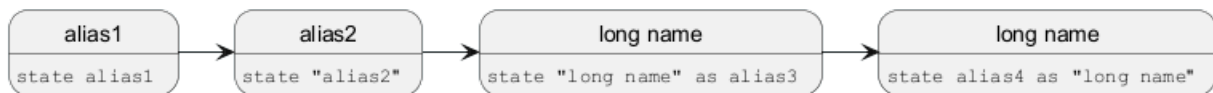
```

state "alias2"
state "long name" as alias3
state alias4 as "long name"

alias1 : "state alias1"
alias2 : "state "alias2""
alias3 : "state "long name" as alias3"
alias4 : "state alias4 as "long name""

alias1 -> alias2
alias2 -> alias3
alias3 -> alias4
@enduml

```



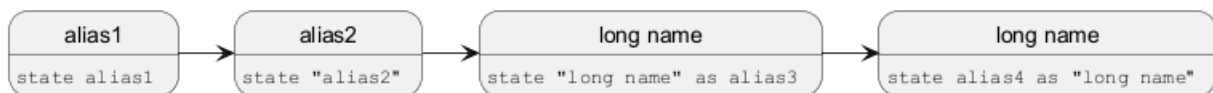
or:

```

@startuml
state alias1 : "state alias1"
state "alias2" : "state "alias2""
state "long name" as alias3 : "state "long name" as alias3"
state alias4 as "long name" : "state alias4 as "long name""

alias1 -> alias2
alias2 -> alias3
alias3 -> alias4
@enduml

```



[Ref. QA-1748, QA-14560]

9.23 Display JSON Data on State diagram

9.23.1 Simple example

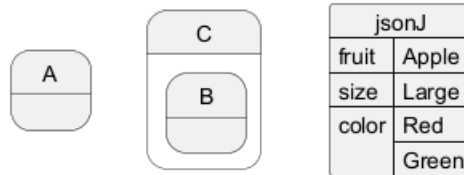
```

@startuml
state "A" as stateA
state "C" as stateC {
    state B
}

json jsonJ {
    "fruit": "Apple",
    "size": "Large",
    "color": ["Red", "Green"]
}
@enduml

```





[Ref. QA-17275]

For another example, see on JSON page.

9.24 State description

You can add description to a state or to a composite state.

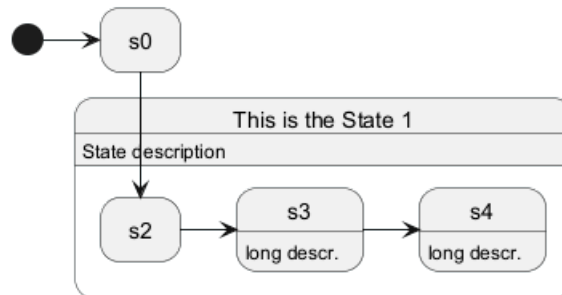
```
@startuml
hide empty description

state s0

state "This is the State 1" as s1 {
  s1: State description
  state s2
  state s3: long descr.
  state s4
  s4: long descr.
}

[*] -> s0
s0 --> s2

s2 -> s3
s3 -> s4
@enduml
```



[Ref. QA-16719]

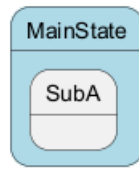
9.25 Style for Nested State Body

```
@startuml
<style>
.foo {
  state, stateBody {
    BackGroundColor lightblue;
  }
}
</style>

state MainState <<foo>> {
  state SubA
```



```
}  
@enduml
```



[Ref. QA-16774]